Issues

Issues — Open Workable Items

Revision: 87 **Last modified:** 2026-08-27T01:44:23Z **Ticket prefix:** `BOB` (operator-mandated, 2026-06-06) **Scope:** Open/active items only. Closed items migrate to `Fixed.md`.

> Tracking: this file + `Issues_Summary.md` are authoritative for open work. The SQLite single-source-of-truth + `docs_chain` engine (BOB-010) is complete.

BOB-008 — RuTracker automated login blocked by CAPTCHA

Status: Operator-blocked **Type:** Bug

OPERATOR DECISION (2026-08-26, §11.4.66 interactive clarification): COMPLETE THE CAPTCHA FLOW ONCE

Session establishment path is DECIDED: the operator drives the interactive CAPTCHA flow at `/api/v1/auth/rutracker/captcha` then `/login` once, and the proxy stores the resulting `bb_session`. The cookie-paste path and the cookies-file autoload path were both offered and NOT chosen. THE BLOCK NARROWS BUT DOES NOT LIFT: the operator must physically drive the flow — no agent can solve a CAPTCHA (§11.4.52 honest operator_attended boundary). AGENT PREP OWED BEFORE THE HAND-OFF, so the operator's single attempt succeeds rather than discovering a broken endpoint mid-flow: verify both endpoints are reachable and correctly wired end-to-end, verify the session is actually PERSISTED after a successful post (not merely accepted), and verify a stored session is actually USED by a subsequent

search. Honest limitation to state up front: a stored bb_session expires, so this path requires repeating — that cost was accepted with the decision.

This answer is recorded as consumer DATA per §11.4.35 — it is the operator's stated choice, not an agent inference, and supersedes any prior agent-chosen default on this question. Options not chosen are named above so a future reader does not re-litigate a settled call (§11.4.112(5) bounded-verdict discipline applied to decisions).

— prior item text follows —

RuTracker automated login blocked by CAPTCHA

BOB-065 — Lava P2: Egress diagnosis and VPN-host SOCKS routing (containers pkg/egress)

Status: Queued **Type:** Task **Severity:** High

[Backfill from RD2-15/GA-05, audit doc 2026-08-08] Lava-porting finding P2 (Egress decision + VPN-host routing, Lava PLAYBOOK sections 0 and 4). Problem Boba-Base shares: on a datacenter host, trackers are network-blocked (DNS-fail/TLS-MITM, not Cloudflare so FlareSolverr cannot fix). Affects Jackett indexer fetches + merge_service/download-proxy + plugin engines. Diagnosis (port the script): curl https://api.ipify.org (host IP) + curl -o /dev/null -w %{http_code} https:/// direct vs via a VPN-host SOCKS proxy. Different egress IP + 200 via proxy confirms. Fix: route outbound through a VPN-connected host (the nezha pattern). SOCKS tunnel ssh -D 127.0.0.1:1080 -N (use –socks5-hostname for remote DNS); point Jackett + download-proxy + qBitTorrent-go at it (P3). For browser-cookie harvest, run the harvester ON the VPN host. Port: containers submodule pkg/egress (tunnel up/verify) + scripts/egress-via-vpn.sh glue; reuse Boba-Base existing ensure-macos-tunnel.sh style. TDD: assert the via-proxy egress IP != direct host IP AND a known-blocked tracker returns 200 via proxy. Source: docs/PORTING-FROM-LAVA.md. Per audit RD2-15 [P0]: Create tracked workable items (BOB-064..067) for the four Lava-porting findings, citing implementing commits as evidence, closed as Implemented.

BOB-066 — Lava P3: BOBA_UPSTREAM_PROXY in download-proxy + qBitTorrent-go + Jackett + compose env-forward

Status: In progress **Type:** Task **Severity:** High

[Backfill from RD2-15/GA-05, audit doc 2026-08-08] Lava-porting finding P3 (Configurable outbound proxy in the services, Lava PLAYBOOK section 3). download-proxy (Python): httpx/requests honor HTTP_PROXY/HTTPS_PROXY/ALL_PROXY/NO_PROXY env natively — add an explicit BOBA_UPSTREAM_PROXY config that sets these for tracker-bound clients, with loopback bypass (NO_PROXY=127.0.0.1,localhost,jackett). qBitTorrent-go: set http.Transport.Proxy (socks5 native, remote DNS) from a BOBA_UPSTREAM_PROXY env — mirror Lava internal/httpx/proxy.go. Jackett: has a built-in proxy setting (configure via its API/ServerConfig). Deploy gotcha (port): the env must be FORWARDED into the containers (docker-compose.yml env / the boba-ctl deploy) — Lava bug was a missing allow-list entry. Verify on distroless via podman inspect, not exec printenv. TDD: a test with a local proxy asserts the service tracker request traverses it; falsifiability: disable the wiring so the test fails. Source: docs/PORTING-FROM-LAVA.md. Per audit RD2-15 [P0]: Create tracked workable items (BOB-064..067) for the four Lava-porting findings, citing implementing commits as evidence, closed as Implemented.

BOB-068 — RD2-00: unattributed, unreviewed Auto-commit mechanism pushing to main

Status: In progress **Type:** Bug **Severity:** High

RD2-00: unattributed, unreviewed Auto-commit mechanism pushing to main

[BOB-136 adoption audit 2026-08-21 -> In progress] Work landed but acceptance NOT met. a7e55f9 completed the Phase-1 investigation (no daemon; shared-checkout race confirmed) and 0972cbc added commit-push-all.sh –scope + challenge, described in its own message as an ‘Interim tooling remedy for BOB-068 while the §11.4.179 architectural

fix (task #67 proposal) is designed'. 35e53db is a DRAFT PROPOSAL only. The §11.4.179 clone-isolation fix is not landed, so the item stays open.

BOB-069 — RD2-40: §11.4.238 QA-discovery-channel ledger — ongoing coverage-escape backfill

Status: In progress **Type:** Task **Severity:** High

RD2-40: §11.4.238 QA-discovery-channel ledger — ongoing coverage-escape backfill

[BOB-136 adoption audit 2026-08-21 -> In progress] a4173c8 stood up the ledger + 4 followups; 709b06c closed 2 Important review findings (CODEGRAPH-1.5.0-GITIGNORE entry, BOB-108 filed). The BOB-009/010 evidence_path gap this item names as remaining open was closed under BOB-086 (f78f383). NOT terminal by the item's own text: it is explicitly scoped 'P1-ongoing', with the full retroactive audit of the ~50 GA-NN/RD2-NN findings deliberately NOT attempted.

BOB-074 — RD2-07: DDoS-class testing fully absent from the mandated test-type matrix

Status: In progress **Type:** Task **Severity:** Medium

RD2-07: DDoS-class testing fully absent from the mandated test-type matrix

[BOB-136 adoption audit 2026-08-21 -> In progress] ae2b5cb authored challenges/scripts/ddos_resilience_challenge.sh (424 lines, 3 public surfaces) + docs/testing/{ddos_resilience,test_type_matrix}.md, so DDoS-class testing is no longer 'fully absent'. But of the three coverages this item's Fix line enumerates, a grep of the shipped challenge finds flood=2 hits, malformed=0, exhaust=0 — malformed-request-flood and resource-exhaustion-under-attack are not covered. 258b7db filed the 6 followups (BOB-109..114). Partial, not complete.

BOB-077 — RD2-10: Identify second host running the Auto-commit rsync/sync mechanism (OPERATOR-DECISION)

Status: Queued **Type:** Task **Severity:** High

OPERATOR DECISION (2026-08-26, §11.4.66 interactive clarification): UNKNOWN — INSTRUMENT THE NEXT OCCURRENCE

The operator does not currently know which host produces the +0500 Auto-commit fast-forwards, so the three candidate answers (second Claude session / intentional rsync job / stale job) all remain open. DECIDED ACTION: stop hunting a host this session cannot reach (§11.4.6 — remote state is not knowable without access) and instead INSTRUMENT the mechanism so the next occurrence identifies itself. Forensic capture to add: committer identity, hostname, timezone offset, push timing, and the git remote used, recorded at Auto-commit time into a tracked forensic log. ACCEPTANCE: the next Auto-commit event yields a captured record naming its origin host — at which point this item resolves to one of the three original branches with evidence rather than a guess. This is the §11.4.101 reversible-safe move: it costs little, blocks nothing, and converts a recurring mystery into a self-identifying event.

This answer is recorded as consumer DATA per §11.4.35 — it is the operator's stated choice, not an agent inference, and supersedes any prior agent-chosen default on this question. Options not chosen are named above so a future reader does not re-litigate a settled call (§11.4.112(5) bounded-verdict discipline applied to decisions).

— prior item text follows —

RD2-10: Identify second host running the Auto-commit rsync/sync mechanism (OPERATOR-DECISION)

BOB-078 — RD2-11: Once identified, wire Auto-commit mechanism through §11.4.234 dedicated commit/push script OR retire it

Status: Queued **Type:** Task **Severity:** High

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-11, P0] Once identified (RD2-10): either wire it through a real §11.4.234 dedicated commit/push script (with the hook-validation stages this project already has via .claude/settings.json PreToolUse guard, extended to a real pre-commit content check) or shut it down if it serves no purpose. Priority: P0. Blocked on RD2-10.

BOB-080 — RD2-13: Retroactive Fable-xhigh code review of the substantive Auto-commit diffs

Status: Queued **Type:** Task **Severity:** High

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-13, P1] Retroactively run the mandatory independent Fable-xhigh code review (§11.4.125/§11.4.142/§11.4.209) against the substantive diffs the Auto-commit mechanism introduced (start.sh three new functions especially, since they have zero test coverage — see Root Cause 4 / RD2-24) — even though the changes already landed, a post-hoc review closes the governance gap and will surface RD2-24 (GA-27 missing tests) if not already caught. Priority: P1.

BOB-082 — RD2-15: Create BOB-064..067 workable items for the four Lava-porting findings (closes GA-05)

Status: Queued **Type:** Task **Severity:** High

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-15, P0 — closes GA-05] Create tracked workable items (BOB-064..067, via the workable-items tool — never raw MD edits) for the four Lava-porting findings, citing implementing commits as evidence, closed as Implemented. GA-05 evidence: grep -in lava|BOB-06[4-7] docs/Issues.md docs/Fixed.md returns zero hits. Priority: P0. NOTE: BOB-064..067 have been created 2026-08-10 as Task/Queued (the four Lava P1..P4 items); the closed-as-Implemented status flip (citing per-finding implementing commits) remains owed under this item.

BOB-085 — RD2-18: Create top-level Boba proxy/merge-service v1.0.0 readiness ledger (closes GA-10)

Status: Queued **Type:** Task **Severity:** Medium

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-18, P2 — closes GA-10] Create the top-level Boba (proxy/merge-service) v1.0.0 readiness ledger (GA-10) — dedupe with the browser_extension existing one as the template. GA-10 evidence: only docs/RELEASE_READINESS_20260616.html/.md/.pdf (dated point-in-time snapshot) and the extension own ledger exist; no top-level proxy/merge-service ledger created. Priority: P2.

BOB-088 — RD2-21: Complete/verify README Tracked-Items + Status Documents table row-completeness (GA-07 remainder)

Status: Queued **Type:** Task **Severity:** Medium

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-21, P1] Complete/verify the README Tracked-Items + Status Documents table row-completeness (GA-07 remaining half). Not independently re-verified this round whether every mandated doc (CONTINUATION.md, Issues.md, Fixed.md, PORTING-FROM-LAVA.md, both new GA/RD2 audit docs, every Status.md/Status_Summary.md pair) actually has a row. Priority: P1.

BOB-090 — RD2-25: HelixQA Challenge entry exercising all three start.sh subcommands end-to-end against real compose stack

Status: In progress **Type:** Task **Severity:** Medium

RD2-25: HelixQA Challenge entry exercising all three start.sh subcommands end-to-end against real compose stack

[BOB-136 adoption audit 2026-08-21 -> In progress] a9366c9 bumped submodules/helixqa to HelixDevelopment/qa@00c5ca4 adding banks/boba-start-sh-reload.yaml (3 cases, all three subcommands). The bank

EXISTS but has never been EXECUTED: the commit states honestly that the helixqa binary was not built in-session (§11.4.3 SKIP feature_disabled_by_config) and 'live bin/helixqa list demo is deferred', with only YAML-parse evidence captured. This item's acceptance is end-to-end execution against the real compose stack — artifact-class evidence cannot close a runtime-class acceptance (§11.4.226).

BOB-093 — RD2-28: Live compose bring-up + verify rutracker ReDoS regex bounds deployed to container (closes runtime half of GA-12)

Status: In progress **Type:** Task **Severity:** High

RD2-28: Live compose bring-up + verify rutracker ReDoS regex bounds deployed to container (closes runtime half of GA-12)

[BOB-136 adoption audit 2026-08-21 -> In progress] 1c0389a proved 3 of the 4 sub-steps with strong runtime evidence: the bounded regex is confirmed in the DEPLOYED /config/qBittorrent/nova3/engines/rutracker.py inside the running container (sha256 76a6bd2e..), with a self-validated challenge + §11.4.115 RED (mutated unsafe form FAILs) and GREEN verdict JSON. The 4th sub-step is NOT met: 'capture timing of a large rutracker result page (<2s)' was not exercised — in docs/qa/BOB-093/live_search_smoke.txt rutracker returned status=empty, results_count=0 in 164ms, so no large page was ever timed.

BOB-094 — RD2-29: Author tests/stress/test_tracker_fetch_stress_chaos.py with §11.4.85 fault injection

Status: In progress **Type:** Task **Severity:** Medium

RD2-29: Author tests/stress/test_tracker_fetch_stress_chaos.py with §11.4.85 fault injection

[BOB-136 adoption audit 2026-08-21 -> In progress] The suite landed (test file at 4b7b21d, evidence + RED-on-mutated-classifier capture at f3ca131) with 9 test functions: 2 stress, 3 boundary, 3 chaos (network_drop, midflight_kill, input_corruption), 1 category_map. This item explicitly requires fault injection across process-kill, network-

fault AND resource-exhaustion. Only 2 of those 3 classes are present — there is no resource-exhaustion scenario in tests/stress/test_tracker_fetch_stress_chaos.py. Partial.

BOB-095 — RD2-30: Author tests/stress/test_scheduler_hooks_sse_stress_chaos.py for Go-side triangle

Status: In progress **Type:** Task **Severity:** Medium

RD2-30: Author tests/stress/test_scheduler_hooks_sse_stress_chaos.py for Go-side triangle

[BOB-136 adoption audit 2026-08-21 -> In progress] d5c8c3e authored qBitTorrent-go/internal/service/sse_broker_stress_chaos_test.go (462 lines, 6 scenarios, 2 RED captures) and found+fixed a real double-unsubscribe 'close of closed channel' defect. But this item names a THREE-component triangle: sse_broker.go + internal/api/hooks.go + internal/api/scheduler_api.go. Only sse_broker.go is covered — there is no stress_chaos test anywhere under qBitTorrent-go/internal/api/. 1 of 3, partial.

BOB-097 — RD2-32: Author DDoS-class coverage for exposed download-proxy/merge endpoints (canonical impl of RD2-07)

Status: Queued **Type:** Task **Severity:** Low

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-32, P3] Author DDoS-class coverage (RD2-07) for the exposed download-proxy/merge endpoints. Priority: P3.

BOB-100 — RD2-39: Bump submodules/jackett one commit (canonical impl of RD2-09)

Status: Queued **Type:** Task **Severity:** Low

[Backfill from GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-39, P3] Bump submodules/jackett one commit (RD2-09). Priority: P3.

BOB-101 — GA-19/RW-09: Is –profile go parity still a release goal? (gates RW-10..13) — OPERATOR-DECISION

Status: Queued **Type:** Task **Severity:** High

OPERATOR DECISION (2026-08-26, §11.4.66 interactive clarification): DEFER PAST v1.0.0

Go `--profile go` parity remains a goal but is NO LONGER a v1.0.0 release blocker. RW-10..13 move to a post-1.0 milestone and stop gating the tag. Nothing advertised is removed, so §11.4.122 stays clean and no §11.4.90 Obsolete closure is warranted. The BOB-141 measured fact stands unchanged: the Go container's Dockerfile runs ONE binary binding only :7187; nothing binds :7186 or :7188 despite compose setting PROXY_PORT/BRIDGE_PORT. CLAUDE.md's port map already describes what the container ACTUALLY does and needs no amendment under this decision.

This answer is recorded as consumer DATA per §11.4.35 — it is the operator's stated choice, not an agent inference, and supersedes any prior agent-chosen default on this question. Options not chosen are named above so a future reader does not re-litigate a settled call (§11.4.112(5) bounded-verdict discipline applied to decisions).

— prior item text follows —

GA-19/RW-09: Is –profile go parity still a release goal? (gates RW-10..13) — OPERATOR-DECISION

BOB-102 — RW-05: LAN-exposure threat model — bind tunnel 127.0.0.1 or keep 0.0.0.0? — OPERATOR-DECISION

Status: In progress **Type:** Task **Severity:** Medium

OPERATOR DECISION (2026-08-26, §11.4.66 interactive clarification): KEEP 0.0.0.0 + MANDATORY AUTH GUARD

The tunnel STAYS LAN-reachable — no bind address changes to 127.0.0.1. The operator explicitly chose to preserve access from other devices on the network. THE DECISION CARRIES A BINDING OBLIGATION: a permanent §11.4.135 regression guard MUST land that FAILS THE BUILD if any LAN-reachable route ever stops demanding authentication. The operator accepted 0.0.0.0 ON THE CONDITION that guard exists — without it this decision is unprotected and the item is NOT closeable. Guard requirements: enumerate routes from the authoritative source (FastAPI app, Go handlers, boba-jackett) never a hand-maintained list; resolve real auth coverage not a grep for the string 'auth' (§11.4.201 real-condition); deliberately-public routes exempt ONLY via a checked-in list with per-entry justification, enumerated as honest gaps never silent; golden-TRUE + golden-FALSE fixtures per §11.4.107(10).

This answer is recorded as consumer DATA per §11.4.35 — it is the operator's stated choice, not an agent inference, and supersedes any prior agent-chosen default on this question. Options not chosen are named above so a future reader does not re-litigate a settled call (§11.4.112(5) bounded-verdict discipline applied to decisions).

— prior item text follows —

RW-05: LAN-exposure threat model — bind tunnel 127.0.0.1 or keep 0.0.0.0? — OPERATOR-DECISION

BOB-104 — §11.4.238 followup: CodeGraph 1.5.0 nested-.gitignore regression challenge

Status: In progress **Type:** Task **Severity:** Medium **Created-By:** Claude

Coverage-escape followup (docs/QA_DISCOVERY_LEDGER.md, BOB-075 agent-code-reading finding, commit e6162f7): CodeGraph 1.5.0 (up from documented 0.9.9) walked into nested-.gitignore-excluded frontend/node_modules and extension/node_modules (32,260 files / 514,456 nodes vs the 2026-06-06 baseline of 509 files / 8,906 nodes) instead of honoring frontend/.gitignore + extension/.gitignore per git check-ignore -v. Author a challenge (challenges/scripts/codegraph_gitignore_honor_challenge.sh or equivalent, e.g. wrapping 'codegraph doctor –sniff-gitignore-honor' if that subcommand exists, else a real re-index + file-count assertion) with §11.4.115 RED_MODE polarity: RED_MODE=1 reproduces the blowup against the live nested-.gitignore tree, RED_MODE=0 asserts the resync stays within the documented baseline order of magnitude. Full evidence: docs/

codegraph/Status.md lines 91-118. UPSTREAM FILED 2026-08-18: real upstream repo is github.com/colbymchenry/codegraph (npm package @colbymchenry/codegraph, confirmed via package.json + gh repo view; NOT vasic-digital/codegraph, correcting this ledger's original assumption). Issue: https://github.com/colbymchenry/codegraph/issues/1567 (full reproduction evidence: git check-ignore -v re-verified first-hand; two bounded synthetic repro attempts up to 63 nested .gitignore files / 960 files against the actual installed v1.5.0 binary did NOT reproduce the blowup in isolation – honest negative result, root cause not pinned to a specific line in either scanning implementation). Draft PR (diagnostic only, not a behavioral fix): https://github.com/colbymchenry/codegraph/pull/1568 – adds a logDebug() call so a future report can confirm which of the two independent gitignore-respecting code paths (git-ls-files-based vs filesystem-walk fallback) actually ran. Both PRs/issue filed via SSH (Hard Stop #2) from a fork at github.com/milos85vasic/codegraph. Status: In-progress pending upstream maintainer triage/merge – boba-side closure of this item still needs the RED/GREEN challenge script (§11.4.115) authored per the original scope, independent of upstream's response.

BOB-106 — §11.4.238 followup: §11.4.84 quiescence-check helper for the unattributed auto-commit path

Status: Queued **Type:** Task **Severity:** Medium **Created-By:** Claude

Coverage-escape followup (docs/QA_DISCOVERY_LEDGER.md RD2-00/BOB-068 entry, docs/GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-00): 20 bare 'Auto-commit' commits exist in this repo's git history (confirmed via git log –oneline –all –grep, e.g. 54e313f/9c8f684/743097a/de9270b/1c36777/41179c2/7c529ca) with no ATM-NNN reference and no TDD trail, landing via ordinary git pull fast-forward from a second session/host with push access to the same remotes (mismatched commit timezone vs the investigating host, per RD2-00 Update). §11.4.84 working-tree-quiescence has no mechanical guard on this path — no gate flags a commit reaching main with a bare/templated message and no ticket citation. Author a §11.4.84 quiescence-check helper (e.g. challenges/scripts/no_unattributed_autocommit_challenge.sh) that scans the commit range since the last known-good release tag and FAILs on any commit message matching a closed bare/templated pattern (e.g. ^Auto-commit$, ^sync:) with no ATM-NNN or task/PR reference, wired into

scripts/pre_build_verification.sh or the §11.4.234 commit-push-all.sh entrypoint. BOB-068 (RD2-00) remains the tracking item for identifying/stopping the source; this item is specifically the new automated CHECK.

BOB-107 — §11.4.238 followup: pre-dispatch existence check for subagent task-brief source inputs

Status: Queued **Type:** Task **Severity:** Medium **Created-By:** Claude

Coverage-escape followup (docs/QA_DISCOVERY_LEDGER.md SCRATCH-LOSS-2026-08-18 entry, evidence: .superpowers/sdd/task-phase1a-report.md line 21): the Phase 1a subagent (a1cc331d) discovered at task start that 5 source files its brief named as required reads (curriculum_amendment_plan_v1.md, ai_curriculum_modules_27_35_extracted.md, and 3 curriculum_analysis_modules_*.md gap analyses) were absent from the session scratchpad — root cause per that subagent's own investigation: the prior producer subagent (ae59171f) hit its session rate limit (a §11.4.147(e) API-quota crash) before writing them. curriculum_amendment_plan_v1.md remains absent from the live scratchpad as of this session's re-check. No mechanical check verifies a task brief's declared input files exist and are non-empty before the downstream consumer subagent is dispatched. Author a pre-dispatch precondition helper (project-side orchestration tooling, e.g. a small script the conductor runs before Task/Agent dispatch when a brief names required input paths) that fails closed with an actionable 'missing input, respawn the producer' message rather than silently letting a downstream agent proceed on absent evidence and fabricate content unsupported by its named sources.

BOB-109 — BOB-074 followup: scaling-class test coverage absent from mandated test-type matrix

Status: Ready for testing **Type:** Task **Severity:** Medium **Created-By:** Claude

docs/testing/test_type_matrix.md's §11.4.27 test-type audit found zero scaling-class coverage anywhere in the tree (no scaling-tagged directory, test file, or HelixQA bank distinguishes growing-dataset/tracker-count/concurrent-user scale-out from stress-under-burst). Scope at least one scaling dimension, e.g. tracker-count scale-out in merge search against challenges/helixqa-banks/boba-services.yaml's tracker set, or the qbittorrent-proxy-go –profile go swap, with a real measured baseline.

BOB-110 — BOB-074 followup: UX-class test coverage (accessibility/usability) absent

Status: Queued **Type:** Task **Severity:** Medium **Created-By:** Claude

docs/testing/test_type_matrix.md's §11.4.27 test-type audit found UI functional coverage (Vitest + Playwright) but nothing framed around usability/accessibility/UX outcomes specifically. Scope an axe-core or equivalent accessibility pass over the Angular frontend, covering WCAG checks, keyboard-nav coverage, and screen-reader labeling.

BOB-111 — BOB-074 followup: configure real rate limiting for boba's 3 public HTTP endpoints

Status: In progress **Type:** Task **Severity:** High **Created-By:** Claude

Source inspection across the whole stack (2026-08-18) verified no rate-limit mechanism exists for :7185 (qBittorrent WebUI proxy), :7187 (merge search service), or :7189 (boba-jackett) – no slowapi/limiter/throttle import in download-proxy/src/, no rate-limit middleware in qBitTorrent-go/internal/middleware/ (only cors.go + logging.go) or internal/jackettapi/ (only auth/cors middleware tests, no rate middleware), and no nginx/reverse-proxy service in docker-compose.yml. Candidate remediations documented in docs/testing/ddos_resilience.md: nginx-in-container reverse proxy with limit_req_zone (most portable, adds a container); a FastAPI slowapi dependency for the merge-search service; a Gin rate-limit middleware for boba-jackett following the existing internal/middleware/ pattern. qBittorrent's own WebUI bandwidth-shaping settings were NOT verified to cover request-rate (only bandwidth) – do not assume they close this gap without checking.

BOB-114 — BOB-074 followup: self-validation golden-bad fixture for the rate-limit detector

Status: Queued **Type:** Task **Severity:** Medium **Created-By:** Claude

challenges/scripts/ddos_resilience_challenge.sh's –self-validate mode currently only ships a golden-bad fixture for the crash-resistance detector (per §11.4.107(10)/§11.4.201). The rate-limiting detector has no matching golden-bad fixture proving it would actually FAIL a synthetic no-rate-limit-enforced artifact, so an unvalidated rate-limit detector could silently pass a broken/absent rate-limit deployment. Add a synthetic fixture (e.g. a stub server that never returns 429/503 under burst) and assert the detector correctly reports the absence, closing the self-validation gap for this detector class.

BOB-121 — External watchdog for the forced-logout architectural gap (task #85, incident #3)

Status: Ready for testing **Type:** Task **Severity:** Important **Created-By:** Claude

Phase 1 design-only proposal: the BOB-116/task-77 resource-pressure preventive systemd –user timer runs inside user@1000.service, the exact pool it monitors, so it cannot fire when that pool dies (proven by incident #3, docs/qa/BOB-120/, 22:42+22:57 fires then blocked 23:45:49-23:49:00). Recommends Option B (user crontab reusing pre-existing crond.service in system.slice, no new root service) kept alongside the existing timer, with Option A (new root-owned systemd unit) as escalation path if Phase 1.5 live cron/cgroup verification is adverse. Proposal: docs/proposals/external-watchdog-for-forced-logout-architectural-gap.md. Operator decision required per §11.4.66 before any implementation – NOT implemented in this task.

Progress 2026-08-21: Flight recorder built, installed, and recording. First finding: NOTHING was watching at all — the root watchdog is LoadState=not-found and the BOB-116 user timer inactive. The blocking spike REFUTED the proposal's rationale while confirming its conclusion: this vixie-cron does invoke pam_systemd so the tick lands in session-N.scope, but that scope is a SIBLING of user@1000.service, and the real incident killed exactly one unit (73 'user@1000.service:

Killing process' lines while session-18.scope merely deactivated). So the recorder does not need to survive — its SCHEDULER does, and crond in system.slice survived all seven incidents. Captures the pre-event memory/PSI/thread ramp that nothing post-hoc can recover, plus boot id, the gap itself, unit start timestamp, cgroup pids, and OOM cgroup attribution. Evidence: 5 records under real cron at exact 60s spacing; a staged analogue on a DISPOSABLE unit reaching 'VERDICT: SESSION TEARDOWN'; golden-good against the REAL BOB-120 log giving the correct diagnosis (k_unit_kill=1, k_oom=0); healthy-host quiet under load 8.15. NOT CLOSED — two operator decisions are owed: whether to keep it installed (one crontab line, zero signals, zero power verbs, 32K, reversible via uninstall.sh), and the root system.slice watchdog which is written but needs one `su`. UNTESTED AGAINST A REAL FORCED LOGOUT: survival is inferred from cgroup topology, not observed, and it does NOT hold against a whole-slice sweep or KillUserProcesses=yes. ## BOB-135 — Test isolation: test_list_hooks_after_create fails in bulk suite (Permission denied /config)

Status: Ready for testing **Type:** Bug **Severity:** Low

Task #109 subagent found: tests/unit/ test_merge_api_route_contracts.py::TestHooksEndpoint::test_list_hooks _after_create fails when run in bulk suite order with 'ERROR api.hooks:hooks.py:102 Failed to save hooks: [Errno 13] Permission denied: /config'. Passes in isolation (2.02s clean). Root cause: full-suite ordering pollution — some earlier test leaves state that makes hooks try to write to /config (which the test env doesn't own). Pre-existing, unrelated to BOB-126/BOB-129 chain. Fix strategy: identify the polluting test, add teardown or use a proper tempdir fixture for hooks storage in the offending test.

BOB-137 — Merge service on 7187 wedges while the same process still serves 7186 (GIL starvation by one spinning thread)

Status: In progress **Type:** Bug **Severity:** High **Created-By:** Claude

Reported-Via: §11.4.202 reporting directive `bug` on 2026-08-20T14:46:52Z **Reported-By:** Claude

What (the report, verbatim): The merge service on port 7187 wedges after hours of uptime: the port stays bound and the container keeps reporting “healthy”, but every HTTP request hangs until the client times out. Measured 2026-08-20 16:41 UTC+2 on a container up 3h53m:

curl http://localhost:7186/ -> HTTP 200 in 0.096s (proxy, same process)
curl http://localhost:7187/ -> HTTP 000 after 6.0s (merge service, WEDGED)

Both ports are served by the SAME process (pid 2330069, fd 3 = 7186, fd 7 = 7187), so this is not a crashed worker – one loop inside a live process has stopped servicing requests while another in the same process is fine.

Thread state at the time of the wedge (/proc/2330069/task, 16 threads): - tid 2330529: state R, wchan 0 <- ONE thread spinning on CPU - tid 2330528: state S, wchan do_sys_poll <- the healthy 7186 poll loop - the other 14: state S, wchan futex_wait_queue

That is the GIL-starvation signature: a thread busy-looping in Python holds the GIL and every other thread queues on the GIL futex. 7186 survives because do_sys_poll releases the GIL; the 7187 async loop needs sustained GIL time to service a request and starves. Process CPU was 20.6% while idle.

Corroborating evidence: seven sockets held by the process sit in CLOSE-WAIT with unread bytes still in the receive queue (Recv-Q 162, 162, 162, 79, 1, 1, 1) – clients sent a request and hung up, and the app never read it or closed the fd. The listener had also accumulated an unaccepted backlog (Recv-Q 6 on the 7187 LISTEN socket) at first observation. The container log’s last line is 2h before the observation, so the service processed nothing in that window.

NOT fd exhaustion: only 48 of 16384 fds were open.

ROOT CAUSE NOT ESTABLISHED. All four `while True` loops in the service (routes.py:166, search.py:1169, streaming.py:161, streaming.py:359) are correctly bounded and awaited, so the spin is not a naive unslept loop. py-spy could not attach to name the spinning frame – the host has kernel.yama.ptrace_scope=1, which denies non-child attach. Per the §11.4.102 Iron Law no fix is proposed until the spinning frame is identified.

Next diagnostic step: obtain a Python stack. Either run py-spy INSIDE the container (musl wheel), or set ptrace_scope=0 for the duration of one dump, or add a SIGQUIT/faulthandler.register() dump hook to main.py so the running service can be asked for its own stacks without ptrace.

DISCOVERY CHANNEL (§11.4.238): found by an agent probing the host by hand while investigating an unrelated test-suite result – NOT by automated QA. This is a coverage escape in its own right; see the sibling item on the health check that was structurally incapable of observing it.

Affected scope / file-scope manifest: download-proxy/src/main.py, download-proxy/src/merge_service/, download-proxy/src/api/streaming.py, docker-compose.yml (qbittorrent-proxy)

Reproduction / context: Leave the qbittorrent-proxy container running for several hours with search traffic (a full tests/security run is sufficient). Then: curl –max-time 6 http://localhost:7186/ returns 200; curl –max-time 6 http://localhost:7187/ returns 000. Confirm with: ss -ltnp | grep 7187 (unaccepted backlog) and awk '{print $3}' /proc//task/*/stat (one R thread, rest futex_wait_queue).

Acceptance criteria: The spinning frame is identified from a real Python stack dump (not inferred), the busy-loop is fixed at its root, and a regression guard proves 7187 still answers after a sustained-traffic soak. Evidence: before/after curl timings on both ports plus a thread-state census showing no permanently-R thread.

[BOB-136 adoption audit 2026-08-21 -> In progress] 1dd7b0a ESTABLISHED the root cause with captured evidence (16 stack dumps showing Deduplicator.merge_results() called synchronously at search.py:914 on the event-loop thread; loop thread sustained 81-98% user-space CPU while all other threads showed d_utime=0). The remediation landed under BOB-145 (0572b71, now Fixed), whose own text is headed 'BOB-145 - the 7187 wedge' and which refuted the assumed O(N^2) cause by profiling. BOB-137 and BOB-145 therefore describe the SAME defect; per §11.4.214 that is a link/dedup decision, not a unilateral close, and no post-fix re-observation of the multi-hour wedge is recorded against BOB-137 itself. Left open pending that linkage decision.

Live verification 2026-08-21 — REDUCED BUT NOT ELIMINATED. Deliberately NOT closed.

The service was confirmed running POST-FIX bytes before any measurement, six independent ways: served sha256 == committed == worktree across 20 merge-service/api/main files; fix markers 10 inside the container vs 0 at the pre-fix commit; `.pyc` header decode showing embedded source mtime+size matching the actual file (so the import reused it and the loaded module was compiled from exactly this source); fix markers present in the LOADED bytecode; and the process starting 823s AFTER the fix hit disk. A first attempt at this comparison reported routes.py as stale — an INSTRUMENT ARTIFACT (marshal back-reference encoding differs between a fresh compile and a pyc load), caught before it became a false positive.

Soak, same script and concurrency, precondition reached (11,340 results over 516 tracker responses / 12 searches ~ 945 per merge, larger than BOB-145's N=800 maximum):

```
PRE-FIX  (quoted from the recorded report): 22 of 26 probes dead
on 7187 (84.6%)
POST-FIX (measured):                         2 of 141 dead (1.4%)
```

The dead-count alone understates the residual: 25.5% of probes stalled >1s (<0.1s 65.2% / 0.1-1s 9.2% / 1-5s 18.4% / >=5s 5.7% / dead 1.4%). Both dead events show the exact BOB-137 asymmetry — 7186 answering in 0.077s while 7187 timed out at 10s — and both coincide with the loop thread sampled at state R with wchan=0, the GIL-starvation signature (tid independently confirmed as the loop thread by its idle wchan do_epoll_wait).

WHY THIS ROW STAYS OPEN: the acceptance evidence this item names is '22/26 dead -> 0/N dead'. Achieved: 22/26 -> 2/141. The user-visible symptom — 7187 unresponsive while 7186 answers in the same process — STILL OCCURS, twice in 15 minutes. That is precisely BOB-145's own predicted residual: search.py:914 is still a plain synchronous call, and removing the symptom needs a change at that CALL SITE (offload or await), which BOB-145 explicitly scoped out.

Two criteria that ARE satisfied: no permanently-R thread (48 of 80 census samples had zero R threads), and the in-process 20s watchdog logged 0 stalls — with a control needle, since that same watchdog produced 167,971 bytes of dumps pre-fix. ## BOB-141 — CLAUDE.md claims the Go profile serves 7186/7187/7188 but its container binds only 7187 — doc contradicts the Dockerfile

Status: Queued **Type:** Task **Severity:** Low **Created-By:** Claude

Reported-Via: §11.4.202 reporting directive `task` on 2026-08-20T14:56:38Z **Reported-By:** Claude

What (the report, verbatim): CLAUDE.md's Architecture section states:

"qbittorrent-proxy-go (Go/Gin, opt-in via –profile go) – replaces the Python proxy on 7186, 7187, 7188"

The container cannot deliver that. Read from source rather than prose:

qBitTorrent-go/Dockerfile:16 CMD ["/app/qbittorrent-proxy"] (ONE binary) qBitTorrent-go/Dockerfile:15 EXPOSE 7187 7188 (declares 2) cmd/qbittorrent-proxy/main.go r.Run(fmt.Sprintf(":%d", cfg.ServerPort)) (binds 1) internal/config/config.go:58 ServerPort = MERGE_SERVICE_PORT (default 7187)

So the qbittorrent-proxy-go container binds 7187 ONLY. webui-bridge is a SEPARATE binary (cmd/webui-bridge, /bridge/health, cfg.BridgePort) that this container never starts, and nothing in it binds 7186 either – although the compose service does set PROXY_PORT=7186 and BRIDGE_PORT=7188 in its environment, which reinforces the wrong impression. EXPOSE likewise declares a port nothing binds.

WHY THIS MATTERS BEYOND TIDINESS: this prose was used as the source of truth when first authoring config/served_ports.yaml, producing a manifest entry of [7186, 7187, 7188] for that service. The healthcheck gate built on it then FAILED a service whose healthcheck was already correct – a §11.4.201(1) false-positive refusal caused directly by trusting the doc over the Dockerfile. The manifest was corrected against source; the doc was not, and will mislead the next reader the same way.

Either the doc is wrong, or the Go container is under-provisioned relative to intent (it should also run webui-bridge and a 7186 listener). Determining WHICH is part of this task – do not simply reword the doc to match the current binary if the intended design was a three-port container.

Related: the Go service's compose block sets PROXY_PORT and BRIDGE_PORT that no process in the container consumes, and EXPOSE lists 7188 unbound. Whichever way the above resolves, those should agree with reality afterwards.

Affected scope / file-scope manifest: CLAUDE.md (Architecture + Port Map), docker-compose.yml (qbittorrent-proxy-go env/EXPOSE), qBitTorrent-go/Dockerfile

Reproduction / context: Read qBitTorrent-go/Dockerfile:16 (single CMD) against CLAUDE.md's 'replaces the Python proxy on 7186, 7187, 7188'; confirm cfg.ServerPort resolves to MERGE_SERVICE_PORT=7187 in internal/config/config.go:58.

Acceptance criteria: Doc and container agree, with the direction of the fix decided deliberately (correct the doc, or provision the container to match the documented intent). PROXY_PORT/BRIDGE_PORT env and EXPOSE lines agree with what the container actually binds.

BOB-143 — Orphaned .worktrees/ dirs (46M, unresolvable gitdir) pollute gate scan scope and manufacture false BOB-126-class findings

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** Claude

Reported-Via: §11.4.202 reporting directive `bug` on 2026-08-20T15:08:18Z **Reported-By:** Claude

What (the report, verbatim): `.worktrees/ci-split-workflows/` (13M) and `.worktrees/completion-initiative-phase-0/` (33M) are ORPHANED: `git worktree list` reports only the main checkout, so neither is a registered worktree. Each contains a `.git` POINTER FILE whose target gitdir no longer exists, so git cannot resolve HEAD, branch, or status inside them – every query returns empty.

They are gitignored (.gitignore:122), so nothing tracks them and nothing will ever notice them drifting.

WHY THIS IS NOT COSMETIC: they pollute the scan scope of whole-tree gates and manufacture false findings. Measured 2026-08-20 by the §11.4.32 sweep:

- `cm_test_mock_pid_explicit_int` reported 2 violations at tests/unit/merge_service/test_deadline_tunable.py:44 – BOTH inside these orphaned trees. The MAIN tree's copy of that file is already hardened (it sets `mock.pid = 12345` and patches os.killpg/os.getpgid) and passes the gate cleanly. The finding read as a live §11.4.263 / BOB-126-class defect and was not one.

- 6 of the 57 “missing anchor carrier” files flagged by the propagation gates were likewise `.worktrees/**`.

That is the §11.4.201(1) false-positive shape sourced from scan scope, and it costs real investigation time: a reader triaging “2 live BOB-126 violations” reasonably treats it as a host-safety emergency.

CLARIFICATION (established during triage, so the record is not alarming): these trees are NOT a kill(-1) vector. Their `download-proxy/ src/merge_service/ search.py` contains ZERO `os.killpg` calls – they predate that cleanup code entirely – so there is no unguarded signal call to reach. Additionally `pyproject.toml` sets `testpaths = ["tests"]`, so a plain `pytest` run does not collect from `.worktrees/`. No host-safety risk was found. The defect is scan-scope noise plus 46M of unreferenced disk.

WHY REMOVAL IS NOT DONE AUTONOMOUSLY (§11.4.122 / §11.4.124 / §11.4.101): because git cannot resolve their HEAD, it is NOT possible to prove their contents are merged into main. Deleting unprovable-provenance work is exactly the irreversible, operator-owned decision §11.4.122 reserves. Two options for the operator: (a) confirm removal (they are stale dev scratch dirs) – reversible only from backup, so a §9.2 pre-op backup should precede it; or (b) keep them and add `.worktrees/` to the gate scan-scope exclusion list as a §11.4.224(E)-fenced, checked-in, justified entry.

Either way the exclusion list is the cheaper immediate mitigation and does not destroy anything.

Affected scope / file-scope manifest: .worktrees/ci-split-workflows/, .worktrees/completion-initiative-phase-0/, gate scan-scope config

Reproduction / context: git worktree list shows only the main checkout; git -C .worktrees/

> log -1 returns empty (gitdir target missing). Run the §11.4.32 sweep and observe cm_test_mock_pid_explicit_int report 2 violations, both under .worktrees/, while the main-tree file passes the same gate.
>
> **Acceptance criteria:** Whole-tree gates no longer report findings sourced from orphaned worktrees: either the dirs are removed after operator confirmation with a §9.2 pre-op backup,

or .worktrees/ is added to a checked-in §11.4.224(E)-fenced exclusion list with justification. Verify by re-running the sweep and confirming zero .worktrees-sourced findings.

BOB-149 — Managed-plugin count diverges 43/42/48 across constitution, CLAUDE.md and the README badge; the badge is hand-maintained and unguarded

Status: Queued **Type:** Bug **Severity:** Low **Created-By:** Claude

Reported-Via: §11.4.202 reporting directive bug on 2026-08-20T22:35:42Z **Reported-By:** Claude

What (the report, verbatim): The managed-plugin count diverges three ways across the repo:

.specify/memory/constitution.md : 43 (and enumerates all 43 by name) CLAUDE.md : 42 (“42 managed plugins”) README.md badge : 48 (plugins-48)

AUTHORITATIVE SOURCE, per the constitution’s own text (“the install-plugin.sh managed list is the canonical curated set”): the PLUGINS=() array in install-plugin.sh holds exactly 43 entries.

Counted with a control needle (§11.4.201(7)(b)): the extractor was verified to match a known member (“rutracker”) before its count was trusted. A first attempt returned 0 because the array entries are QUOTED (“academictorrents”) and the pattern was unquoted — a false-null caught by the needle rather than reported as “no plugins”.

So the constitution is CORRECT and the other two are the drifted copies.

Neither wrong number is harmless: - CLAUDE.md’s 42 is the file agents read as project instruction, so the wrong number is the one most likely to be propagated into new work. - README.md’s 48 matches NEITHER the curated array (43) NOR plugins/*.py (36) NOR plugins/**/*.py (69). It is not a stale-but-once-true number; nothing in the repo currently equals 48, so its provenance is unknown.

The badge is additionally UNGUARDED: compute-badges.sh does not derive the plugins badge at all — it is one of the hand-maintained ones. That is the same shape as the challenges/pre-build badges fixed at a6f36fa, which had been stale by 7 and 14 while the script printed “cross-checked, matches existing badge”. The plugins badge escaped that fix because it was never in the script’s scope.

Acceptance: 1. CLAUDE.md states 43, matching the authoritative array. 2. The README plugins badge is DERIVED by compute-badges.sh from the PLUGINS=() array (not hand-typed), so it cannot silently drift again. 3. tests/unit/ test_compute_badges_all_badges_updated.sh is extended to cover the plugins badge, so the guard covers every machine-derived badge rather than the subset that happened to be fixed first. 4. Verified in both directions (§11.4.201(1)): the guard FAILs when the badge disagrees with the array, and PASSes when they agree.

Filed rather than fixed inside the v1.4.0 constitution amendment so the documentation fix and the governance change stay independently reviewable.

Affected scope / file-scope manifest: CLAUDE.md, README.md (plugins badge), scripts/compute-badges.sh, tests/unit/ test_compute_badges_all_badges_updated.sh

Reproduction / context: Count the PLUGINS=() array in install-plugin.sh (43, needle-verified) and compare against CLAUDE.md (‘42 managed plugins’) and the README badge (plugins-48).

Acceptance criteria: CLAUDE.md says 43; the README badge is derived by compute-badges.sh from the array; the badge guard covers it; both polarities verified.

BOB-150 — pre_build_verification.sh invariant labels read N/50 but only 41 invariants are labelled

Status: Queued **Type:** Task **Severity:** Low **Created-By:** AI

Reported-Via: §11.4.202 reporting directive `task` on 2026-08-21T14:46:07Z **Reported-By:** AI

What (the report, verbatim): The pre-build runner announces each invariant as [N/44], but only 35 invariants carry a label and only 35 distinct CM-* gate names are announced. Numbers 33-38, 40, 42 and 43 are unused; there are no duplicates. An operator reading the output sees '[44/44]' scroll past and reasonably concludes 44 invariants ran, when 35 did - the output overstates coverage by 9. This is PRE-EXISTING and was surfaced while wiring CM-OWNERSHIP-INVARIANTS, which deliberately took free slot 33 precisely to avoid a 35-label renumbering that would have conflicted with concurrent work. That gate neither introduced nor fixed this. Filing rather than absorbing it silently, per the closed-or-tracked rule: an unstated finding reads as no finding.

Affected scope / file-scope manifest: scripts/ pre_build_verification.sh

Reproduction / context: grep -oE '[[0-9]+/44]' scripts/ pre_build_verification.sh | wc -l -> 35 (denominator says 44); numbers 33-38, 40, 42, 43 are unused, no duplicates. Distinct CM-* names announced in the file: 35, which agrees with the label count and not with the denominator.

Acceptance criteria: Either the denominator matches the real number of labelled invariants, or the numbering is compacted to be contiguous - and whichever is chosen, a gate or the runner itself derives the denominator rather than restating it as a literal, so the two cannot drift apart again.

BOB-151 — CM-SCRIPT-DOCS-SYNC is a named gate with no implementation, and 24 of 36 scripts have no companion doc

Status: Queued **Type:** Task **Severity:** Medium **Created-By:** AI

Reported-Via: §11.4.202 reporting directive `task` on 2026-08-21T15:05:33Z **Reported-By:** AI

What (the report, verbatim): The §11.4.18 script-documentation rule names a gate CM-SCRIPT-DOCS-SYNC, but no executable file in this repository implements it. Because nothing measures the obligation, 24 of 36 scripts under scripts/ have no docs/scripts/.md companion - two thirds of them. This is the named-gate ledger gap the constitution itself warns about: a gate that is named but never

implemented reads as coverage while providing none, and the corpus grows words instead of enforcement. Surfaced while writing the two ownership companions, which were themselves only written because the feature plan asked for them explicitly - not because any gate demanded it. Filing rather than absorbing: an unstated finding reads as no finding, and a 67% documentation gap that nothing reports will not shrink on its own.

Affected scope / file-scope manifest: scripts/*.sh, docs/scripts/*.md, scripts/pre_build_verification.sh

Reproduction / context: grep -rl CM-SCRIPT-DOCS-SYNC –include='*.sh' –include='*.py' . (excluding submodules) -> 0 files. Control needle in the same command shape: CM-OWNERSHIP-INVARIANTS -> 3 files, CM-KILLPG-PGID-GUARD -> 7 files, so the search is not blind. Then: for s in scripts/*.sh; do [-f docs/scripts/$(basename $s .sh).md] || echo missing; done -> 24 of 36 missing.

Acceptance criteria: Either the gate is implemented and the 24 missing companions are written (the count becoming a monotone-decreasing ratchet so it cannot grow), or the obligation is explicitly scoped down to a defined subset with the reason recorded. What is NOT acceptable is the current state, where the rule is stated and nothing measures it.

BOB-152 — Constitution sweep walks vendored third-party code: 82% of one gate's 38,291 findings come from submodules/

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** AI

Reported-Via: §11.4.202 reporting directive bug on 2026-08-21T15:10:28Z **Reported-By:** AI

What (the report, verbatim): The constitution sweep passes –root at the repository root, so every gate walks vendored third-party code the project neither wrote nor ships: submodules/helixqa/tools/opensource/perfetto, chroma, skyvern, mem0 and so on. Measured today, 82% of one gate's findings originate there. This is a false-positive refusal at scale: it fails the sweep over code that cannot be fixed here, and it buries the 4497 first-party

findings that might actually matter under 31496 that do not. A second, subtler instance: cm_killpg_pgid_guard flags its OWN golden-bad fixtures and the BOB-126 incident docstrings - the gate reporting on the very artifacts that prove it works. Surfaced while fixing the .worktrees/ half of this problem (BOB-143), which was the smaller 6% slice; that fix was deliberately scoped to .worktrees/ rather than quietly widened to cover this, because excluding submodules/ is a materially larger decision about what the sweep is for and deserves its own review.

Affected scope / file-scope manifest: scripts/verify-all-constitution-rules.sh, config/constitution-sweep.conf, constitution/scripts/gates/*

Reproduction / context: config/constitution-sweep.conf line 27 passes 'DEFAULT –root @ROOT@', so every gate walks the whole repository. Per-tree census of cm_oracle_strategy_named_and_independent over the full root: 38291 findings total - 31496 (82%) from submodules/, 2280 (6%) from .worktrees/, 4497 from the real tree. cm_killpg_pgid_guard: 18 findings - 9 from submodules/, and of the 8 in the real tree several are the gate's OWN golden-bad fixtures and BOB-126 docstrings.

Acceptance criteria: The sweep scans code this project actually ships. Vendored third-party trees under submodules/ are excluded or scoped explicitly, and any gate's own golden-bad fixtures are excluded from its own scan - with the exclusion validated in BOTH directions (a planted violation in first-party code must still FAIL) so this does not become narrow-until-green.

BOB-154 — Host venv and production container run different starlette versions (1.4.1 vs 1.6.0)

Status: Ready for testing **Type:** Bug **Severity:** Medium **Created-By:** AI

Reported-Via: §11.4.202 reporting directive bug on 2026-08-21T15:46:07Z **Reported-By:** AI

What (the report, verbatim): The host virtualenv used to run this project's unit tests resolves starlette 1.4.1 while the running qbittorrent-proxy container resolves 1.6.0. Every other implicated

package matches, so this is a genuine unpinned-transitive-dependency drift rather than a deliberate difference. It matters because it silently weakens every test result: a green suite on the host is evidence about 1.4.1, and production is 1.6.0. A behavioural change between those versions would be invisible to the tests that exist to catch it, which is the build-once-run-the-same-bytes property the constitution asks for. Surfaced while investigating BOB-129, where the defect happened to reproduce IDENTICALLY at both versions - which is what allowed that ticket's slowapi/starlette-incompatibility premise to be refuted. That was luck, not design: the next divergence may not be version-independent, and nothing would tell us.

Affected scope / file-scope manifest: download-proxy/ requirements.txt, .venv, container qbittorrent-proxy

Reproduction / context: .venv/bin/python -c 'import starlette;print(starlette.__version__)' -> 1.4.1 ; podman exec qbittorrent-proxy python -c 'import starlette;print(starlette.__version__)' -> 1.6.0. Same fastapi (0.141.1), same slowapi (0.1.10), same limits (5.8.0) - starlette alone diverges.

Acceptance criteria: The interpreter that runs the tests and the interpreter that serves production resolve the same versions, pinned so they cannot drift apart silently - or, if a divergence is deliberate, it is declared and a check asserts the declared pair rather than leaving it to chance.

BOB-156 — BOB-145 event-loop regression guard is load-sensitive and flaky: 8786ms under host load vs a 900-1500ms ceiling calibrated on a quiet host

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** AI

Reported-Via: §11.4.202 reporting directive bug on 2026-08-21T17:01:20Z **Reported-By:** AI

What (the report, verbatim): The regression guard added with the BOB-145 fix asserts an event-loop block ceiling calibrated on a quiet host. Under real host contention the block window scales with BOTH N and load, so the same code that passes at 349ms

median can measure 8786ms - worse than the pre-fix number the test exists to detect. That makes it FLAKY, and a flaky test is corrosive in a specific way this project has already recorded: every ignored red trains everyone to dismiss the next one, so a real regression eventually gets waved through as ‘that one again’. Two directions are wrong: raising the ceiling until it stops failing would blind it to the defect, and leaving it flaky poisons every future run. Found while verifying BOB-137 against the live service.

Affected scope / file-scope manifest: tests/unit/merge_service/test_dedup_event_loop_blocking.py

Reproduction / context: Under host load 18-24 on 8 cores (concurrent agents), the same N=400 merge froze the event loop for 8786ms - WORSE than the 3970ms pre-fix figure the ceiling was calibrated against. 2 fail / 2 pass across four runs.

Acceptance criteria: The guard gives the same verdict on a loaded host as on a quiet one - or it measures something contention-independent. A threshold that only holds when nothing else is running is not a regression guard, it is a weather report.

BOB-159 — Warm ./start.sh over an already-running stack leaves the FR-004d repair window open

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** T041 independent review (IMPORTANT-2), partially remediated

OPERATOR DECISION (2026-08-26, §11.4.66): REPAIR, THEN RE-VERIFY UNTIL STABLE

The warm ./start.sh path keeps running the ownership repair against a live stack, but walks, re-verifies, and repeats until a pass finds nothing new. Options not chosen: refuse the repair while the stack is up and direct the operator to –recreate; stop the stack on the warm path too (correct but ends ‘warm’); accept and document the window. WHY THIS MATTERS MORE THAN A NORMAL ITEM: this is the exact defect the whole 002-user-owned-downloads feature exists to end. The –recreate path was already fixed to order precondition -> down -> repair -> up so the walk sees a tree no container can write. The warm path was not, so a

container can create a new non-owned file AFTER the repair has passed that directory — the ownership problem leaking back in through the other door. REQUIRED BY THE CHOSEN OPTION, and it is the hard part: the loop MUST be BOUNDED, and on giving up it MUST report honestly that ownership is UNPROVEN rather than silently exiting 0 (§11.4.201(6) — a loop that stops finding new files because it ran out of iterations returns the same quiet zero as a genuinely clean tree). An active download creating files faster than the walk completes is the realistic non-converging case and must be named in the give-up message.

Recorded as consumer DATA per §11.4.35 — the operator's stated choice, not an agent inference. Options not chosen are named so a future reader does not re-litigate a settled call.

— prior item text follows —

Reported-Via: §11.4.202 reporting directive `bug` on 2026-08-21T19:01:17Z **Reported-By:** T041 independent review (IMPORTANT-2), partially remediated

What (the report, verbatim): The –recreate path was fixed this round: run_ownership_precondition -> stack_down -> run_ownership_repair -> stack_up, so the repair walks a tree no container can write to. That ordering is now asserted behaviourally by three new checks in tests/unit/ test_start_reload_recreate.sh, each killed by its paired 1.1 mutation (11/11 RED-OK).

The WARM path is NOT covered by that fix and this item tracks the remainder. On a warm ./start.sh the gate runs before THIS invocation brings anything up, but if the stack is ALREADY running, containers write while the repair walks. A container can then create a new non-operator-owned file BEHIND the walk, after which the completion marker records complete over a tree that is not, and those stragglers are never repaired without a manual –force.

BOUNDED, not dismissed: after any successful pass the marker is valid for the scope fingerprint and the repair short-circuits without walking (marker_is_valid), so the window requires a stale-or-absent marker AND a live stack together. Declaring a new scope entry re-arms the marker, which is exactly how that combination arises in practice.

NOT DONE THIS ROUND, with the reason stated rather than hidden: the clean guard is a cheap marker-only probe mode on scripts/ownership_repair.sh that answers has-this-scope-been-repaired without walking the tree. The existing –dry-run cannot serve: it deliberately SKIPS the marker fast-path (FORCE=0 && DRY_RUN=0 guard) and always walks, so using it as a warm-start probe would walk the tree twice on every start. Adding that mode requires editing ownership_repair.sh, which another stream was actively editing during this round; duplicating marker_is_valid into start.sh instead would be a near-identical fork (11.4.251). Deferred to this item rather than done unilaterally or silently.

The misleading comment at the warm-start call site (which claimed the gate runs before any container writes) has been corrected to state this boundary honestly.

Affected scope / file-scope manifest: start.sh (warm-start dispatch, run_ownership_gate call site); scripts/ownership_repair.sh (marker fast-path)

Reproduction / context: 1. Ensure the stack is UP. 2. Change config/owned_paths.yaml so the scope fingerprint changes (this re-arms the marker by design, data-model E2). 3. Run a warm ./start.sh (no –recreate). 4. The ownership repair walks and chowns the declared tree while containers are still running and able to write into it.

Acceptance criteria: A warm ./start.sh cannot complete an ownership repair while a container that writes to a declared location is running. Either the repair is deferred with an actionable refusal naming –recreate, or the stack is quiesced for the walk. Asserted behaviourally in tests/unit/test_start_reload_recreate.sh alongside the existing PRECONDITION_BEFORE_DOWN / REPAIR_AFTER_DOWN / REPAIR_BEFORE_UP checks, each with a paired 1.1 mutation that kills it.

BOB-160 — tests/pre_build/ and tests/ownership/ ran by nothing — closed by extending invariant 30 + wiring ci.sh

Status: Queued **Type:** Task **Severity:** Medium **Created-By:** AI

Reported-Via: §11.4.202 reporting directive `task` on 2026-08-21T19:02:48Z **Reported-By:** AI

What (the report, verbatim): Independent §11.4.209 review (IMPORTANT-3) found two of the feature's strongest automated checks were never executed by any runner: (1) tests/ownership/test_container_writes_owned_files.py — the §11.4.115 RED-turned-regression-guard for FR-011/FR-007, proven via a docker-compose.yml revert mutation — was moved out of tests/integration/ (commit 58d340a, to escape an autouse fixture hang) but no runner (ci.sh, test.sh, run-all-tests.sh) was ever extended to cover tests/ownership/. (2) tests/pre_build/test_*.sh, the §1.1 paired-mutation meta-tests for the scripts/pre_build/check_cm_*.sh gate family, was executed by NOTHING — self-reported honestly in commit 04742d7's own message ('STILL OPEN (reported, not fixed): tests/pre_build/ is executed by NOTHING') but never tracked as a workable item (a §11.4.197 loss-of-requirements gap) and never wired into scripts/pre_build_verification.sh invariant 30, which globbed only tests/unit/test_*.sh.

Affected scope / file-scope manifest: ci.sh; scripts/pre_build_verification.sh invariant 30 (CM-BASH-UNIT-TESTS-EXECUTED)

Reproduction / context: grep -rn test_container_writes_owned_files ci.sh test.sh run-all-tests.sh scripts/ returned zero runner hits (only docs/specs mentioned the path); grep in scripts/pre_build_verification.sh showed invariant 30's for-loop globbing only tests/unit/test_*.sh, never tests/pre_build/test_*.sh.

Acceptance criteria: ci.sh gains a runtime-gated pytest tests/ownership/ stage (skips honestly, rc=0, when no podman/docker present); scripts/pre_build_verification.sh invariant 30's glob covers both tests/unit/test_*.sh and tests/pre_build/test_*.sh, verified by an extracted standalone run of the invariant's exact block reporting a non-zero RAN count that includes files from both directories.

BOB-161 — The §11.4.69 CM-NO-FAIL-OPEN-SKIP gate is mandated but does not exist in this project

Status: Queued **Type:** Task **Severity:** High **Created-By:** BOB-092 remediation, residual finding

Reported-Via: §11.4.202 reporting directive `task` on 2026-08-21T19:31:12Z **Reported-By:** BOB-092 remediation, residual finding

What (the report, verbatim): §11.4.69 names CM-NO-FAIL-OPEN-SKIP as one of three mandatory pre-build gates: it audits sink-side probe helpers and FAILs if any code path converts an empty or unreachable response into a PASS-counting SKIP for a feature class with a sink-side probe. This project does not have it.

Why this matters now rather than in the abstract: the BOB-092 remediation just removed TWO fail-opens of exactly this class from tests/e2e/test_live_stack_evidence.py — the nnmclub SKIP-on-404 (already removed at 7baef2b) and a surviving sibling at test_iptorrents_is_authenticated_in_search that skipped on ‘not authenticated and status != success’ while asserting ‘treating as transient outage’. That assertion was false for rejected credentials (upstream_http_403) and for a broken container (plugin_env_missing), both of which are definitive product failures the product already classifies via error_type.

Two fail-opens of one class, found one at a time, is the §11.4.146 extend-to-all-cases signal and the §11.4.238 coverage-escape signal together: the regime did not surface either, an agent reading code did.

The remediation’s own guards are AST-structural and lethal under three discriminating mutations, but they LIVE IN THE FILE THEY GUARD, so deleting that file evades them entirely. A pre-build gate is the durable home because it audits the corpus rather than one file.

Honest boundary (§11.4.6): this item does NOT claim the two remediated fail-opens are unguarded — they are guarded, with runtime evidence (13 passed in 97.36s against the live stack). It claims the CLASS has no corpus-wide detector, so the next instance in a different file is invisible again.

Affected scope / file-scope manifest: scripts/pre_build/ (gate absent); tests/e2e/test_live_stack_evidence.py (guards currently live inside the file they guard)

Reproduction / context: grep -rl 'CM-NO-FAIL-OPEN-SKIP' scripts/ tests/ returns exactly ONE hit, and it is tests/e2e/test_live_stack_evidence.py — the file the guards live in, not a gate. Control needle: the same query for CM-OWNERSHIP-INVARIANTS (a gate that does exist) returns 3 files, so the instrument can see gate tokens and the single hit is a real absence, not a blind zero (§11.4.201(7)(b)).

Acceptance criteria: A scripts/pre_build/ gate named CM-NO-FAIL-OPEN-SKIP exists, is wired into scripts/pre_build_verification.sh, and audits sink-side probe helpers for code paths converting an empty/unreachable/error response into a PASS-counting SKIP for a feature class that HAS a sink-side probe. It ships a golden-TRUE fixture (a real fail-open -> gate FIRES) and a golden-FALSE-with-carrier (an honest topology/geo skip, and a comment merely MENTIONING the phrase -> gate MUST NOT fire), per §11.4.107(10)/§11.4.201(1). Paired §1.1 mutation makes the gate FAIL before the gate is trusted.

BOB-162 — Two new guards exist but no seam invokes them — plus the brownfield adoption decision the commit guard needs

Status: Queued **Type:** Task **Severity:** Medium **Created-By:** BOB-106/BOB-107 remediation, residual

Reported-Via: §11.4.202 reporting directive `task` on 2026-08-21T19:35:02Z **Reported-By:** BOB-106/BOB-107 remediation, residual

What (the report, verbatim): The TESTS for both guards are now executed: pre-build invariant 30's glob was extended from tests/unit + tests/pre_build to also cover tests/hooks. MEASURED before/after: the glob expanded to 27 suites with 0 under tests/hooks while 3 existed there; it now expands to 30 with 3 under tests/hooks. All three pass.

But the GUARDS themselves are still invoked by nothing.

check-brief-inputs.sh is honestly conductor-run rather than automatic: a brief's required-input list lives in free-form prose that the Agent tool's structured input does not expose, so an automatic prompt-scraping gate would itself be an unproven pattern-match. Its seam is the dispatch procedure, and that is a documentation + habit change, not a hook.

unattributed-commit-guard.sh is different and is why this item exists. Run against real history it names 14 violating commits since the last tag (and 21 bare Auto-commit subjects exist across all refs). Wiring it into the pre-build or commit seam AS-IS would therefore refuse every commit immediately, on pre-existing debt nobody can fix in the moment.

That is precisely the 11.4.224(E) brownfield-adoption shape, and 11.4.66/11.4.122 put the choice with the operator, not with an agent inventing a ratchet. The options, stated so the decision is a decision and not a default: (a) immediate hard floor – the seam refuses until all 14 are attributed; (b) one-time monotone-decrease ratchet (the 11.4.135 pattern) – snapshot 14 as the baseline, the count may fall and may never rise, so day one is green and the disease cannot spread; (c) changed-commits-only – gate only commits created from now on, with a scheduled deadline for the historical 14; (d) report-only for a stated period, then escalate.

Recommendation, with the reasoning rather than just the pick: (b). It is the pattern this constitution already uses for exactly this situation, it makes the debt visible and bounded immediately, and it cannot be satisfied by deleting the guard's name (11.4.227 closes that gaming channel). But it is the operator's call and this item is not closed until that answer is recorded.

Honest boundary (11.4.6): this item does NOT claim the 14 commits are harmful in themselves – it claims their ATTRIBUTION is absent and that nothing currently prevents the 15th.

Affected scope / file-scope manifest: scripts/hooks/unattributed-commit-guard.sh; scripts/hooks/check-brief-inputs.sh; scripts/pre_build_verification.sh; scripts/commit-push-all.sh

Reproduction / context: Both guards run correctly by hand and are covered by passing tests (9/9 and 15/15, each proven non-bluff against a no-op stub). Neither is invoked by scripts/

pre_build_verification.sh or scripts/commit-push-all.sh, so neither exerts standing detection pressure (11.4.226: a guard with no execution seam is not coverage; registration is not coverage).

Acceptance criteria: Each guard is invoked by a named seam, OR carries a registered deferral pointing at this item. For unattributed-commit-guard.sh specifically, the operator has answered the adoption question below and the answer is recorded as consumer DATA – never an invented ratchet.

BOB-163 — Now that :7187 really rate-limits, the DDoS challenge's cross-endpoint isolation assertion reads a sibling 429 as endpoint-degraded

Status: In progress **Type:** Bug **Severity:** Medium **Created-By:** BOB-114 remediation, pre-existing defect surfaced by BOB-111 landing a real limiter

OPERATOR DECISION (2026-08-26, §11.4.66 interactive clarification): YES — 429 IS RESPONSIVE IF Retry-After IS WELL-FORMED

A 429 carrying a VALID Retry-After header counts as the sibling endpoint being RESPONSIVE. Assertion (c) of the DDoS challenge accepts 2xx OR a well-formed 429 (positive integer seconds or valid HTTP-date — PARSED, not merely present/non-empty). DEGRADED remains: connection failure, timeout, any 5xx, and a 429 with absent or malformed Retry-After. RESIDUAL RISK, accepted knowingly and to be stated in the script source per §11.4.6: a genuinely wedged endpoint that happens to answer 429-with-Retry-After would pass — the Retry-After parse narrows that window, it does not close it. Options (b) limiter-aware drain and (c) exempt healthz probe were both offered and NOT chosen. STILL OPEN, not covered by this decision: the detector counts 429 only, not 503, though the script header says '429 (or equivalent)'; counting 503 would collide with the crash detector's 5xx tally. Decide when BOB-111 lands limiters on :7185 and :7189.

This answer is recorded as consumer DATA per §11.4.35 — it is the operator's stated choice, not an agent inference, and supersedes any prior agent-chosen default on this question.

Options not chosen are named above so a future reader does not re-litigate a settled call (§11.4.112(5) bounded-verdict discipline applied to decisions).

— prior item text follows —

Reported-Via: §11.4.202 reporting directive bug on 2026-08-21T19:41:08Z **Reported-By:** BOB-114 remediation, pre-existing defect surfaced by BOB-111 landing a real limiter

What (the report, verbatim): PRE-EXISTING, NOT INTRODUCED – and proven so rather than asserted: git diff shows assertion (c) is BYTE-IDENTICAL to HEAD, and the failure reproduces against the unmodified HEAD script. What changed is not the challenge but the SYSTEM: BOB-111 appears at least partially delivered, because :7187 now returns real 429s where it previously returned none. :7185 and :7189 still produce zero 429s.

This is a 11.4.248-class corrosion risk and that is why it is filed rather than left as a footnote: run_all_challenges.sh will now fail INTERMITTENTLY, and an intermittent red trains everyone to re-run until green, at which point a real regression is dismissed as ‘probably the flaky rate-limit one’.

THE DECISION, which is an operator call under 11.4.66 and which I deliberately did NOT invent:

Does a 429 from a WORKING limiter count as the sibling endpoint being RESPONSIVE?

1. YES – a 429 proves the endpoint is alive and correctly protecting itself. Assertion (c) accepts 2xx OR 429, and only a connection failure / 5xx / timeout counts as degraded. Risk: a genuinely wedged endpoint that happens to answer 429 would pass.
2. NO – keep requiring 2xx, but make the challenge limiter-aware: drain or wait out the window before the sibling probe (retry-after is served, so the budget is knowable rather than guessed).
3. Probe siblings on a path the limiter exempts (a healthz-class route), so the isolation question is asked without spending the bucket.

Recommendation with reasoning, not just a pick: (a) composed with a bounded liveness check – a 429 carrying a well-formed Retry-After IS evidence of a live, correctly-behaving service, and

(b) makes the challenge slower and couples it to a window value that will drift. But the semantics of ‘responsive’ here are a product judgement, so the answer is recorded, not assumed.

RELATED, stated as fact rather than folded in: the detector counts 429 only, not 503, though the script header says ‘429 (or equivalent)’. Counting 503 would collide with the crash detector’s 5xx tally. Worth deciding when BOB-111 lands limiters on :7185 and :7189.

Affected scope / file-scope manifest: challenges/scripts/ddos_resilience_challenge.sh assertion (c) cross-endpoint isolation; run_all_challenges.sh which invokes it

Reproduction / context: Assertion (c) requires a sibling-endpoint probe to answer ^2 (a 2xx). :7187 now enforces a real limiter (measured live: x-ratelimit-limit: 120, x-ratelimit-remaining: 86, retry-after: 45, server: uvicorn). A full challenge run sends roughly 250 requests to :7187, so sibling probes fired during the OTHER endpoints’ tiers land on an exhausted bucket and receive 429 – which assertion (c) scores as ‘endpoint degraded’. Timing-dependent: one run measured PASS=5 FAIL=2; the UNMODIFIED HEAD script against the same live stack ~30s later measured PASS=7 FAIL=0 SKIP=2.

Acceptance criteria: The operator has answered the classification question below, the answer is recorded as consumer DATA, and assertion (c) implements it. The challenge then produces the same verdict across 3 consecutive runs against a rate-limited stack (11.4.50 deterministic consistency), and the fix ships a paired 1.1 mutation proving assertion (c) still catches a genuinely degraded sibling endpoint – narrowing it must not blind it (11.4.201(1)).

BOB-164 — Live dashboard fails WCAG AA colour contrast on 21 nodes — brand heading measures 1.43:1 against a 3:1 floor

Status: In progress **Type:** Bug **Severity:** Medium **Created-By:** BOB-110 UX-class coverage, discovered by the new axe-core suite on its first live run

Reported-Via: §11.4.202 reporting directive bug on 2026-08-21T19:56:53Z **Reported-By:** BOB-110 UX-class coverage, discovered by the new axe-core suite on its first live run

What (the report, verbatim): This is a REAL user-facing defect, not a test-tuning artifact, and it was found by the automated regime rather than by a human squinting at the page – which is exactly the 11.4.238 posture the project is aiming for.

The static-grep oracle would NOT have found it. Measured: the served root ships an empty pre-hydration, so any check reading the raw HTML audits a page nobody sees. The violation only exists in the hydrated DOM, which is why 11.4.170 requires a rendered oracle and forbids value-equality assertions as the proof a UI is correct.

Severity reasoning, stated rather than assumed: this is user-visible and affects the primary dashboard heading, but it degrades legibility rather than breaking function, and the surface is operator-facing rather than public. Medium, not High.

The failing test was left FAILING on purpose (11.4.238) instead of silenced or marked xfail. tests/ux/ currently reports 1 failed, 16 passed; that 1 is this defect. Anyone reading a red UX suite should read it as this item, not as flakiness – and when this is fixed the suite goes fully green, which is the signal that it is closed.

HONEST SCOPE LIMIT (11.4.6): only the dashboard landing view was scanned. The /jackett/* sub-routes and the ng-serve-hosted :4200 route set were NOT audited – the commands to close both are recorded in docs/testing/ux_accessibility.md. So this item's 21 nodes are a floor, not a total.

Affected scope / file-scope manifest: the Angular dashboard served at http://localhost:7187/ (same compiled SPA as frontend/); production component CSS, not test files

Reproduction / context: Run tests/ux/ test_live_dashboard_accessibility.py against the running merge service. axe-core v4.13.0, scanning a real Playwright-rendered JS-hydrated DOM, reports color-contrast violations on 21 nodes. Measured pairs: .brand / h1 text #9d001e on background #3c3f41 = 1.43-1.62:1 (WCAG AA large-text floor is 3:1); tagline #808080 on #3c3f41 = 2.68:1 (body-text floor is 4.5:1).

Acceptance criteria: axe-core reports ZERO color-contrast violations against the live rendered dashboard, with the fix made in production component CSS rather than by relaxing the assertion or excluding the rule (11.4.120: reconcile to the correct mechanism, never weaken the check). The existing tests/ux/ suite is the guard and already fails today, so the RED is captured – closure requires it flipping GREEN against the live surface, which is runtime-class evidence per 11.4.226.

BOB-165 — Every documented python3 -m pytest command fails at collection: user-site rpds carries a 3.13 ABI extension under python 3.14

Status: Queued **Type:** Bug **Severity:** High **Created-By:** surfaced while capturing closure evidence for BOB-092; diagnosed rather than worked around

Reported-Via: §11.4.202 reporting directive bug on 2026-08-21T20:00:08Z **Reported-By:** surfaced while capturing closure evidence for BOB-092; diagnosed rather than worked around

What (the report, verbatim): This is a STALE-ABI-AFTER-INTERPRETER-UPGRADE defect, not a missing package. The package is installed; its compiled half is built for the previous interpreter. That distinction matters because ‘pip install rpds-py’ style advice can appear to succeed while leaving the 313 artefact in place.

WHY IT WAS NOT NOTICED EARLIER, stated as fact: the paths that DO work all avoid system python3. ci.sh selects an interpreter through _select_python; the long-running suites in this session ran .venv/bin/python -m pytest and passed. So the automated regime is green while the DOCUMENTED operator command is broken – an 11.4.238-shaped gap, since the escape was found by an agent capturing evidence rather than by the regime.

WORKAROUND (measured, not theorised): PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 with the needed plugins passed explicitly (-p pytest_timeout …) avoids the schemathesis autoload that drags in jsonschema -> referencing -> rpds. That is a

workaround, NOT the fix: it silences the import path rather than repairing the install, and it will surprise the next person who runs the documented command verbatim.

RELATED, and worth deciding together rather than twice: BOB-154 wants .venv rebuilt on 3.12 to match the container (which runs CPython 3.12.13 while both system and venv run 3.14.6). There are therefore THREE interpreters in play. Whoever rebuilds the venv should confirm rpds resolves for the TARGET interpreter afterwards, or this same class reappears one directory over.

HONEST BOUNDARY (11.4.6): this item does NOT claim any test is wrong or any product code is broken. The product and the venv-run suites are unaffected. What is broken is the documented entry point.

Affected scope / file-scope manifest: host user site-packages (~/.local/lib/python3/site-packages/rpds/); every CLAUDE.md-documented 'python3 -m pytest …' invocation; any tooling that uses system python3 rather than .venv/bin/python

Reproduction / context: python3 -m pytest tests/e2e/ test_live_stack_evidence.py -q –import-mode=importlib -> ModuleNotFoundError: No module named 'rpds.rpds', raised during collection via the schemathesis plugin autoload -> jsonschema -> referencing._core -> rpds. MEASURED root cause: system python3 is 3.14.6, but ~/.local/lib/python3/site-packages/ rpds/ ships rpds.cpython-313-x86_64-linux-gnu.so – an extension built for 3.13. rpds/**init**.py does 'from .rpds import *', and a 313-tagged .so is not importable by 3.14, so the submodule genuinely does not exist for this interpreter. The venv is CORRECT and unaffected: .venv/lib64/python3/site-packages/rpds/ ships rpds.cpython-314-x86_64-linux-gnu.so and'.venv/bin/python -c import rpds' succeeds.

Acceptance criteria: python3 -m pytest tests/unit/ -v –import-mode=importlib – the command CLAUDE.md documents – reaches collection without ModuleNotFoundError, OR CLAUDE.md is corrected to document the supported runner explicitly. Either way the documented command and the working command agree (11.4.99: a guide that misleads is the documentation-layer equivalent of a PASS-bluff).

WORKAROUND SIDE EFFECTS MEASURED 2026-08-21 — the recipe is not free, and the item previously implied it was. PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 disables ALL plugin autoload, not just the broken one, so anything the suite silently relied on must be re-enabled by hand:

- It BREAKS 5 pre-existing env-dependent tests in tests/unit/test_plugin_rutracker.py — TestConfig::test_default_mirrors, test_env_mirrors_override, test_get_env_with_default, test_get_mirrors_from_env_empty, test_get_mirrors_from_env_whitespace — which need an autoloaded env plugin. PROVEN not caused by any in-flight change: running HEAD's OWN copy of that file under identical flags produces the same 5 failures.
- It makes –timeout=60 (set in pyproject.toml) an UNKNOWN ARGUMENT unless -p pytest_timeout is passed explicitly, so pytest.mark.timeout is silently inert under the naive recipe — a test believed to be time-bounded is not.
- With autoload ON and only schemathesis disabled (-p no:schemathesis), that same file is 96/96 green.

CONSEQUENCE FOR ANYONE USING THE WORKAROUND: `-p no:schemathesis` alone is strictly better than blanket autoload-disabling where it suffices, because it removes only the broken plugin. Where the blanket form IS used, -p pytest_timeout must be added or timeouts are inert, and the 5 env-dependent failures must be recognised as workaround artifacts rather than filed as defects. That last point is the real risk: the workaround manufactures failures that look exactly like product defects.

This does not change the root cause (the venv's rpds ships a cpython-313 ABI tag CPython 3.14 cannot import) or the fix (rebuild the venv — BOB-154). It documents that the interim recipe has a blast radius, so nobody reads a workaround artifact as a regression.

BOB-167 — Two SSE routes, one rate-limit class: /search/stream carries @_rl('sse_stream') but the sibling /theme/stream carries no limiter and falls to the 120/min default

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** Claude **Assigned-To:** Claude

WHAT. download-proxy exposes two Server-Sent-Events routes. They are the same expensive class — long-lived connections that hold a worker and a generator for their lifetime — but only one is rate-limit classed:

routes.py:801 @router.get('/search/stream/{search_id}') @_rl('sse_stream') <- classed routes.py:150 @router.get('/theme/stream') async def stream_theme(...) <- NO limiter decorator

_rl(cls) resolves to limiter.limit(limit_for(cls)), so /search/stream is bound to the sse_stream class while /theme/stream falls through to the application default. Measured live on the running stack: /api/v1/theme/stream reports x-ratelimit-limit 120.

PROVENANCE + A CORRECTION WORTH KEEPING (§11.4.6). This was surfaced by the BOB-109 scaling agent, whose report framed it as: 'sse_stream_limit_decorator is defined and imported/applied nowhere … SSE falls through to the 120/minute default: 24x less protected'. That framing is WRONG and was NOT filed as given. The agent searched for one symbol NAME; the wiring uses a different mechanism (@_rl('sse_stream')), and it IS applied — to /search/stream. Verified by reading routes.py:795-806 and the _rl helper at routes.py:46-51, with a control needle confirming other *_limit_decorator symbols show real usage in api/**init**.py so the zero-hit was not a blind search. The agent's MEASUREMENT (120 on /theme/stream) was correct and is what makes this a real finding; its MECHANISM was not. Both halves are recorded so the next reader does not re-derive the same wrong cause.

WHY IT MATTERS. An unclassed SSE endpoint is the cheapest way to pin server resources: each connection is held open, and the default class permits 120/min of them. The declared sse_stream class exists precisely because this route shape needs a tighter bound than ordinary GETs.

ACCEPTANCE. (a) A decision, recorded, on whether /theme/stream belongs in the sse_stream class or genuinely warrants the default — this is a policy question, not automatically a bug to patch. (b) If it belongs in sse_stream, the decorator is applied and a test drives BOTH SSE routes and asserts each returns its INTENDED class limit from x-ratelimit-limit, so a future route added without a class is caught. (c) A guard that enumerates SSE-shaped routes and fails on any that carries no explicit rate-limit class — the general form, so the third SSE route does not repeat this. (d) Honest boundary: this does not claim 120/min is exploitable in this deployment; with network_mode host and no reverse proxy every caller shares the 127.0.0.1 bucket, which BOB-111 measured and recorded separately.

NOT CLAIMED. No change made. The limit values were read from headers, never driven to exhaustion — the limiter is per-IP and shared with concurrent agents on this host (§11.4.119).

BOB-168 — run_all_challenges.sh lists scaling_horizontal_challenge.sh which does not exist on disk, so the runner references a challenge that can never execute

Status: In progress **Type:** Task **Severity:** Low **Created-By:** Claude **Assigned-To:** Claude

WHAT. scripts/run_all_challenges.sh:66 lists “scaling_horizontal_challenge.sh” in its challenge set. challenges/scripts/scaling_horizontal_challenge.sh does not exist:

```
$ sed -n '66p' scripts/run_all_challenges.sh
    "scaling_horizontal_challenge.sh"
$ ls challenges/scripts/scaling_horizontal_challenge.sh
  ls: cannot access ...: No such file or directory
```

VERIFIED independently, not taken on report — surfaced by the BOB-109 scaling agent and re-checked here by direct invocation.

WHY IT MATTERS. Whether this is cosmetic or a §11.4.201 gate-honesty defect depends entirely on how the runner treats a missing entry, and that is the first thing to determine: if it SKIPs silently, the challenge bank advertises coverage it does not have

(a §11.4.266 claim-vs-reality row with no passing challenge behind it); if it FAILs, the runner is permanently red for a reason unrelated to the system under test, which trains readers to ignore it. Neither outcome is acceptable; they need different fixes.

ACCEPTANCE. (a) Determine and record the runner's actual behaviour on the missing entry by invoking it, not by reading it. (b) EITHER author the challenge, OR remove the entry — with §11.4.124 discipline: check git history for whether it once existed and was deleted, since a silently-dropped challenge is the more interesting defect. (c) If the runner silently skips missing entries, that is its own finding: a missing challenge must be loud (§11.4.3 SKIP-with-reason at minimum), never absent-and-quiet.

SEVERITY. Low as a defect, but it sits on the challenge-coverage seam, so (c) may deserve its own item.

CORRECTION 2026-08-22 (§11.4.6) — THIS ITEM'S PREMISE WAS FALSE, and the error was mine. Recorded openly rather than quietly edited.

scaling_horizontal_challenge.sh EXISTS. It is at submodules/challenges/challenges/scripts/scaling_horizontal_challenge.sh, executable, 3107 bytes, dated Aug 7. So do all 14 other entries and the meta-runner. It was never deleted: added to the roster at 1c959ba, the challenge itself added in the submodule at 873c0b1, and a pickaxe search across both repositories and all branches finds ZERO deletions.

HOW I GOT IT WRONG. run_all_challenges.sh reads submodules/challenges/challenges/scripts/. I checked challenges/scripts/ — a DIFFERENT directory belonging to a DIFFERENT, glob-based runner. Two similarly-named directories; I measured the wrong one. That is a §11.4.201(9) field-identity error. Worse, I wrote that it was "verified by invocation": I had listed a directory and never run the aggregator, so the claim overstated the evidence class as well as getting the answer wrong.

THE LESSON, which generalises past this item. In the SAME commit I correctly caught a sibling agent making this exact class of error on BOB-167 — a zero-hit produced by searching one symbol name — and applied a control needle there. Then I omitted the needle one paragraph later on my own measurement. The distinction that would have caught it: A CONTROL NEEDLE PROVES THE INSTRUMENT CAN SEE; IT DOES NOT PROVE IT IS POINTED AT THE THING UNDER TEST. §11.4.201(7)(b) as

commonly applied guards blindness; it does not guard mis-aiming. The needle for THIS query would have been to resolve the path the runner itself reads, not to confirm that some directory listing works.

WHAT IS ACTUALLY WRONG, and it is worse than what I filed. The missing-entry path runs no challenges by definition, so it can be exercised safely. Run against a checkout where the submodule is absent — the ordinary state of a clone made without –recursive — the REAL runner (byte-identity sha-asserted, not a replica) reports:

```
PASS: 0   FAIL: 0   SKIP: 16   TOTAL: 16
OBSERVED EXIT CODE: 0
```

An ENTIRELY UN-RUN BANK REPORTS SUCCESS. Not one dangling entry tolerated — the whole suite silently passes without executing anything. That is §11.4.201(6) false-null verbatim, and it is observed-reachable rather than constructed (§11.4.115(G)).

REVISED ACCEPTANCE. (a) ANSWERED, though not as filed: the runner reports a loud, counted SKIP and then never blocks, because the exit expression reads only the FAIL count. (b) The dangling-entry half is VOID — nothing is dangling. (c) The real work is the exit contract: a roster entry the runner cannot execute must not be indistinguishable from a healthy run. See the decision recorded in docs/qa/BOB-168/ decision_missing_vs_skip.md.

RELATED, REPORTED NOT FIXED: scripts/ pre_build_verification.sh:1792 carries the same SKIP/MISSING conflation one level up, for the other runner. Filed separately.

BOB-169 — 286 of 326 exported .html docs are headless pandoc fragments with no DOCTYPE and no charset, so UTF-8 section marks and arrows render as mojibake when opened directly

Status: In progress **Type:** Bug **Severity:** Medium **Created-By:** Claude **Assigned-To:** Claude

WHAT. The §11.4.65 markdown-export mandate requires every in-scope doc to ship .html/.pdf twins. 286 of the 326 .html files under docs/ (excluding dist/ and node_modules/) are pandoc FRAGMENTS — they begin at

with no <!DOCTYPE>, no / , and critically no

. Census run 2026-08-21.

CONCRETE HARM, measured not assumed. Sample docs/ BOBA_DATABASE.html: DOCTYPE 0, charset 0, and 30 lines carrying non-ASCII — the distinct characters present are § — ’ ” ” →. With no charset declaration a browser opening the file directly (file:// or a plain static host sending no charset header) falls back to its default encoding, typically windows-1252, and renders § as Â§ and → as â†’. Those are not incidental characters in this corpus: every constitutional cross-reference

in these documents is a § literal, so the mojibake lands on the most load-bearing token in the text. Fragments also carry no viewport meta, so they do not scale on mobile.

HOW IT SURFACED. Chasing a CM-DOCS-CHAIN-ENGINE-VERIFY failure on the features-status context. docs_chain sync regenerated docs/features/Status.{html,docx,pdf} and the HTML diff was 183 insertions / 0 DELETIONS — the body was untouched and a full pandoc preamble was PREPENDED, proving the committed file had been a fragment. Its sibling docs/codegraph/Status.html already began with <!

DOCTYPE>, so two derivatives in the same docs_chain config were being produced in different modes. That mismatch is what made verify FAIL, and the gate's own message ('derived docs drift from .md sources') misattributes it: the content was not drifting from the source, the generator was inconsistent.

THE DETECTION GAP (§11.4.238). Pre-build invariant 16 CM-MARKDOWN-EXPORT-SYNC PASSED on the whole corpus — 'all in-scope docs have fresh .html/.pdf siblings'. It checks PRESENCE and FRESHNESS, never VALIDITY, so a 0-byte-preamble fragment

satisfies it exactly as a well-formed document does. This was found by reading a failing gate's diff, not by the regime: a §11.4.238 discovery-channel escape, and the missing check is the interesting half.

ROOT CAUSE IS NOT YET PROVEN (§11.4.6). Two generators exist — scripts/generate_markdown_exports.sh (invoked via workable-items-export.sh) and the docs_chain engine — and they demonstrably disagree on standalone vs fragment mode for the same source. WHICH one emits fragments, and whether it does so always or under specific flags, is NOT established here and must not

be assumed. Whoever takes this item determines it by invoking both on one fixture and comparing, before changing either.

ACCEPTANCE. (a) Determine by invocation which generator emits fragments and under what conditions; record it. (b) Make the emitting generator produce standalone documents (charset + viewport at minimum), so the two paths agree — §11.4.251: one artifact should not have two generators that disagree. (c) Regenerate the 286 affected files. (d) Extend CM-MARKDOWN-EXPORT-SYNC (or add a sibling) to assert VALIDITY, not just presence:

every exported .html declares a charset. Paired §1.1 mutation — strip the charset from one export and the gate must FAIL. Include a negative control so a legitimately standalone doc does not fire (§11.4.201(1)). (e) Honest boundary: this is about the HTML twins only; the .pdf twins embed their own encoding and are not implicated by this measurement.

ALREADY DONE. docs/features/Status.{html,docx,pdf} regenerated via 'docs_chain sync features-status' (evidence qa-results/docs_chain/20260821T202805Z);

verify –all now exits 0. That is 3 of the 289 files; the remaining 286 are untouched.

CORRECTION 2026-08-21 (§11.4.6) — recorded openly rather than quietly edited, per the convention BOB-136's own body establishes. An earlier revision of this item asserted: "the .pdf twins embed their own encoding and are not implicated by this measurement." That assertion is FALSE. It was written without probing a single PDF.

Probing docs/BOBA_DATABASE.pdf via pdftotext returns 'Â§ 5', 'Jackettâ€™s' and 'â†' — UTF-8 byte sequences (§ = 0xC2 0xA7)

decoded as latin-1 — while the SOURCE .md at the corresponding construct is clean UTF-8. So the corruption was introduced during export, not authored.

The PDF case is WORSE than the HTML case, not merely additional. An HTML fragment still holds correct UTF-8 BYTES on disk; a browser told the right encoding renders it correctly, so adding

fixes it with no re-render. In a PDF the mis-decoded characters are baked into the text layer as glyphs — no viewer setting recovers them, and only regeneration from clean source fixes it.

MECHANISM (hypothesis, NOT proven — §11.4.6): the PDF is plausibly rendered FROM the charset-less HTML fragment, so the missing declaration propagates into the PDF pipeline and freezes there. Consistent with both artifacts sharing one root defect, but the pipeline was NOT traced. Establish it by invocation before relying on it.

SCOPE NOT MEASURED: exactly ONE pdf was probed. The 286-file census covered .html only. How many of the ~300 PDFs carry baked mojibake is UNKNOWN and must be COUNTED, not extrapolated from a single sample. Acceptance (e) is

therefore REPLACED: it previously scoped PDFs out; it now requires the PDF corpus to be counted and regenerated alongside the HTML.

Evidence: docs/qa/BOB-169/pdf_mojibake_correction_20260821.log

ROOT CAUSE PROVEN 2026-08-21 — acceptance (a) is SATISFIED, do not repeat it. Full evidence: docs/qa/BOB-169/root_cause_proven_20260821.md

THE GENERATOR: scripts/generate_markdown_exports.sh:57 runs

```
pandoc -f markdown -t html5 -o "$html" "$md" --metadata
```

`title=...` with NO –standalone/-s, so pandoc emits a body fragment with no DOCTYPE, no head, and no . A tell that the flag was intended and lost: –metadata title= is passed on that same line, and a title can only render inside a standalone document’s

, challenge markers 0 GET /forum/tracker.php?nm=debian -> 403, 5351 B, challenge markers 1, LOGIN markers 0, trs-tr- 0

The zero login markers are the load-bearing detail: the server is not asking us to authenticate, it is refusing the request before authentication is considered. Supplying valid

credentials or a fresh cookie jar therefore does NOT address this.

WHY IT MATTERS. rutracker is one of the three trackers the merge service fans a query across (with kinozal and nnmclub). A 403 on its search path means it contributes nothing to every merged result set, silently — the fan-out still 'succeeds', it just returns one fewer tracker's worth of results. From a user's seat this looks like thin results, not an outage.

IT EXPLAINS TWO PREVIOUSLY-UNEXPLAINED OBSERVATIONS. docs/qa/BOB-093/live_search_smoke.txt

records rutracker returning status=empty, results_count=0 in 164ms, and the BOB-136 adoption audit reached the same observation independently. Both recorded the symptom; neither had the cause. 164ms is far too fast for a real search across a remote forum and is exactly what an immediate 403 costs.

IT ALSO BLOCKS BOB-093's FOURTH SUB-STEP. That item requires timing a large rutracker result page under 2s. No large page has ever been obtained, and this is why. That sub-step is not merely waiting on credentials, as

previously assumed — it is unmeetable until the 403 is resolved.

PROVENANCE, INCLUDING A CORRECTION I MADE MID-INVESTIGATION (§11.4.199). An independent reviewer reported a Cloudflare challenge served to curl even with cookies. Verifying, I probed index.php, got a clean 200 with no challenge markers, and was one step from recording their claim as REFUTED. That would have been wrong for a textbook reason: index.php is not the search path, so my probe never reached the precondition, and a repro that misses the precondition

proves nothing — it is not evidence of absence. Probing the actual search endpoint reproduced it immediately. The reviewer was substantially right; my probe was aimed at the wrong endpoint. Recorded because the near-miss is the instructive part, and because the two probes TOGETHER give a sharper result than either the original claim or my near-refutation: not ‘rutracker is behind Cloudflare’ (the index is open), but ‘the SEARCH endpoint specifically is refused’.

ACCEPTANCE. (a) Determine whether the 403 is permanent policy, rate/reputation-based,

or triggered by a client signature the plugin can legitimately present (user-agent, header order, TLS fingerprint) — by measurement, not assumption, and WITHOUT evasion techniques that would violate the site's terms. (b) Whatever the outcome, the failure must become LOUD: a tracker returning 403 must be reported as a tracker ERROR in the merge result, never folded into 'empty' — a silent contributor is the §11.4.201(6) false-null at the product layer, and it is what let this sit unexplained across two separate investigations. (c) A test asserting that a 403 from any tracker surfaces as an

error and not as zero results, with a paired §1.1 mutation.
(d) If the endpoint is genuinely unavailable to us, record that as an honest capability boundary (§11.4.112) and stop advertising rutracker as a live search source until it is — including in the README and the merge-service docs.

HONEST BOUNDARY.
Measured from ONE host, ONE time, UNAUTHENTICATED. Whether an authenticated session with a browser-like client succeeds was NOT tested and must not be assumed either way. The finding is that the current code path gets a 403; it is not a claim about what every client would get.

BOB-173 — Hook create and delete return HTTP success even when persistence fails, because _save_hooks swallows every exception — a user is told their webhook exists when it does not

Status: Ready for testing **Type:** Bug **Severity:** High **Created-By:** Claude **Assigned-To:** Claude

WHAT. download-proxy/src/api/hooks.py:96-102:

```
def _save_hooks(hooks: list[dict[str, Any]]) -> None:
    try:
        os.makedirs(os.path.dirname(HOOKS_FILE),
exist_ok=True)
        with open(HOOKS_FILE, 'w') as f:
            json.dump(hooks, f, indent=2)
    except Exception as e:
        logger.error(f'Failed to save hooks: {e}')
```

It catches EVERY exception, logs, and returns None. The signature returns None, so the caller has no channel to learn the write failed. Both call sites then report success unconditionally:

```
:152  _save_hooks(hooks)                     :174
_save_hooks(hooks)
:154  logger.info('Created hook: ...')       :175
logger.info('Deleted hook: ...')
:156  return HookResponse(hook_id=..., ...)  :176  return
{'message': 'Hook deleted', ...}
```

USER-VISIBLE CONSEQUENCE, which is why this is High and not a code-hygiene nit. A user POSTs a webhook, receives HTTP 200 and a hook_id, and the hook was never written — their automation silently never fires, and the API told them it exists. Symmetrically, a user DELETEs a hook, is told ‘Hook deleted’, the file still holds it, and it fires again after the next restart. In both directions the product reports the opposite of what happened, and the only trace is a log line nobody is watching.

THIS IS THE §11.4.252 SHAPE. The path combines two dangerous capabilities from that anchor’s taxonomy — MUTATION of a shared resource (a filesystem write) and EXTERNAL SIDE EFFECT (hooks are outbound calls the system will or will not make) — so it is required to FAIL CLOSED: verify the precondition, refuse

when it cannot be satisfied, and surface the refusal. Instead it fails open, and a bare ‘except Exception:’ that only logs is the exact anti-pattern §11.4.252 enumerates. At the product layer it is also a §11.4.201(6) false-null: a successful write and a swallowed failure are indistinguishable to the caller.

PROVENANCE. Surfaced as an out-of-scope observation by the independent reviewer of the BOB-135 test-isolation work — this swallow is what turned that defect into an ‘assert 0 == 1’ mystery, because the EACCES on /config was logged and discarded while the endpoint kept returning 200. Verified here directly from source before filing (the function body and both call sites read above), not taken on report. Recorded as a §11.4.238 discovery-channel escape: found by an agent reading code during an unrelated investigation, not by the automated QA regime — the coverage gap is a defect of equal standing to the defect itself.

ACCEPTANCE. (a) _save_hooks propagates failure — raise, or return a status the callers must consume. (b) Both endpoints translate a persistence failure into an HTTP error (500-class), never a success body; a create that did not persist must not return a hook_id. (c) A test drives each endpoint with the hooks file unwritable (read-only dir or a patched open raising OSError) and asserts a non-2xx status AND that a subsequent GET does not list the phantom hook — assert on the user-observable outcome, not on the log line. (d) Paired §1.1 mutation: restore the swallow; the test must FAIL. (e) Audit the same file for sibling swallows — this is a pattern, and one instance is rarely alone. (f) Honest boundary: this does not claim the write currently fails in production; it claims that WHEN it fails the user is told the opposite, and the BOB-135 investigation shows it does fail in at least one real environment.

NOT CLAIMED. No change made. No assessment of how often the write fails in the operator’s deployment.

RECORDING A SHELL ERROR OF MY OWN (§11.4.6): the first version of this description was written with the anti-pattern snippet inside backticks in a double-quoted shell argument, so the shell ran it as command substitution and the text was replaced by nothing — the stored description read ‘and is the exact anti-pattern’. This is the SECOND time this session that backticks-inside-double-quotes has corrupted content (the first mangled a commit message). Fixed here by editing through a Python client with no shell quoting in the path. Noted because a silently-

truncated defect description is exactly the kind of quiet corruption §11.4.201(7)(c) warns about — the path is part of the instrument, and it failed without erroring.

BOB-174 — A corrupt hooks file reads as zero hooks and the next create silently destroys every existing hook, while the non-atomic write manufactures the corruption

Status: In progress **Type:** Bug **Severity:** High **Created-By:** Claude **Assigned-To:** Claude

WHAT. A corrupt hooks file is silently indistinguishable from “no hooks configured”, and the next create then DESTROYS every existing hook while returning HTTP 200. Three defects on one path, filed together because fixing any one alone leaves the data loss reachable.

A1 — download-proxy/src/api/hooks.py:104-112. _load_hooks wraps the read in `except Exception: logger.error(...)` and falls through to `return []`. A truncated or malformed hooks.json is therefore reported to every caller as an empty, healthy hook list. Confirmed at source.

A2 — the consequence, and the reason this is High. create_hook loads, appends, saves. Given a corrupt file that load turns into [], the save writes a one-element list over the top. Every previously-configured hook is gone. Measured by the implementing agent against the ALREADY-FIXED tree, so this survives the BOB-173 write-failure fix:

```
BEFORE  on disk : ['prod-hook-0', 'prod-hook-1', 'prod-
hook-2']
GET     reports : 200 {'hooks': [], 'count': 0}    <- claims
ZERO hooks configured
DELETE  reports : 404 {'detail': 'Hook not found'} <- for a
hook that IS in the file
POST    reports : 200 hook_id=3c8a5930-...
AFTER   on disk : ['3c8a5930-...']
```

Three prod hooks destroyed, HTTP 200 throughout, nothing surfaced to the user.

A5 — _save_hooks writes with a plain `open(path, "w")`. A crash, ENOSPC, or a kill mid-write truncates the file in place. That is precisely the corruption A1 then reads as “no hooks” and A2 overwrites. The same codebase already has the correct pattern: theme_state.py:115-126 uses tmp-file + os.replace.

WHY THE THREE ARE ONE ITEM. A5 manufactures the corrupt file, A1 misreads it as empty, A2 destroys the contents. Fixing only A1 leaves truncation reachable; fixing only A5 leaves an existing corrupt file a data-loss trigger; fixing only A2 leaves the API lying about what is configured. The chain is the defect.

DELIBERATELY NOT FIXED UNDER BOB-173, and the reasoning is sound. The implementing agent identified all three while auditing sibling swallows as that item required, and declined to fix them in the same change because: they change GET semantics on an endpoint the frontend consumes (200 -> 5xx); they need a CORRUPT-file reproduction rather than the UNWRITABLE-file one BOB-173 built; and they need a design decision that is genuinely not obvious — a MISSING hooks file legitimately means “no hooks configured”, while a CORRUPT one does not, and today’s code cannot tell those apart. Expanding BOB-173’s scope to cover them would have meant shipping that decision unexamined.

ACCEPTANCE. (a) Distinguish MISSING from CORRUPT: a missing file remains an empty list; a corrupt one is an error, never silently []. (b) Decide and RECORD what GET does on corruption — 5xx, or 200 with an explicit degraded marker the frontend can render. This is the design decision, and it should be stated rather than inferred from whatever the patch happens to do. (c) create/delete MUST NOT overwrite a file they could not parse — refuse, do not clobber (§11.4.252: mutation plus external side effect must fail closed). (d) Make the write atomic via tmp + os.replace, reusing theme_state.py’s existing pattern rather than re-inventing it (§11.4.28). (e) Tests asserting the USER-OBSERVABLE outcome, not log lines: seed a corrupt file, assert GET does not claim zero hooks, assert a create refuses rather than destroying, and assert the file still holds the original hooks afterwards. (f) Paired §1.1 mutation per guard. (g) NEGATIVE CONTROLS (§11.4.201(1)): a MISSING file must still yield an empty list and a working create; a VALID file must behave exactly as today. A fix that makes every load fail closed by failing always is not a fix.

A3, RECORDED SEPARATELY, NOT PART OF THIS CHAIN. VALID_EVENTS and HookEventType are two sources of truth for one closed set. Verified identical today, unguarded against drift: if they diverge a hook registers successfully and then silently never fires. One assertion pinning them would close it.

NOT CLAIMED. No change made. The BOB-173 write-failure fix is real and orthogonal — it makes a FAILED write honest; it does nothing about a SUCCESSFUL write of wrong data derived from a misread file.

BOB-175 — update –location Fixed –status can still mint a row that update's own validator rejects, because the status-location guard is one-directional

Status: Queued **Type:** Task **Severity:** Low **Created-By:** Claude **Assigned-To:** Claude

WHAT. The workable-items `update` seam guards the status-location invariant in one direction only. After BOB-166, `update --status <terminal>` on an Issues-located row is refused. The mirror is still open: `update --location Fixed --status <non-terminal>` exits 0 and creates a row that `update`'s own validator immediately rejects.

Verified empirically by the independent reviewer of BOB-166:

```
update --location Fixed --status 'In progress'   -> exit 0
validate                                          -> FAILS,
naming the row (check (f), fixedLocationNonTerminalStatus)
```

So the command can still mint a state its own validate call refuses. That is the §11.4.196(F) shape at the seam layer: the invariant is CONFIGURED in validate but not ENFORCED at every write path that can violate it.

WHY IT WAS DELIBERATELY LEFT OPEN IN BOB-166, and why that was right. Three reasons, all checked rather than asserted: (1) the real corpus has ZERO instances of this direction against TEN of the direction BOB-166 closed — the asymmetry in the data matched the asymmetry in the code; (2) detective coverage ALREADY existed and demonstrably fires, so unlike BOB-166's direction this cannot rot silently; (3) and the load-bearing one —

`reopenCmd` documents and SANCTIONS a transient Fixed-plus-non-terminal state via its `--location Fixed` operator override. A naive preventive guard at the update seam would collide with that override's semantics. Closing this therefore requires a design decision about how the two interact, which is a different question from the one BOB-166 was scoped to answer, and expanding that item would have meant shipping the decision unexamined.

ACCEPTANCE. (a) Decide and RECORD how a preventive guard on this direction coexists with reopenCmd's sanctioned override — this is the actual work, and the answer should be written down rather than inferred from whatever the patch does. Candidates worth weighing: exempt the reopen path explicitly, require an override flag on `update` mirroring reopen's, or leave the direction detective-only with the rationale recorded so the asymmetry is a decision rather than an oversight. (b) If a guard lands, it reuses `terminalStatuses()` — BOB-166 established one predicate source specifically so the two directions cannot drift. (c) Paired §1.1 mutation. (d) NEGATIVE CONTROL (§11.4.201(1)), and it is the sharp one here: the sanctioned reopen path MUST still work. A guard that breaks reopenCmd's documented override is a false-positive refusal, and worse than the gap it closes, because it breaks a workflow operators are told to use.

HONEST BOUNDARY. This is not a live data defect. Zero corpus instances, and the detective gate catches it at the next validate. It is filed so a scoped, deliberate omission stays visible as a tracked decision instead of living only in a code comment where the next reader will mistake it for an oversight (§11.4.197: a started thread reaches a documented terminal state, never quiet abandonment).

PROVENANCE. Raised by the implementing agent of BOB-166 as a known asymmetry it deliberately did not close, independently verified and endorsed as an acceptable scope boundary by that item's reviewer, on the condition that it become a tracked item rather than a comment. This is that item.

BOB-176 — A cookies-only rutracker configuration never enables the tracker, because the enablement gate checks username/password while the search path prefers cookies

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** Claude **Assigned-To:** Claude

WHAT. _get_enabled_trackers() in download-proxy/src/merge_service/search.py gates rutracker on RUTRACKER_USERNAME and RUTRACKER_PASSWORD only. It never consults RUTRACKER_COOKIES. So an operator configured with cookies alone — no username, no password — never has rutracker enabled at all, and the tracker is silently absent from every fan-out rather than failing visibly.

WHY THAT IS INCOHERENT WITH THE REST OF THE CODE. _search_rutracker treats COOKIES as the PREFERRED auth path, not a fallback. And the nnmclub sibling DOES check its own *_COOKIES variable. So the same codebase holds three positions at once: cookies preferred at the search site, cookies ignored at the enablement gate, and cookies honoured for a sibling tracker.

WHY IT MATTERS MORE THAN IT LOOKS. CLAUDE.md documents a standing operator mandate (2026-08-15) that per-tracker Netscape cookies files at ${TRACKER_COOKIE_DIR}/cookies_.txt are auto-loaded into .env as _COOKIES before every boba-svc up, restart, install Stage 6 and start.sh boot — and rutracker is named in that set. So the SANCTIONED, documented configuration path for this tracker produces exactly the state this gate ignores. An operator who follows the documented instructions gets a tracker that never runs, with no error to explain it.

FAILURE MODE. Silent absence, not a visible failure — the same §11.4.201(6) false-null shape as BOB-172, one layer earlier. BOB-172 fixes a tracker that RAN and was REFUSED being reported as empty; this is a tracker that never ran at all, and the merged result simply has one fewer contributor with nothing to indicate it.

PROVENANCE. Found by the BOB-172 implementing agent while tracing the enablement path to locate the status-handling defect. Recorded here rather than fixed in that change: it is a separate defect on a separate seam with its own oracle, and folding it in would have widened BOB-172 past its captured evidence.

ACCEPTANCE. (a) The enablement gate honours RUTRACKER_COOKIES as an independently sufficient credential, matching what _search_rutracker already prefers and what the nnmclub sibling already does. (b) A test asserting that a cookies-only configuration ENABLES rutracker, driving the real _get_enabled_trackers rather than a replica. (c) Paired §1.1 mutation restoring the username/password-only gate — the test must go red. (d) NEGATIVE CONTROL (§11.4.201(1)): a configuration with NO credentials at all must still leave rutracker DISABLED. A fix that enables an unauthenticated tracker is worse than the gap, because it produces failing searches instead of absent ones. (e) Audit the other trackers' gates for the same asymmetry rather than fixing only the one that was noticed — three positions in one codebase suggests nobody has checked them together (§11.4.118).

HONEST BOUNDARY. Not verified against a live cookies-only deployment; read from source by the BOB-172 agent and recorded on its report. The reading is specific and checkable, but whoever takes this should confirm by invocation before relying on it.

BOB-177 — Four of five private-tracker HTTP-refusal guards are invisible to the test suite

Status: Queued **Type:** Bug **Severity:** High **Created-By:** Claude **Assigned-To:** Claude

WHAT: BOB-172 wired an identical 4-line HTTP-refusal guard at five private-tracker fetch sites in download-proxy/src/merge_service/search.py (rutracker cookie :1390, rutracker credential :1468, kinozal :1648, nnmclub :1761, iptorrents :1934). Only the rutracker COOKIE path is exercised by tests.

EVIDENCE (reviewer-authored mutation R1, per 11.4.194(6)(d), during the BOB-172 independent review): deleting ONLY the kinozal guard wiring (search.py:1655-1658) while leaving the classifier intact left the full merge_service suite at 883 passed, ZERO failures. The suite cannot see four of the five sites.

FAILING SCENARIO: a refactor drops or subtly breaks the wiring at kinozal, nnmclub, iptorrents, or rutracker-credential. A 403 at that site silently folds back to status=empty with error=None – the exact BOB-172 signature – and no test reddens.

FIX DIRECTION (reviewer preferred): collapse the five duplicated wirings into one shared helper,
e.g. _check_search_response(tracker_name, status, body) -> bool, so there is ONE copy to test and a sixth site cannot be added unguarded (11.4.251 byte-identical-fork extraction). Alternative: parametrise the guard tests across all five sites with per-site stub sessions.

ACCEPTANCE: mutating the wiring at ANY of the five sites reddens at least one test. Prove it by running the same R1 deletion at each site in turn.

BOB-178 — Kinozal login-leg HTTP failure sets no diagnostic, reproducing the BOB-172 false-null one leg over

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** Claude **Assigned-To:** Claude

WHAT: download-proxy/src/merge_service/search.py:1638-1640 handles a failed kinozal LOGIN response with if login_resp.status not in (200, 301, 302): logger.error(…); return [] – and sets NO diagnostic.

FAILING SCENARIO: Cloudflare returns 403 on takelogin.php. The kinozal chip reads status=empty, error=None. That is the BOB-172 signature exactly: a refusal reported to the user as an empty result set.

CONTRAST establishing this is an oversight, not a design choice: the rutracker and nnmclub login failures DO set diagnostics (upstream_captcha / auth_failure), and the iptorrents login failure falls through to the search fetch where BOB-172's new guard catches it. Kinozal is the one leg with neither.

FIX DIRECTION: stash _classify_upstream_http_status(login_resp.status, "") before the early return, or an auth_failure diag for the status-in-(200,301,302)-but-no-cookie case.

ACCEPTANCE: a stubbed 403 on the kinozal login endpoint produces a non-None error on the kinozal chip, with a RED captured against the pre-fix code first (11.4.115).

BOB-179 — Two adjacent tracker false-null classes remain open and must be stated as gaps, not implied closed

Status: Queued **Type:** Task **Severity:** Medium **Created-By:** Claude **Assigned-To:** Claude

WHAT: the BOB-172 independent review demonstrated two further paths that still report a refusing or unreachable tracker as an empty result. Both are PRE-EXISTING and neither is a regression from BOB-172, but the fix evidence log presents the five-site coverage without stating the boundary (11.4.194(5) requires un-analysed dimensions be explicit gaps, never silently assumed safe).

GAP A – exception before resp.status is read. Reviewer probe B, captured: a connection-refused rutracker yields status=empty, error=None, http_status=None, metadata.errors=[], metadata.status=completed. Each *search** method swallows exceptions (except Exception: logger.error; return results) before _search_one's error handling can see them, so a tracker that is DOWN is indistinguishable from one that is genuinely empty.

GAP B – soft refusal at HTTP 200. Reviewer probe A, captured: _classify_upstream_http_status(200,) returns None, which is CORRECT by design (a 2xx is a usable response; body-marker triggering would risk the over-fire the negative controls exist to

prevent). But all five search GETs use aiohttp's default allow_redirects=True, so a session-expiry 302 -> login-page-200 chain parses to zero rows and reports empty.

FIX DIRECTION: Gap A needs the exception to reach a diagnostic rather than being swallowed. Gap B needs a DISTINCT detector (final-URL check or login-form marker) with its own RED – explicitly NOT a widening of the status-code trigger.

ACCEPTANCE: both gaps closed with their own REDs, or explicitly closed per 11.4.112 with evidence. Immediate sub-task: append a stated-gaps paragraph to docs/qa/BOB-172/fix_evidence_20260822.log.

BOB-180 — Zero-result search with any captcha-flavoured diagnostic emits a RuTracker-specific headline

Status: Queued **Type:** Bug **Severity:** Low **Created-By:** Claude **Assigned-To:** Claude

WHAT: download-proxy/src/api/routes.py:727-745 sets status=captcha_required with a hardcoded message naming RuTracker: 'RuTracker requires CAPTCHA. Use /api/v1/auth/rutracker/captcha'.

FAILING SCENARIO: a whole search returns zero results and the captcha-flavoured diagnostic came from NNMClub's Turnstile, not RuTracker. The user is told to visit a RuTracker captcha endpoint for an NNMClub problem.

SEVERITY BOUNDED: the full errors[] and tracker_stats travel in the same response payload, so the truth is present and a client that reads them is not misled – only the headline is wrong. The branch was revived by BOB-172's error propagation (correctly: suppressing that propagation would recreate the same false-null one layer up, 11.4.247) and is already covered at the routes layer by tests/unit/api_layer/test_routes_coverage.py:782.

FIX DIRECTION: derive the message from the tracker(s) that actually erred rather than hardcoding one.

SCOPE NOTE: api/ was owned by a sibling stream during BOB-172; the author was correct not to touch it.

ACCEPTANCE: an NNMClub-only captcha diagnostic on a zero-result search produces a headline naming NNMClub.

BOB-181 — Export generator silently ignores a path argument and always scans the whole tree

Status: Queued **Type:** Bug **Severity:** High **Created-By:** Claude **Assigned-To:** Claude

WHAT: scripts/generate_markdown_exports.sh accepts no arguments. It unconditionally scans PROJECT_ROOT/*.md, docs/ recursively and scripts/ recursively (scan loops at lines 105/111/116 as of 2026-08-23; they were 96-109 when this was filed — line numbers shifted with the BOB-169 fix, the substance is unchanged and was independently verified by execution: invoking the generator with an explicit path produced an out-of-scope export and NOT the requested one), and PROJECT_ROOT is derived from BASH_SOURCE (line 17). A caller who writes 'bash scripts/generate_markdown_exports.sh docs/qa/SOME/file.md' gets a WHOLE-TREE run with the argument silently discarded.

WHY IT MATTERS: the caller reasonably believes the run was scoped to one file. It was not. In a shared checkout with parallel work streams this regenerates any export whose .md is newer than its derived files anywhere in the tree – landing artifacts in other streams' scopes and, with the BOB-068 auto-commit hazard, into another stream's commit.

CLASS: a false affordance – the script's interface implies a capability it does not have (11.4.201: an interface must mean what it claims).

FIX DIRECTION: either accept optional path arguments and honour them (scoping the run to exactly those files), or REFUSE with a clear message when arguments are passed. Silently discarding them is the one option that must not remain.

ACCEPTANCE: passing a path either scopes the run to it, or exits non-zero naming the unsupported argument. A RED capturing today's silent-discard behaviour first (11.4.115).

LIVE INSTANCE, MEASURED 2026-08-23 (this is no longer hypothetical): During the BOB-169 fix the conductor invoked the generator with an explicit path to regenerate ONE document. The argument was discarded and the whole tree was scanned. It generated docs/qa/BOB-174/DESIGN_DECISION.{html,pdf,docx} at 11:50 – a SIBLING STREAM's document, while that stream's author was still editing the source (.md mtime 12:00). The exports were therefore stale on arrival, and additionally carried the pre-fix BOB-169 charset defect. The sibling author independently reported them as stale in its own hand-off, having no idea another stream had created them.

SECOND-ORDER LESSON worth keeping with this item: the conductor's attempt to verify the blast radius ALSO failed, and failed silently. 'git status –porcelain | grep -E ".(html|pdf)$"' returned only its own files and read as clean – but docs/qa/BOB-174/ is an UNTRACKED DIRECTORY, which porcelain collapses to a single line, so the grep was structurally incapable of seeing the files inside it (11.4.201(7)(a): matched the wrong surface; the quiet zero read as absence). The instrument that DOES see it is 'find

> -newermt '. Any future verification of 'which files did this run touch' must not use porcelain output as the corpus.

BOB-182 — Operator decision owed on the export-charset ratchet, plus an auto-lowering baseline

Status: In progress **Type:** Task **Severity:** Medium **Created-By:** Claude **Assigned-To:** Claude

OPERATOR DECISION (2026-08-26, §11.4.66 interactive clarification): KEEP THE MONOTONE RATCHET

§11.4.224(E) brownfield adoption answer, recorded as consumer DATA: the adoption model for CM-EXPORT-CHARSET-VALID is the MONOTONE-DECREASE RATCHET. The count may only go down, never up. Immediate hard floor, changed-code-only-with-deadline, and per-corpus phase-in were all offered and NOT chosen. This closes half of BOB-182 — the half that was genuinely the operator's. THE REMAINING HALF IS A REAL DEFECT STILL OWED:

BASELINE is a hardcoded constant, so the ratchet does not actually ratchet — on improvement the gate PASSES and PRINTS the value to lower it to, but nothing lowers it, and the gate keeps permitting regression back to the original count after the corpus heals. The fix must NOT collapse §11.4.249 role separation: a gate that writes its own threshold during a pre-build run becomes a PRODUCER as well as a GATE. Acceptable shapes include a persisted baseline the gate READS but never writes with lowering via a separate explicitly-invoked command, or a gate that FAILS loudly when the live count is below baseline so drift becomes an actionable refusal.

This answer is recorded as consumer DATA per §11.4.35 — it is the operator's stated choice, not an agent inference, and supersedes any prior agent-chosen default on this question. Options not chosen are named above so a future reader does not re-litigate a settled call (§11.4.112(5) bounded-verdict discipline applied to decisions).

— prior item text follows —

WHAT: CM-EXPORT-CHARSET-VALID (pre-build invariant 50) adopts its 301 pre-existing violations via a monotone-decrease ratchet rather than a hard floor. Two things are owed.

1. THE ADOPTION DECISION IS THE OPERATOR'S, NOT THE SCRIPT'S. 11.4.224(E) requires the brownfield adoption question be SURFACED to the operator and the answer recorded as consumer DATA – never an invented ratchet. What exists today is a header comment plus a BOBA_EXPORT_CHARSET_BASELINE override: that DOCUMENTS a decision the agent made, it does not RECORD an answer the operator gave. The independent reviewer judged the choice defensible and would not reverse it (a hard floor on 301 violations makes the build unreachable, which 11.4.234 forbids; 11.4.101 favours the reversible option) but correctly held that it is not yet compliant. Options for the operator: immediate hard floor (BASELINE=0) / keep the monotone ratchet / per-corpus phase-in / changed-code-only with a scheduled full-corpus deadline.

2. THE RATCHET DOES NOT ACTUALLY RATCHET. BASELINE is a hardcoded constant. On improvement the gate PASSES and PRINTS the value to lower it to, but nothing lowers it, so the gate keeps permitting regression all the way back to 301 after the corpus heals. The header now says tightening is MANUAL rather than claiming automatic behaviour that does not exist (11.4.6), but the honest statement is a stopgap, not the fix.

WHY IT WAS NOT BUILT WITH THE FIX: a gate that writes its own threshold during a pre-build run becomes a PRODUCER as well as a GATE (11.4.249 role separation), and that is a design change the operator should approve rather than receive as a side effect.

ACCEPTANCE: the operator's adoption answer recorded as consumer DATA; and either a persisted baseline the gate lowers and never raises (with the role-separation question answered), or an explicit decision that manual tightening is acceptable.

BOB-184 — Icon-glyph controls are unverified for non-text contrast because neither contrast oracle can measure them

Status: Queued **Type:** Task **Severity:** Medium **Created-By:** Claude **Assigned-To:** Claude

Three icon-glyph controls (.bridge-retry, .theme-toggle, .caret) render text made of non-BMP code points. axe-core declines to decide their contrast, reporting them under the incomplete channel with messageKey nonBmp and resolving NEITHER fgColor NOR bgColor, so the Python recomputation in docs/qa/BOB-164/axe_contrast_scan.py cannot decide either: 40 such nodes across 16 palette x mode combinations. BOB-164 round 2 stopped them being scored as silent passes by naming them GLYPH_UNMEASURED, printing them on every run and fencing them, so any NEW unmeasured glyph node blocks. What remains genuinely unproven is their contrast itself: as non-text user-interface components they owe 3:1 under WCAG SC 1.4.11, and no

oracle in this repo measures that today. Acceptance: a measurement path that resolves the effective foreground and backdrop for glyph controls (for example reading getComputedStyle colour against the composited backdrop, or replacing the glyphs with inline SVG carrying explicit fill tokens), each of the three controls shown to clear 3:1 in all 16 palette x mode combinations, and the GLYPH_UNMEASURED fence shrunk by exactly the nodes then proven.

BOB-185 — Rendered-DOM contrast oracle cannot see data-driven nodes, leaving badge and status pills arithmetic-only

Status: Queued **Type:** Task **Severity:** Medium **Created-By:** Claude **Assigned-To:** Claude

docs/qa/BOB-164/axe_contrast_scan.py scans a static dist with no backend, so every node whose existence depends on fetched data is absent from the page it measures: the results table renders empty, so .type-badge and .quality-badge never appear, and the hooks list renders empty, so .status pills never appear. That is precisely why the BOB-164 round-1 regression went unseen in a rendered DOM even though it was scanned in all 16 themes, and it is why round 2 had to close the class in the arithmetic oracle instead. Those nodes are now covered by frontend/src/app/models/style-contrast.spec.ts, which extracts declared foreground-on-fill pairs from the stylesheets, but that is an ARITHMETIC claim about declared colour pairs, not a rendered-pixel one: it cannot see opacity applied by an ancestor, a composited backdrop, or a cascade this repo’s static extractor does not model. Acceptance: the scan drives the dashboard with seeded fixture data (a stub backend, a route fixture, or an injected component harness) so the badge and status nodes are present in the DOM, axe measures them directly, and a control needle proves the rendered oracle sees a seeded sub-floor badge before any clean result from it is believed.

BOB-187 — ownership_precondition.sh is unfenced: same unreviewed .env-driven scope as the now-fenced repair, still writes probe files at any declared path

Status: In progress **Type:** Bug

scripts/ownership_precondition.sh consumes the SAME unreviewed .env-driven scope as ownership_repair.sh and creates probe files inside it, but it is NOT fenced. T028 remediation added ownership_path_fence() to scripts/lib/ownership.sh and wired it into ownership_repair.sh, closing the path-escape class there; the precondition was outside that agents file scope and still accepts whatever config/owned_paths.yaml expands to. Blast radius is lower than a recursive chown (it writes probe files rather than mutating ownership of an existing tree) but the INPUT is identical and equally unreviewed, so the same QBITTORRENT_DATA_DIR value that would have driven a filesystem-wide chown will drive probe-file creation at an arbitrary location. Note the escape is empirically confirmed for the repair path, not merely reasoned: during T028 remediation a probe declaring the verbatim shipped entry with QBITTORRENT_DATA_DIR=/ hung the suite, and ps showed the real artifact executing “find / (! -uid 1000 -o ! -gid 1000) -printf …” before it was killed by verified pid. Acceptance: ownership_precondition.sh calls the SAME ownership_path_fence() predicate from scripts/lib/ownership.sh (never a second dialect, 11.4.251), refuses fail-closed on a rejected entry, and ships a paired 1.1 mutation proving the refusal fires PLUS a negative control proving the six live shipped entries still ACCEPT so the fix is not a 11.4.201(1) false-positive refusal. Discovered out-of-band during T028 remediation and reported by the agent as needing tracking, so it is also a 11.4.238 coverage escape.

BOB-188 — Gate invariant 17 runs a STALE TRACKED binary that structurally cannot see the violations it exists to catch

Status: In progress **Type:** Bug

OPERATOR DECISION (2026-08-26, §11.4.66 interactive clarification): UNTRACK THE BINARY — BUILD ON DEMAND

§11.4.30 answer, recorded as consumer DATA: the compiled `workable-items` binary is NOT to be git-tracked. ‘Any build derivate which we can recreate by executing proper mechanism for generating MUST NOT be versioned’ applies without exception here. Keeping it tracked with only the invariant-52 fingerprint gate, and keeping it tracked with a commit-time rebuild, were both offered and NOT chosen. REQUIRED END STATE: the gate builds from source, or REFUSES HONESTLY when the Go toolchain is absent (§11.4.201(11) — probe the artifact through its real invocation path, never fake a pass, never silently fall back to a stale binary). Staleness becomes structurally impossible rather than merely detected. ACCEPTED COST: a fresh clone needs a Go toolchain before invariant 17 can run — already true for the qBitTorrent-go backend, so this adds no new host requirement. The invariant-52 fingerprint gate remains useful as defence-in-depth for as long as any binary exists on disk.

This answer is recorded as consumer DATA per §11.4.35 — it is the operator’s stated choice, not an agent inference, and supersedes any prior agent-chosen default on this question. Options not chosen are named above so a future reader does not re-litigate a settled call (§11.4.112(5) bounded-verdict discipline applied to decisions).

— prior item text follows —

pre_build_verification.sh invariant 17 (CM-WORKABLE-ITEMS-VALIDATE) resolves its binary through the candidate loop at :534, whose FIRST entry is constitution/scripts/workable-items/bin/workable-items. That file is GIT-TRACKED (md5 17644a248363, identical to bin/workable-

items-linux) and executable, so it WINS resolution over the current untracked sibling constitution/scripts/workable-items/workable-items (md5 43376a6d0184). The tracked binary is STALE: it does not contain the guards its own source now has.

MEASURED, needle-proven, 2026-08-25. String presence in the tracked binary: “refusing to set terminal status” -> 0; “Issues-location item has TERMINAL status” -> 0; control needle from the SAME updateCmd, “at least one mutable field flag is required” -> 2 (so the instrument sees through that path and the zeros are real absences, not a blind read). The untracked sibling returns 1 / 1 / 2 for the same three queries. A check whose message string is absent from the binary cannot fire.

RUNTIME PROOF: the same injected violation (a row set to terminal status while current_location=Issues) run through both binaries -> stale reports 1 violation (only the body_md desync class), current reports 2 including the location-status class verbatim. This is exactly why the ten BOB-166 rows (BOB-087/129/131/144/145/146/148/153/155/157) survived undetected: the gate that was supposed to catch them was running a binary structurally incapable of seeing them.

The 11.4.108 SOURCE->ARTIFACT gap: source correct, artifact stale, every gate reading the artifact reports green. This is the THIRD instance of that class found in one session, alongside BOB-169 (a charset gate that never opened a file) and BOB-183 (a served bundle five days older than its sources).

COMPOUNDING: the comment at :519-523 directly above the loop asserts “The naive bin/workable-items path never existed in this checkout (bin/ is a gitignored local build-output dir nothing ever populated)”. That statement is now FALSE - bin/ holds two tracked binaries. A stale comment asserting the absence of the very file that now shadows resolution is how this stayed invisible.

Also an 11.4.30 question the operator owns: bin/workable-items and bin/workable-items-linux are VERSIONED BUILD ARTIFACTS in a constitution submodule that is inherited by

reference by every consumer, so every consumer inherits whichever binary was last committed. Flagged independently by two separate agents this session.

ACCEPTANCE: (a) resolution must never prefer a stale artifact over a current one - either the tracked binaries are removed and resolution falls through to build-on-demand, or a freshness check refuses a binary older than its sources (see the BOB-183 fingerprint gate for a working pattern); (b) the false comment at :519-523 corrected; (c) a paired 1.1 mutation proving the new refusal fires, plus a negative control proving a genuinely fresh binary still passes so the fix is not an 11.4.201(1) false-positive refusal; (d) the 11.4.30 tracked-binary decision recorded as operator DATA. Discovered out-of-band during BOB-166 remediation - a 11.4.238 coverage escape in its own right.

BOB-189 — CM-NO-FAIL-OPEN-SKIP scanner flags fail-CLOSED SSRF guards as fail-open (§11.4.201(1) FAIL-bluff in our own gate)

Status: Queued **Type:** Bug

WHAT: the fail-open scanner’s shape-(A2) heuristic cannot distinguish ‘return False’ meaning PROCEED-AS-IF-FINE from ‘return False’ meaning REFUSE. It flags six textbook fail-CLOSED guards as fail-open defects: _is_safe_fetch_url (x3), _qbit_add_succeeded (x2), and the hooks path-boundary guard. INDEPENDENTLY VERIFIED 2026-08-25 by the conductor rather than taken on the triage agent’s word: download-proxy/src/api/routes.py:1122 defines _is_safe_fetch_url returning bool, and its only call site at :1476 reads ’if not _is_safe_fetch_url(url): logger.warning(Refusing SSRF-unsafe download URL (non-public target); skipping); continue’ - a False return REFUSES the URL. WHY THIS MATTERS: per §11.4.201(1) a false-positive refusal is a FAIL-bluff of equal severity to a false-negative pass, and here the consequence is worse than noise - acting on the finding would mean making the SSRF guard stop returning False, i.e. deleting an SSRF protection to satisfy a gate. A gate that instructs you to remove a security

control is actively dangerous, not merely imprecise. REPRO: run the fail-open scan; observe six hits; read each call site. ACCEPTANCE: the scanner distinguishes refuse-shaped from proceed-shaped False returns (call-site-aware, or an audited waiver list with per-entry justification), the six drop out, AND a golden-FALSE fixture containing a real fail-closed guard is added so the discrimination is itself falsifiable per §1.1.

BOB-191 — Fail-open scanner is a MATCHER hole blind to 5 enumerated languages (Go/Rust/Ruby/C/…) — enumerated-but-unanalysable prints PASS instead of SKIP (§11.4.201(6))

Status: Queued **Type:** Bug

WHAT: the §11.4.252 fail-open scan reports 0 hits for qBitTorrent-go, and that zero is NOT evidence. The triage agent ran control needles through the scanner's own path per §11.4.201(7)(b): a Python 'except Exception: pass' needle was SEEN, a TypeScript 'catch (e) {}' needle was SEEN (so frontend/src's zero IS a real zero), but a Go empty-'if err != nil {}' needle was NOT SEEN. An instrument that cannot see the idiom returns the same quiet zero as a clean tree, and only one of those is honest. DISTINCT FROM BOB-189, deliberately not merged with it per §11.4.214: BOB-189 is a false MATCH (fail-closed guards reported as fail-open); this is a false NULL (an entire language unscanned). Same scanner, opposite failure directions, different fixes - merging them would hide one behind the other. IMPACT: Go is the language of qbittorrent-proxy-go and boba-jackett (port 7189, which owns encrypted tracker credentials), so the unscanned surface is exactly where §11.4.252's credential-plus-mutation combination is most likely. Nobody has assessed it; the dashboard says clean. ACCEPTANCE: the scanner grows a Go arm covering the empty-err-block and swallowed-error idioms, its needle is SEEN through the real path, qBitTorrent-go's result is re-derived, and every finding

is triaged as this Python/TS pass was. Until then qBitTorrent-go's fail-open posture is UNKNOWN and must be reported as UNKNOWN, never as 0.

BOB-192 — Remediate the 6 ratcheted CM-NO-FAIL-OPEN-SKIP findings — each needs a live-stack-verified classify-or-fail rewrite

Status: Queued **Type:** Task

WHAT: the BOB-161 gate lands with 6 real fail-open skips RATCHETED rather than fixed. Ratcheting is the constitution's named brownfield default (§11.4.135/ §11.4.224(E)) and this repo's own precedent, so the choice is correct - but the remediation it defers is real work that must be owned somewhere. WHY THIS ITEM EXISTS: the gate's own header asserted the 6 were 'TRACKED SEPARATELY (§11.4.197)' while no tracker row existed. The §11.4.209 independent review verified the absence and raised it as IMPORTANT-5, noting that without a row those findings are precisely the parked-unverified debt class §11.4.226(4) names - the population an operator samples and finds broken. A prose claim of being tracked is not tracking. WHAT EACH NEEDS: a skip that fires on evidence the host ANSWERED must either classify the response and FAIL on it, or take a §11.4.69 reason that is honestly derivable from the environment rather than from the response - verified against the live stack, not asserted. Two of the six sit under '# allow-skip:' markers at tests/unit/ test_tracker_auth_live.py:105 and :108, which the reviewer confirmed genuinely are fail-open, so that marker must not be treated as absolution. ACCEPTANCE: all 6 remediated with RED-first evidence per §11.4.115, the gate's BASELINE ratcheted to 0, and the ratchet's monotone-decreasing property preserved throughout (§11.4.227(A)). NOTE the reviewer's MINOR-1: a count-baseline absorbs a one-out-one-in swap, so remediation progress must be checked against the finding SET, not only the count.

BOB-193 — SSE result loss is add-before-yield: a raising emit permanently drops a result and the dedup set already recorded it

Status: Queued **Type:** Bug

WHAT: in download-proxy/src/api/streaming.py the dedup set is written BEFORE the yield — seen_hashes.add() / seen_hashes_local.add() execute at :317 and :356 ahead of emitting. If format_event raises, the result is never sent AND its hash is already recorded, so it is permanently dropped. Independently verified by the §11.4.209 reviewer at both sites: mid-search the hash persists across subsequent polls so the result never re-emits; at completion the stream breaks immediately after, so there is no later chance either. Results later in the SAME batch are not yet added and do re-emit on the next poll, which is why the observed symptom is partial loss rather than total. WHY IT IS FILED SEPARATELY: the §11.4.252 fail-open pass made this loss OBSERVABLE by adding a log line, which is the right first move — but observability is not repair. The underlying at-most-once semantics remain, and per §11.4.226(5) a mitigation cannot close the defect it mitigates. THE DECISION IS A REAL TRADE, not an oversight to correct blindly: moving to add-after-successful-yield gives at-least-once, but a DETERMINISTIC serialization failure would then retry the same result on every poll forever — trading silent loss for a hot loop. Options are (a) keep at-most-once and treat the log line as the contract, (b) add-after-successful-yield with a bounded per-hash retry budget, (c) add-before-yield but remove the hash on emit failure so exactly one retry occurs. ACCEPTANCE: an explicit decision recorded, implemented, and covered by a test that drives a raising emit and asserts the chosen semantics — with a RED observed first per §11.4.115. Raised as MINOR-1 by the independent review of the fail-open batch; filed rather than absorbed so the residual defect is not retired along with the logging that revealed it.

BOB-194 — Pre-build gates walk .claude/worktrees, so a stale agent worktree can fail the main build (§11.4.201(1))

Status: In progress **Type:** Bug

WHAT: agent worktrees under .claude/worktrees/ are ephemeral copies pinned at arbitrary commits - never built, never shipped, deleted when the agent finishes. Pre-build gates that walk the tree with find(1) from the repo root descend into them anyway. MEASURED INSTANCE (2026-08-25): CM-GO-TOOLCHAIN-MATCHES-BUILDER FAILED the main build with exactly one finding, from .claude/worktrees/agent-afda1df906c537ab4 - a worktree 86 commits behind still carrying 'FROM golang:1.23-alpine' against a go.mod long since at 1.26.2. The main tree was CORRECT throughout. Per §11.4.201(1) a false-positive refusal is a FAIL-bluff of equal severity to a false pass: it blocks real work and teaches people to bypass the gate. FIXED FOR ONE GATE: '.claude' added to that gate's PRUNE_DIRS with a deterministic §1.1 pair - a stale mismatch inside .claude/worktrees must NOT fail a healthy tree, the SAME mismatch outside it MUST still fail, so the prune cannot widen into blindness (§11.4.201(6)). Mutation verified: reverting the prune fails 2 checks. SURVEY, measured not inferred: 8 of 13 pre_build gates walk the tree; only the fixed one prunes .claude. Gates using 'git ls-files' are structurally immune - worktrees are untracked. Empirically probed: check_cm_closure_seam_binds, check_cm_killpg_pgid_guard and check_cm_no_production_mutation_residue all rc=0 with ZERO worktree references - clean in practice today. UNMEASURED, not clean: check_cm_test_mock_pid_explicit_int and check_cm_test_mock_pid_patched_when_real_pid were not reached before the probe budget expired. HYPOTHESIS REFUTED, recorded so nobody re-walks it: an earlier revision of this item speculated that check_cm_plugin_count's 90s timeout was caused by walking worktrees. It is NOT. That gate's find is scoped to $PLUGINS_DIR (check_cm_plugin_count.sh:224), so

worktrees are outside its walk entirely. Its slowness is real - still running past 54s on a re-measure - but has a different, still-unidentified cause and belongs to its own item, not this one. ACCEPTANCE: every tree-walking gate either prunes agent scratch or is proven immune by measurement, each with a paired mutation. NOTE the asymmetry that bounds the risk: a stale worktree can only ADD findings to a security-relevant scan (killpg, mutation-residue), never remove them - so the exposure is false refusal and eroded trust, not a missed defect.

BOB-195 — CM-DANGEROUS-COMBINATION-FAIL-CLOSED cannot see contextlib.suppress, and ruff SIM105 pushes authors into that blind spot

Status: In progress **Type:** Bug

OPERATOR DECISION (2026-08-26, §11.4.66 interactive clarification): TEACH THE SCANNER With NODES — KEEP SIM105

The §11.4.252 fail-open scanner is FIXED to see `contextlib.suppress`; ruff's SIM105 rule STAYS ENABLED in pyproject.toml. Disabling SIM105, and the belt-and-braces both-at-once option, were offered and NOT chosen. Rationale carried with the decision: fixing the gate makes SIM105 harmless, whereas disabling SIM105 alone would leave the gate blind to any suppress written by hand, inherited from a dependency's style, or already present in the tree. REQUIRED BEHAVIOUR: `with contextlib.suppress(...)` and `with suppress(...)` (the `from contextlib import suppress` binding) wrapping a dangerous-combination call are DETECTED with the same severity and message shape as the try/except/pass form; a narrow `suppress(SpecificError)` around a NON-dangerous call must NOT fire, since a false-positive refusal is a §11.4.201(1) FAIL-bluff of equal severity to a missed real one. CONSEQUENCE FOR THE NUMBER: once landed, the scanner's finding count

stops being a floor over try/except shapes and becomes a genuine census — any prior count citing completeness was scoped to try/except only.

This answer is recorded as consumer DATA per §11.4.35 — it is the operator's stated choice, not an agent inference, and supersedes any prior agent-chosen default on this question. Options not chosen are named above so a future reader does not re-litigate a settled call (§11.4.112(5) bounded-verdict discipline applied to decisions).

— prior item text follows —

WHAT: the §11.4.252 fail-open scanner matches Try-handler shapes (A1)/(A2). contextlib.suppress is a With node, so a swallow written that way is invisible. VERIFIED BY THE CONDUCTOR with a control needle through the same invocation path (§11.4.201(7)(b)), because a first attempt returned 0 for BOTH forms and was itself blind - the gate takes –root, not a positional, so a bare path exits on an unknown arg (§11.4.201(7)(c): the invocation path is part of the instrument). Correct run: try/except/pass around os.unlink(user_supplied_path) -> 'FAIL - swallowed exception … :5', the needle sees. contextlib.suppress(Exception) around the IDENTICAL call -> 'PASS - no swallowed-exception … anti-patterns found'. Both combine untrusted input with an irreversible unlink; both swallow everything; one is caught and one is not. WHY IT IS WORSE THAN A PLAIN GAP: pyproject.toml enables ruff's SIM ruleset, and SIM105 is 'use contextlib.suppress instead of try-except-pass'. Our own linter therefore instructs authors to rewrite the form this gate CAN see into the form it CANNOT. The two tools are pulling in opposite directions and the lint one runs more often. Every SIM105 autofix silently shrinks this gate's coverage while the count goes down, which reads as progress. CONSEQUENCE FOR THE NUMBER: the scanner's finding count is a FLOOR over try/except shapes, not a census of swallows. Any statement of the form 'N fail-open sites remain' must be read as 'N among try/except shapes'. ACCEPTANCE: the scanner recognises contextlib.suppress (a With whose items call contextlib.suppress with a broad exception type) and applies the same ≥2-capability test; a golden-TRUE fixture in suppress form; a golden-FALSE fixture where suppress wraps a single-capability diagnostic emitter and must NOT

fire; and an explicit decision on the SIM105 tension - either exempt the shapes this gate governs, or accept suppress and teach the gate to read it. Discovered by the §11.4.252 remediation agent when its own MINOR-3 fix required using contextlib.suppress at three diagnostic-emitter sites; those three uses are legitimate and justified in-source.

BOB-196 — check_cm_plugin_count runs for over a minute on ~69 files — cause unidentified, and it is not worktree traversal

Status: Queued **Type:** Task

WHAT: the CM-PLUGIN-COUNT pre-build gate takes 189 SECONDS to verify 8 documented counts. Measured 2026-08-25: exceeded a 90s probe budget (rc=124), then a bounded re-run completed at 'rc=0 wall=189s' - it PASSES, it is simply slow. CAUSE, MEASURED NOT GUESSED: the marker loop at check_cm_plugin_count.sh:273-290 iterates EVERY LINE of every governed document and spawns a pipeline per line. Per iteration: 'printf | grep -oE | wc -l | tr -d' (4 processes) plus 'printf | sed -n' (2) = ~6. Governed docs are AGENTS.md 689 + CLAUDE.md 494 + docs/features/ Status.md 670 = 1853 lines, so ~11,118 process spawns, measuring ~102 ms/line. Classic per-line-subprocess shell trap. THE IRONY WORTH PRESERVING, because it explains why nobody simplified it: the in-code comment shows '-oE | wc -l' was chosen DELIBERATELY over 'grep -c' after measuring the §11.4.201(12) footgun - on this host (ugrep 7.8.4) 'grep -coE' returned 3 at top level but 1 inside a 'set -euo pipefail' subshell, a context-dependent reading. The correctness fix is right and must be kept; it is the PER-LINE application of it that costs the three minutes. HYPOTHESIS REFUTED, recorded so nobody re-walks it: this is NOT worktree traversal. The gate's find is scoped to $PLUGINS_DIR (:224), so the five agent worktrees are outside its walk. That guess was raised on BOB-194 and is dead. LIKELY FIX (unverified, stated as a candidate not a conclusion): pre-filter with ONE grep for lines containing 'CM-PLUGIN-COUNT:' before entering the loop, reducing ~1853 iterations to the ~8 lines that carry a marker, while

leaving the per-line parsing logic - and its footgun-avoiding form - completely unchanged. WHY IT MATTERS beyond impatience: this gate is on the pre-build critical path, and CLAUDE.md leans on it as the mechanical authority for the three distinct plugin rosters (43 curated / 43 engines / 12 bootstrap) whose conflation was BOB-149. A gate slow enough to tempt anyone into skipping it protects nothing. ACCEPTANCE: runtime reduced with the counts and the footgun-avoiding parse preserved byte-for-byte in behaviour, proven by the existing paired mutation still biting plus a before/after wall-clock recorded. §11.4.6: nothing here questions the gate's VERDICTS - only its runtime.

BOB-197 — Auth is env-gated and OFF by default, so the LAN-bound merge service ships open — needs a boot-time invariant, not a static gate

Status: In progress **Type:** Bug **Created-By:** Claude **Assigned-To:** milos85vasic

OPERATOR DECISION (2026-08-26, §11.4.66): GENERATE THE TOKEN AND ARM IT

The conductor generates a 32-byte random token and writes BOBA_API_TOKEN into .env; the operator configures clients with it. DONE 2026-08-26: token generated via secrets.token_hex(32), appended to .env with a comment naming BOB-197 and the routes.py:83 env-gating fact, chmod 600, value shown to the operator ONCE and never written to any other file. Safety verified BEFORE writing (§11.4.30/§11.4.10): .env matched .gitignore:27 '*.env' and was untracked (needle-proven — the same matcher confirmed docker-compose.yml IS tracked, so the negative is real); a §9.2 pre-op backup was taken and is itself ignored; post-write re-check confirmed .env still ignored and absent from git status. Pre-store audit found the only tracked BOBA_API_TOKEN= occurrences are the docker-compose passthrough ${BOBA_API_TOKEN:-} and prose in the leak-audit challenge — no literal value has ever been committed. STILL OWED, and the item stays open until it lands: the

BOOT-TIME INVARIANT (§11.4.254) that refuses to start LAN-bound when the token is unset or empty. Without it the arming is a state fix, not a defect fix — §11.4.226(5) is explicit that a state-only repair cannot close a defect item. Options not chosen: operator sets it themselves with the invariant blocking until then; invariant warns first and blocks on a named date.

Recorded as consumer DATA per §11.4.35 — the operator's stated choice, not an agent inference. Options not chosen are named so a future reader does not re-litigate a settled call.

— prior item text follows —

OPERATOR DECISION (2026-08-26): arm BOBA_API_TOKEN and keep 0.0.0.0, backed by a BOOT-TIME invariant that refuses to start LAN-bound with the token unset. This item IS that invariant.

MEASURED FACT, not inference. download-proxy/src/api/routes.py:83 require_api_token() reads BOBA_API_TOKEN at request time and returns immediately when unset or empty. Its own docstring states it plainly: "BOBA_API_TOKEN unset/empty -> return (OPEN). This is the DEFAULT and preserves the current no-auth contract + dev workflow". docker-compose.yml:237 passes BOBA_API_TOKEN=${BOBA_API_TOKEN:-}, so the container inherits empty unless the operator declares it. The operator .env does NOT declare it, needle-verified per 11.4.201(7)(b): the same matcher hit QBITTORRENT_DATA_DIR in the same file, so the miss is a real absence and not a blind read. The qbittorrent-proxy container was RUNNING and LAN-bound at measurement time. Consequence: the ten routes the BOB-102 guard resolves as auth-wired are open in practice on a default deploy.

WHY A STATIC GATE CANNOT COVER THIS. The BOB-102 gate asserts static route WIRING - that a Depends(require_api_token) marker exists on handler, decorator or router. It cannot assert runtime ARMING, which depends on an env var read per request. Claiming the static gate covers it would be exactly the bluff the gate exists to prevent (11.4.262: machine evidence must match the layer it claims). This is a boot-time invariant class per 11.4.254.

ACCEPTANCE: a boot-time check that, when the listener is LAN-bound (not loopback), refuses to start with a non-zero exit and a named cause if BOBA_API_TOKEN is unset or empty, so the open state becomes UNREACHABLE rather than the default. Paired 1.1 mutation: remove the check, prove the service starts LAN-bound-and-open. Golden-FALSE per 11.4.201(1): a loopback-bound listener with the token unset must NOT be refused, or the check becomes a false-positive refusal blocking legitimate dev work.

11.4.238 COVERAGE-ESCAPE NOTE: found by a subagent reading source during BOB-102 guard construction, NOT by the automated QA regime - which is itself the defect class 11.4.238 names. The boot-time check IS the new automated check that would have caught it.

BOB-198 — qbittorrent-proxy-go registers CORS, Logger and rate-limit middleware but no auth at all — 22 LAN-bound routes open including download and hook deletion

Status: Queued **Type:** Bug **Created-By:** Claude **Assigned-To:** milos85vasic

OPERATOR DECISION (2026-08-26, §11.4.66): REFUSE TO START LAN-BOUND

The Go profile gets a boot-time guard: bind loopback only, or refuse to start. It does NOT get auth middleware in this cycle. Rationale carried with the decision: the profile is opt-in via –profile go, was not running when measured, and is used for development — so closing the exposure costs a guard rather than a parity implementation. Options not chosen: write auth middleware to parity with the Python route set (real work, and it would partially un-defer BOB-101); document dev-only with no guard (rejected — nothing would enforce it, and the next –profile go run on this network silently re-opens 22 routes). ACCEPTANCE: a boot-time check that resolves the listener's bind address and refuses to start when it is not loopback, with a paired §1.1 mutation proving the check FAILs on a 0.0.0.0 bind, and a golden-FALSE

proving a loopback bind is NOT refused (§11.4.201(1)). Scope reminder recorded so it is not lost: this is NOT covered by BOB-101's parity deferral, which was about ports; folding it in would be a §11.4.112(5) verdict leak.

Recorded as consumer DATA per §11.4.35 — the operator's stated choice, not an agent inference. Options not chosen are named so a future reader does not re-litigate a settled call.

— prior item text follows —

MEASURED, conductor-verified independently of the reporting subagent. qBitTorrent-go/cmd/qbittorrent-proxy/main.go registers exactly three middlewares, at lines 62, 63 and 68: middleware.CORS, middleware.Logger, middleware.GinRateLimit. There is NO authentication middleware and no per-route auth dependency. main.go:119-121 binds addr=":" which is ALL interfaces, so every route is LAN-reachable when the profile runs.

SCOPE: 22 routes, of which the mutating ones matter most - POST /api/v1/download, POST /api/v1/magnet, DELETE /api/v1/hooks/:id, POST and DELETE /api/v1/schedules. A LAN peer can queue downloads and delete webhook configuration with no credential.

PARITY GAP, stated as fact: PUT /api/v1/theme is auth-gated in the Python service and open in Go. The same asymmetry holds for the download, hooks and schedules mutations. Two implementations of one advertised surface disagree about whether it needs a credential.

RELATIONSHIP TO BOB-101 (Go parity deferred past v1.0.0, operator decision 2026-08-26): that deferral covered PORT parity - the Go container binding :7186 and :7188 as well as :7187. It did NOT cover an authentication hole, and the operator was not asked about one, because the hole had not been measured when the question was posed. Treating this as covered by the deferral would be a 11.4.112(5) verdict leak: citing a bounded decision outside the scope its evidence supports. This item stands separately and its severity is NOT settled by BOB-101.

MITIGATING FACT, so severity is not overstated: the Go profile is opt-in via –profile go and was NOT running at measurement time (the container list showed qbittorrent-proxy, the Python service, with no Go sibling). The exposure is latent, not live.

ACCEPTANCE: either auth middleware landed in the Go service with parity to the Python route set, or an explicit operator decision recording the Go profile as dev-only that must refuse to start LAN-bound - with a guard enforcing whichever is chosen.

BOB-199 — Fail-open scanner flags narrow try/except/pass but not narrow contextlib.suppress — SIM105 still moves those sites out of scope

Status: Queued **Type:** Task **Created-By:** Claude **Assigned-To:** milos85vasic

OPERATOR DECISION (2026-08-26, §11.4.66): FLAG NARROW SUPPRESS ONLY WITH AN IRREVERSIBLE CAPABILITY

The scanner flags a narrow contextlib.suppress ONLY when combined with an irreversible capability (delete / truncate / kill) — the shape that actually causes harm. Idiomatic narrow tolerances stay quiet, so the gate keeps its credibility and no false-positive storm trains readers to ignore it. Options not chosen: accept the asymmetry permanently with a header statement; flag all narrow suppress and absorb existing sites via a justified exemption list. MEASUREMENT CORRECTED TWICE, and the second correction explains the first: the original ‘37 narrow sites, all idiomatic’ was propagated by this conductor without verification, then re-measured by the authoring agent as OBSERVER CONTAMINATION (§11.4.201(10)) — 27 vendored third-party + 9 sibling-agent worktree COPIES of the very tree being measured + 1 real = 37. The instrument was counting other agents’ duplicates of its own subject. Reproducible figures, each with its scope stated: constitution repo 0 narrow with-suppress; boba DANGER_ROOTS 0

narrow with-suppress and 23 narrow except-pass; boba repo minus vendored 1 narrow with-suppress (a suppress(OSError)); narrow except-pass RETRACTED-AND-RESTATED 2026-08-26 per 11.4.201(9): the figure 97 previously recorded here reproduces under NO scope and is withdrawn. Measured replacements, each with its scope and both independently reproduced: 34 narrow except-pass git-tracked first-party (canonical); 50 narrow except-pass on-disk minus scratch dirs (the 'minus vendored' scope this sentence states). Brackets that explain the bad number: 87 ALL-except-pass git-tracked and 113 ALL-except-pass on-disk-minus-scratch straddle 97, so 97 was a class-scope conflation (narrow-vs-all), not a corpus-scope one. The 35 cited later in this record is the same measurement at git-tracked scope and agrees with 34 within one site. The 'every one idiomatic' claim was also false — the except-pass census contains SystemExit x2 (tests/unit/test_plugin_torrentscsv.py:305,328), which is not a benign tolerance. CORRECTION 2026-08-26: a 'KeyboardInterrupt x1' previously written in this sentence was itself unsourced and is WITHDRAWN — it reproduces at NEITHER stated scope. Independent AST count over git-tracked *.py: ZERO genuine 'except KeyboardInterrupt: pass'. The only genuine instance in the checkout is .worktrees/ci-split-workflows/tests/unit/test_main.py:63 — a sibling-agent scratch worktree, i.e. the exact observer-contamination class (11.4.201(10)) this very record teaches to exclude — and the only main-tree textual match (tests/unit/test_graceful_shutdown.py:165) sits inside a triple-quoted string literal, a carrier not code (11.4.201(7)(a)). Recorded because the error is instructive: it was introduced BY the retraction that removed the 97, so a correction is not exempt from the discipline it applies.

Recorded as consumer DATA per §11.4.35 — the operator's stated choice, not an agent inference. Options not chosen are named so a future reader does not re-litigate a settled call.

— prior item text follows —

RESIDUAL ASYMMETRY surfaced by the BOB-195 fix, reported rather than silently closed. After teaching the scanner With nodes, a BROAD suppress (Exception / BaseException) is detected with the same severity as try/except/pass. A NARROW suppress

(suppress(FileNotFoundError)) is not - but the semantically identical narrow try/except FileNotFoundError: pass IS flagged. So ruff SIM105 rewriting a narrow handler still moves that site out of the gate's scope, a smaller version of the exact mechanism BOB-195 was filed to close.

WHY IT WAS NOT CLOSED IN THAT PASS, with the measurement that decided it: the authoring agent reported 37 narrow suppress sites, all idiomatic. CORRECTION (2026-08-26, independent review): that figure DOES NOT REPRODUCE and this conductor propagated it into this item as fact without verifying - the error is mine, not the reviewer's. Measured: the constitution repo has 0 narrow `with suppress(X)` sites; boba first-party has exactly 1 (a suppress(OSError)). The nearest corpus match is 35 narrow `except X: pass` handlers in boba first-party - a DIFFERENT construct - and their class census (OSError x7, ImportError x7, BrokenPipeError x3, RuntimeError x3, ValueError x2, FileNotFoundError x2, IndexError x2, SystemExit x2) contradicts 'every one idiomatic': a bare `SystemExit: pass` is not a benign tolerance. The two populations must not be conflated, and the distinction is the whole substance of this item. The false-positive-storm argument therefore rests on the 35 narrow except-handlers, not on 37 suppress sites, and its strength should be re-judged on that basis, which under 11.4.201(1) is a FAIL-bluff of equal severity to the gap it would close, and worse in practice because it trains readers to ignore the gate. The subagent recorded the asymmetry in the gate header as a known gap rather than shipping the storm. That was the right call, and is why this is a separate tracked item rather than an unfinished one.

THE DECISION IS THE OPERATOR'S, mirroring BOB-195 itself: (a) accept the asymmetry permanently, with the gate header stating it so no reader mistakes the count for a census; (b) flag narrow suppress ONLY when combined with an irreversible capability (delete / truncate / kill), catching the shape that actually matters and leaving the 37 idiomatic sites quiet; (c) flag all narrow suppress and absorb the 37 via a justified exemption list like the LAN-route guard uses.

RELATED FACT worth carrying: the five newly-visible production sites in download-proxy/src/merge_service/search.py (1294, 1303, 1322, 1329, 1333) wrap proc.kill / os.killpg / proc.wait in the BOB-126 cleanup path. The

conductor verified the 11.4.263 pgid guard IS present and correct at both killpg sites - _pid and _pgid each checked isinstance(int) and > 1 before the syscall, with the BOB-126 forensic reasoning inline - so those sites are true-by-the-scanner's-definition but SAFE in fact, and want a justified exemption entry rather than a code change.

BOB-200 — cm_dangerous_combination_fail_closed gate reads a dead find(1) as a topology SKIP (exit 0) instead of failing closed

Status: Queued **Type:** Bug **Severity:** Medium

WHAT: scripts/gates/ cm_dangerous_combination_fail_closed.sh builds its scan file list with find(1). If find itself DIES (permission error, resource exhaustion, interrupted), the gate receives an EMPTY file list and interprets that as 'no files in scope' -> honest topology SKIP -> exit 0. The failure IS loud on stderr but SILENT in the exit code, so any caller gating on exit status reads a dead instrument as a clean corpus.

WHY IT MATTERS: this is a textbook 11.4.201(6) FALSE-NULL – a blind instrument and a clean artifact return the identical quiet zero. It is also a 11.4.252 fail-OPEN on exactly the 'cannot enumerate the corpus' condition where the gate's own stated discipline is to fail CLOSED. A gate that cannot see must refuse, not pass.

AFFECTED SCOPE: constitution/scripts/gates/ cm_dangerous_combination_fail_closed.sh (the find invocation and the empty-list branch). PRE-EXISTING – NOT introduced by the BOB-195 change; found by the independent round-2 reviewer while reviewing that change and explicitly scoped OUT of its remediation.

REPRODUCTION: make find(1) fail during the gate run (unreadable scan root, or a find stub returning non-zero with empty stdout) and observe the gate exit 0 with a topology-SKIP message, indistinguishable from a genuinely empty corpus.

ACCEPTANCE: (1) the gate distinguishes ‘find succeeded and found zero files’ from ‘find failed’ – check find’s exit status, not only its output; (2) a failed enumeration is a FINDING (non-zero exit) naming the unresolved precondition, never a SKIP; (3) a genuinely empty scan root still SKIPs honestly at exit 0 – the 11.4.201(1) golden-FALSE guard, so the fix does not become a false-positive refusal; (4) paired 1.1 mutation: restore the swallow-find-failure behaviour and the new fixture MUST fail.

BOB-201 — ownership_repair lexical fence does not resolve intermediate symlink components - a static symlink can steer the walk outside the declared scope

Status: Queued **Type:** Bug **Severity:** Medium

WHAT: scripts/ownership_repair.sh fences declared scope entries LEXICALLY (string normalisation + prefix/traversal refusal). A lexical fence cannot see a symlink sitting at an INTERMEDIATE path component. A static (already-present, no race required) symlink in the middle of an otherwise-legal declared path steers the recursive walk at a tree the operator never declared. Measured by the T028 round-2 independent reviewer as surviving mutation M5.

WHY IT IS FILED SEPARATELY: the in-source honest-boundary note and Case 24 fixture assert this reach is ‘tracked as BOB-159’. It is NOT. VERIFIED 2026-08-26 by direct query: BOB-159’s description is 4685 chars with ZERO occurrences of ‘symlink’ or ‘intermediate’ (control-needled - the body is readable through the same query path), and no item in the tracker recorded this reach at all. A dangling tracker citation means a real defect is recorded NOWHERE -

the 11.4.214 lost-defect shape, where the pointer looks like coverage and is not. This item IS that record; the in-source note must now cite THIS id.

SCOPE / SEVERITY BOUNDS, stated honestly (11.4.6): the reach is BOUNDED, not arbitrary. It requires an attacker or accident to have already placed a symlink inside a declared scope path. It is NOT reachable from the shipped config/owned_paths.yaml as it stands (shipped-scope control needle: 6 rows, RC=0). It is a real widening of what a recursive chown can touch, not a theoretical one - the reviewer measured it, no race needed.

WHAT THIS ITEM DOES NOT CLAIM: that the lexical fence is broken. The fence does what it says - it refuses lexical escapes ('/', system trees, '..' climbs) and was verified doing so under 12 reviewer-authored mutations. This is a documented LIMIT of lexical fencing, now recorded as a real item instead of a citation to an unrelated one.

ACCEPTANCE: (1) decide deliberately and record the decision - resolve components (realpath/-P semantics) and re-fence the resolved path, OR keep the lexical fence and state the limit as an accepted operator-owned risk; the choice is operator-owned per 11.4.66 because resolving makes the fence depend on filesystem state at check time, which has its own failure modes; (2) if resolution is chosen, a golden-FALSE set proving legitimate symlinked-but-in-scope download roots still walk (11.4.201(1) - a fence that over-refuses is a FAIL-bluff of equal severity); (3) the in-source note and the Case 24 fixture cite THIS item, not BOB-159; (4) paired 1.1 mutation: restore the un-resolved behaviour and the fixture MUST fail.

BOB-202 — ownership_repair tab-delimited scope read collapses an empty field - an omitted 'kind' silently shifts every later field and turns optional:true into non-optional

Status: Queued **Type:** Bug **Severity:** Medium

WHAT: scripts/ownership_repair.sh:493 reads parsed scope rows with ‘while IFS=$’ read -r e_path e_kind e_opt e_pres e_rec’. A scope entry that OMITS ‘kind’ emits the row ‘’. TAB is an IFS-WHITESPACE character in bash even when IFS is set to it explicitly, so a RUN of consecutive tabs collapses into ONE delimiter instead of delimiting an empty field. Every field after the omission shifts left by one.

MEASURED 2026-08-26 on GNU bash 5.2.37, control-needled (the with-field case was run FIRST and produced output, proving the instrument sees; an earlier probe of mine returned nothing for BOTH cases because printf lacked a trailing newline - an 11.4.201(7)(b) false-null I discarded rather than reported):

kind PRESENT: path=[P] kind=[dir] opt=[1] pres=[0] rec=[1]
kind EMPTY: path=[P] kind=[1] opt=[0] pres=[1] rec=[]

CONSEQUENCE: ‘optional’ is read out of ‘preserve_mode’s slot. An entry declared ’optional: true’ is therefore treated as NON-optional, so an absent path that should skip honestly instead becomes a hard failure - and ‘preserve_mode’/‘recursive’ are likewise read from the wrong slots, with ‘recursive’ arriving EMPTY. The shift is silent: no parse error, no diagnostic, no refusal.

REACHABILITY: NOT reachable from the shipped config/owned_paths.yaml - all six declared entries carry ‘kind’ (verified). This is a latent defect in the reader, not a live misbehaviour of the product today.

PROVENANCE: surfaced out-of-band by the T028 round-5 remediation agent while isolating an unrelated instrument slip, and deliberately left unfixed and unfiled by it because (a) ownership_repair.sh had to stay at hash ae025b74602414ca for that round’s do-not-regress set, (b) it was outside that round’s two-NIT remit, and (c) it had not root-caused whether the repair belongs in the parser, the consumer, or the schema. That judgement was correct; this item is the filing it deferred. Independently re-measured by the conductor before filing - not accepted on report.

11.4.238 COVERAGE-ESCAPE NOTE: no automated check found this. It was found by a human-directed agent isolating a different problem. Per 11.4.238 the coverage gap is itself a defect of equal standing to the bug, and closing only the bug is the violation.

ACCEPTANCE: (1) root-cause the correct layer - parser (never emit a row with an empty field), consumer (read with a delimiter that is not IFS-whitespace, or read positionally), or schema (make 'kind' mandatory and REFUSE an entry lacking it, consistent with the whole-run refusal already landed for empty expansions); the choice is operator-owned per 11.4.66 because it changes the scope-file contract; (2) a fixture with an omitted 'kind' and 'optional: true' asserting the entry is treated as OPTIONAL (or the row refused), RED-first against current code; (3) a golden-FALSE proving a fully-specified row still parses identically - 11.4.201(1), the reader must not start refusing valid scopes; (4) paired 1.1 mutation restoring the collapsing read so the fixture FAILs; (5) an automated check that would have caught it, per 11.4.238.

PRECEDENT FOUND 2026-08-26 (surfaced by the T028 round-5 reviewer, independently verified before recording): the SIBLING script scripts/ownership_precondition.sh ALREADY documents this exact collapse class at lines 565-578 and already ships the repair as split_tsv() at line 579 (used at 437, 699, 775). Its in-source note carries a real forensic measurement dated 2026-08-21: the FIRST version of that script used the collapsing read and reported "no compose service mounts this location" for config/ and for the download root WHILE FIVE SERVICES MOUNT THEM - a false statement about what was checked, produced by the instrument rather than the system, which its own comment classifies as the 11.4.201(7)(c) 'the path is part of the instrument' failure. It also records why the unit suite could not see it: the fixture scope has no compose service at all, so only running the real invocation against the real scope surfaced it.

WHAT THAT CHANGES FOR THIS ITEM: (a) acceptance-criterion (1) 'root-cause the correct layer' now has a PRECEDENT rather than an open design question - the consumer layer, via a split_tsv-style reader that does not use tab-as-IFS; adopting the sibling's existing function is preferable to inventing a second dialect (11.4.251 - two

answers to one question across two files that read the SAME scope rows is exactly the divergence that produced the round-1 ‘two readers of one scope file disagree’ finding); (b) this is a RECURRENCE of a class already diagnosed and closed once in this codebase, not a novel discovery - the fix landed in one reader on 2026-08-21 and the sibling reader kept the collapsing form, which is the 11.4.238 escape shape at the code layer: the lesson was captured in a comment instead of in a check; (c) the 11.4.238 coverage-escape note above is SHARPENED - the sibling’s own comment already states that a fixture-scope suite cannot see this class, so the required automated check must exercise a row with a genuinely empty middle field, not merely a well-formed one.

BOB-203 — LIVE: unauthenticated mutating LAN routes accepted on ports 7186 and 7187, and the armed BOBA_API_TOKEN is inert

Status: Queued **Type:** Bug **Severity:** Critical

MEASURED LIVE 2026-08-26 against the running stack from LAN 192.168.1.90 (NOT loopback - qBittorrent’s localhost bypass would have exercised a different code path). Evidence: docs/qa/BOB-198/runtime_auth_verification_20260826.md.

VERDICT: unauthenticated mutating LAN requests ARE currently accepted. Two live services, four named routes: 1. POST /api/v2/torrents/stop port 7186 -> HTTP 200 with NO credentials (real torrent control) 2. POST /api/v1/hooks port 7187 -> 422 field-validation (request REACHED FastAPI body validation, which runs AFTER middleware) 3. POST /api/v1/schedules port 7187 -> 422 (same) 4. DELETE /api/v1/hooks/ port 7187 -> 404 ‘Hook not found’ (the handler EXECUTED A LOOKUP)

The 422/404 responses are the decisive evidence, not the 200: a 401 never appears anywhere, and reaching body validation or a handler lookup proves no auth middleware intervened.

THE ARMED TOKEN IS INERT. BOBA_API_TOKEN was armed in .env earlier this session on an operator decision. Port 7187 returns BYTE-IDENTICAL responses with and without it across three header shapes. Port 7189 does not open for it in five header shapes. Root cause identified: docker-compose.yml injects BOBA_API_TOKEN into EXACTLY ONE service - download-proxy (line 237) - and into neither qbittorrent-proxy-go nor boba-jackett. And even on 7186, where it IS injected, POST /api/v2/torrents/stop returned 200 unauthenticated, so injection alone is not enforcement.

PER-SERVICE POSTURE (measured): 7185 qbittorrent-nox LIVE, binds * - same mutating surface visible from LAN as via the proxy 7186 download-proxy LIVE, binds 0.0.0.0 - mutating control accepted unauthenticated 7187 merge service LIVE, binds 0.0.0.0 - 34 routes via openapi.json, NO auth layer at all 7188 bridge NOT BOUND - honest SKIP 7189 boba-jackett Go LIVE, binds * - writes 401-gated, reads OPEN incl. /api/v1/jackett/credentials 9117 jackett LIVE, binds * EVERY live service binds all interfaces. None is loopback-only.

RELATIONSHIP TO BOB-198 (its named subject was NOT running): qbittorrent-proxy-go is profiles:-gated (opt-in –profile go) and absent from the container set, so its ‘22 unauthenticated routes’ is neither confirmed nor refuted - honest 11.4.3 SKIP. The exposure recorded HERE is a DIFFERENT and ADDITIONAL surface on the Python service and the proxy. Do not treat this item as closing BOB-198.

11.4.238 COVERAGE ESCAPE - first class, equal standing to the bug: the automated QA regime did NOT find this. The static gate check_cm_lan_routes_authenticated went through SIX independent review rounds proving routes are statically wired to auth middleware, and its own honest boundary (tracked BOB-197) states it asserts static wiring ONLY. Nobody had measured runtime until a directed agent did. This is precisely the 11.4 covenant’s founding failure shape: a green gate over a broken-for-the-user reality. The owed remedy is an automated runtime check that would have caught it, with a RED capturing this exact exposure - not merely a fix to the routes.

ANTI-BLUFF PROVENANCE: control needle - Jackett returned a real 401 through the SAME instrument, so the 200s are not instrument blindness; port 7188 returned exit-7, so absences are real. Negative control - a deliberately wrong token still got 401 on 7189. Decisive probes 3/3 deterministic. Mutating probes used non-existent ids against a provably EMPTY torrent list ([] before and after), so effect was nil and verified on both sides. Token referenced by name only, never printed (11.4.10); the report was leak-scanned with a needle proving the scanner fires on a known leak.

ACCEPTANCE: (1) operator decision (11.4.66) on the intended posture per service - loopback-only bind, real auth middleware, or accepted-LAN-risk-with-rationale; (2) whichever is chosen, an automated runtime check per 11.4.238 that fails RED against today's exposure; (3) 11.4.108 layer-3/4 evidence on a clean deployment, not static analysis; (4) paired 1.1 mutation.

BOB-204 — Credential DELETE reports 204 success while the .env plaintext delete error is discarded — credential survives on disk

Status: Queued **Type:** Bug **Severity:** critical

WHAT: internal/jackettapi/credentials.go:234-253 (boba-jackett, port 7189). The DELETE credential handler checks the DB delete, then discards the errors of BOTH `_ = envfile.Delete(...)` (line 240) and `_ = d.Jackett.DeleteIndexer(id)` (line 249), then returns an UNCONDITIONAL 204 No Content. If the .env delete fails, the plaintext credential variables REMAIN ON DISK while the API reports the credential deleted. Second instance, same file: credentials.go:166-176 returns the error code `env_write_failed_db_rolled_back` while the rollback's OWN error is discarded (`_ =` at :169 and :171) — the response body asserts a rollback that was never confirmed, a §11.4 bluff in the API contract itself.

SCOPE: 4 combined dangerous capabilities on one path (credential access + filesystem mutation + external side effect + irreversible delete) where §11.4.252 fail-closed-on-dangerous-combination requires only 2. Composes §11.4.10 (credentials must never leak — a credential the operator believes deleted, still on disk, IS the leak class) and §11.4.252.

REPRODUCTION: read the cited lines; the discard is syntactic and unconditional. A runtime repro drives DELETE with the .env path made unwritable (chmod 0444 or a directory-level deny) and observes 204 while the variable persists in .env.

WHY IT WAS NOT CAUGHT: the CM-DANGEROUS-COMBINATION-FAIL-CLOSED gate (constitution/scripts/gates/cm_dangerous_combination_fail_closed.sh) enumerates .go at line 243 but gates its ast analyser on *.py at line 537, so Go falls to two text matchers that key on `catch` — a keyword Go does not have. All 106 .go files are structurally unanalysable while being counted as analysed. That blindness is BOB-191; this item is the DEFECT it hid.

ACCEPTANCE: (1) both discard sites either handle the error or the handler returns a non-2xx naming the unresolved precondition per §11.4.252(2); (2) the env_write_failed_db_rolled_back code is only emitted when the rollback actually succeeded, else a distinct honest code; (3) a RED test drives the unwritable-.env path and observes the pre-fix 204, flips GREEN post-fix (§11.4.115); (4) a golden-FALSE fixture proves the new guard does not refuse the healthy path (§11.4.201(1)).

DISCOVERY CHANNEL (§11.4.238): found by an agent reading source during the BOB-191 investigation, NOT by the automated QA regime — this is itself a coverage escape and BOB-191 carries the escape audit.

BOB-205 — cmd/boba-ctl (947 LOC container orchestrator, shell-exec + mutation surface) is absent from DANGER_ROOTS — never scanned at all

Status: Queued **Type:** Bug **Severity:** major

WHAT: the §11.4.252 fail-closed scanner is driven per-root by invariant 39 at scripts/pre_build_verification.sh:1481-1544 over a hand-maintained DANGER_ROOTS list. `cmd/boba-ctl/` — 4 files, 947 LOC — is NOT in that list, so it is never scanned by any arm. It is the container orchestrator: a shell-exec plus state-mutation surface, precisely the §11.4.252 dangerous-combination class the gate exists for.

DISTINCT FROM BOB-191: BOB-191 is a MATCHER hole (the root IS scanned; the Go files inside it are structurally unanalysable while counted as analysed — a false null that prints green). This is a SCOPE hole (the root is not scanned at all). Different failure shapes, different fixes; per §11.4.214 they are distinct-but-similar, deliberately not merged.

WHY BOTH EXIST: the root list is hand-maintained, so a new first-party root joins the tree without joining the gate. §11.4.251 (role-as-data-pack) points at the fix direction — replace the hand-maintained list with a declared manifest derived from the same source of truth the build uses, so a root cannot exist without being enumerated.

ACCEPTANCE: (1) cmd/boba-ctl is scanned — either by being added to DANGER_ROOTS or by the manifest replacing it; (2) whichever is chosen, a RED fixture proves a planted fail-open inside cmd/boba-ctl is SEEN pre-fix-absent / post-fix-present (§11.4.115); (3) if the manifest route is taken, a fixture proves a NEWLY-ADDED first-party root is picked up without a hand edit — that is the invariant that stops this recurring; (4) the honest-blindness path (§11.4.3 SKIP-with-reason, which the gate already implements correctly for unenumerated extensions) is preserved, never converted into a silent PASS.

DISCOVERY CHANNEL (§11.4.238): found by an agent auditing the scanner's own root list during the BOB-191 investigation, NOT by the automated QA regime — a coverage escape; BOB-191 carries the escape audit.

BOB-207 — probe_location() is GID-BLIND: precondition verifies uid only while the repair fixes uid AND gid — false ok demonstrated live on this host, no privileges, no exotic filesystem

Status: Queued **Type:** Bug **Severity:** critical

WHAT: scripts/lib/ownership.sh:299-320 probe_location() reads stat -c '%u' and compares against ownership_operator_uid (id -u). It NEVER reads '%g'. Meanwhile ownership_operator_gid is defined at scripts/lib/ownership.sh:50 and consumed ONLY by scripts/ownership_repair.sh:360 — so the REPAIR establishes uid AND gid, while the PRECONDITION that verifies the repair worked checks uid ONLY. The precondition can therefore report ok while exactly half the property the repair exists to establish is wrong.

MEASURED, LIVE ON THIS HOST, UNPRIVILEGED (a setgid directory is all it takes — no loopback, no mount, no sudo): dir: uid=1000 gid=10 mode=2755 (chgrp wheel + chmod g+s, both unprivileged) real file created there: uid=1000 gid=10 probe_location verdict = ok rc=0 <– operator gid is 1000, file gid is 10

SEVERITY RATIONALE: this is the SAME CLASS that BOB-206 alleged (a probe reporting ok while ownership is only partly held) but on an axis that is LIVE rather than hypothetical, needs no unusual filesystem, and reproduces with two unprivileged commands. BOB-206 was dismissed; this is the real defect that investigation surfaced underneath it.

ACCEPTANCE: (1) probe_location reads and compares gid as well as uid, OR the asymmetry is deliberately justified in-source with the reason (if only uid matters to the user-

visible goal, say so and explain why the repair sets gid at all); (2) a RED fixture builds the setgid directory above, observes ok pre-fix, flips to a refusal post-fix (§11.4.115); (3) a golden-FALSE fixture proves a correct uid+gid location is still ok (§11.4.201(1)); (4) whichever way it is resolved, precondition and repair agree on WHICH property they are talking about — that disagreement is the primitive defect (§11.4.250).

DISCOVERY CHANNEL (§11.4.238): found by the BOB-206 verification stream, not by the automated QA regime. No existing test exercises a gid mismatch.

BOB-208 — probe_location() does not check the want side: a failing id(1) yields a FALSE REFUSAL whose message names the correct uid as wrong (§11.4.201(1))

Status: Queued **Type:** Bug **Severity:** major

WHAT: scripts/lib/ownership.sh:301 sets want="$(ownership_operator_uid)" with NO exit-code check and NO emptiness guard. The file runs under set -uo pipefail but NOT set -e, so a failing id(1) leaves want empty and execution continues.

MEASURED (id shimmed to fail): verdict = wrong-owner:1000 on a perfectly healthy location. The refusal message NAMES THE CORRECT UID AS WRONG, which is worse than a bare failure — it sends the reader to fix a value that is already right.

WHY IT MATTERS: §11.4.201(1) — a false-positive refusal is a FAIL-bluff of exactly equal severity to a false-negative pass. It halts real work and teaches operators to bypass the guard. And §11.4.201(4): on an unresolvable signal the guard must take the conservative-safe default AND SAY SO HONESTLY. 'Could not resolve the operator uid' is honest; 'wrong-owner:1000' when 1000 is correct is not.

LIKELIHOOD: low (id(1) rarely fails) — but unguarded, and it fails toward a misleading refusal rather than toward an honest unknown.

ACCEPTANCE: (1) the want side is checked for both rc and emptiness; (2) an unresolvable want yields a distinct honest verdict naming the unresolved precondition, never a wrong-owner claim; (3) a RED fixture shims id to fail and observes wrong-owner:1000 pre-fix, the honest verdict post-fix; (4) golden-FALSE: a healthy location with a working id is still ok.

DISCOVERY CHANNEL (§11.4.238): found by the BOB-206 verification stream reading the probe, not by the automated QA regime.

BOB-209 — probe_location() stat guards are asymmetric halves, a stat failure is mislabelled unwritable, and the probe temp file has no EXIT trap (§11.4.14)

Status: Queued **Type:** Bug **Severity:** major

THREE DEFECTS IN ONE FUNCTION, scripts/lib/ownership.sh:299-320.

1. ASYMMETRIC GUARDS. The file branch (:308) checks stat's EXIT CODE but not output emptiness. The directory branch (:314-316) checks EMPTINESS but not the exit code. Neither checks both. Each branch is blind to exactly the failure mode the other guards against.

2. MISLABELLED VERDICT. A stat failure on an EXISTING file is reported as 'unwritable'. That is semantically wrong — stat failing is not a writability fact, and it sends the reader to check permissions on a path whose permissions may be fine. §11.4.6: state the real condition or state that it could not be resolved; do not substitute a different condition.

3. UNTRAPPED CLEANUP (§11.4.14). rm -f “${probe}” at :315 is a plain statement, not a trap. An interrupt between mktemp and rm strands a .ownership-probe.XXXXXX file INSIDE A DECLARED LOCATION — and the declared set includes the git-tracked download-proxy/ tree, so the stranded file lands in version control’s path. §11.4.14 requires cleanup on EVERY exit path via trap.

ACCEPTANCE: (1) both branches check rc AND emptiness; (2) a stat failure yields a verdict naming stat-failed, not unwritable; (3) cleanup moves into a trap covering interrupt paths; (4) a RED per defect — including one that interrupts between mktemp and rm and asserts no residue survives; (5) golden-FALSE proving the healthy path still returns ok.

DISCOVERY CHANNEL (§11.4.238): found by the BOB-206 verification stream. The verifying agent confirmed it left zero residue itself, needle-proven that its find could see.

BOB-210 — detect_rootless() header promises an unverified reading can never manufacture a refusal, but :413 emits a CONFIDENT rootful that reaches the R3 refusal — and the shim oracle shares the code’s unvalidated premise (§11.4.245)

Status: Queued **Type:** Bug **Severity:** major

WHAT: scripts/ownership_precondition.sh:342-348 documents the docker branch of detect_rootless() as documented-not-measured (accurate — verified verbatim). The header then claims the branch is built so that ‘an unverified reading can never manufacture a refusal’. IT CAN. Line :413 emits a CONFIDENT rootful verdict whenever docker info succeeds with non-empty output containing no name=rootless field. Only the FAILURE modes fall through to unknown (:387-394).

THE PATH TO HARM: PUID=0 is declared in docker-compose.yml:32 (and again for jackett — deliberate per project policy). With a confident rootful verdict, that reaches the R3 refusal at :517. So a genuinely ROOTLESS Docker host whose docker info output SHAPE differs from the documented one gets a FALSE REFUSAL produced by a branch the source itself admits was never measured — precisely the outcome the header promises is impossible.

THE ORACLE PROBLEM (§11.4.245 independence): the shim tests DO cover this path (tests/unit/test_ownership_rootless_detection.sh:283) — but the shim's output shape was authored from the SAME documentation as the code. Oracle and code share the unvalidated premise, so the test cannot discover that the premise is wrong. It confirms the code matches the doc; nobody has confirmed the doc matches Docker.

ACCEPTANCE: (1) either the header claim is corrected to match :413's real behaviour, or :413 is changed to yield unknown when the reading is unverified — the two must agree (§11.4.6); (2) the docker branch's premise is validated against REAL docker info output from a real rootless daemon, or the branch is honestly marked unmeasured at the point of USE, not only in the header; (3) if validation is operator-gated (needs a rootless Docker host), record it as such per §11.4.21 rather than leaving the header's promise standing.

ALSO VERIFIED GOOD, recorded so it is not re-investigated: the unknown branch is handled IDENTICALLY at both consumers (R3 :519-531, R4 :832-836 — both a named SKIP, both return 0, neither refuses), ROOTLESS_VERDICT is cached at :487 so both read ONE measurement, and the carrier-trap golden-FALSE fixture EXISTS (test_ownership_rootless_detection.sh:279-308 drives name=rootless-lookalike inside a profile path and must NOT read rootless; structural guard at :396-411 uses exact field equality, not substring).

DISCOVERY CHANNEL (§11.4.238): found by the BOB-206 verification stream.

BOB-211 — LATENT + OPERATOR-GATED: on a uid-flattening mount whose uid is not the operator, chown fails EPERM with no fstype-aware diagnosis, and fmask/dmask silently defeat the preserve_mode 600 contract

Status: Queued **Type:** Bug **Severity:** minor

STATUS: LATENT on this host (measured: all six declared locations sit on ONE btrfs mount, fully ownership-expressive) and OPERATOR-GATED for verification. Retained from the dismissed BOB-206 rather than discarded with it, because the scenario is well-founded elsewhere: the declared mount is a udisks auto-mounted REMOVABLE NVMe, so another operator's portable drive being exFAT or NTFS is entirely ordinary.

TWO RESIDUALS: (1) REPAIR-LAYER DIAGNOSIS. On a uid-flattening mount whose flattened uid is NOT the operator, probe_location correctly REFUSES (verified: uid=0 and uid=100999 both refuse). But it then hands ownership_repair.sh a chown that will return EPERM, and nothing in the repair path is fstype-aware — so the operator sees a permission failure whose real remedy is a REMOUNT OPTION, which no message says. Reasoned, NOT proven: the shim used to verify BOB-206 models the stat READ, not the chown WRITE, and cannot settle whether chown actually fails. (2) MODE CONTRACT. vfat flattens MODE via fmask/dmask, so the preserve_mode: true / 600 contract on .env and config/boba.db would be silently unenforceable there. §11.4.10-adjacent: a credential file believed to be 600 that is world-readable by mount option is a leak the mode check cannot see.

WHY OPERATOR-GATED: a genuine end-to-end RED needs a real uid-flattening mount. The privilege-free route is bindfs -u (FUSE; fusermount IS present and user_allow_other IS set) — but bindfs is ABSENT on this host, as are mkfs.vfat and

losetup. So verification needs either a bindfs install or a privileged loopback mount. Neither was attempted (§11.4.21 — high-blast-radius host mutation is operator-gated).

ALSO RECORDED: NO test anywhere exercises a uid-flattening filesystem — vfat/exfat/ntfs/flatten/loopback/mkfs/losetup/bindfs/fusermount all measure 0 across the three ownership test files, needle-proven (probe_location = 3 hits through the same instrument).

ACCEPTANCE: (1) operator decides whether to install bindfs or provide a privileged loopback so the RED can be built; (2) if verified, the repair path gains fstype-aware diagnosis naming the remount remedy; (3) the mode contract either detects mode-flattening mounts and refuses, or documents honestly that it cannot be enforced there.

DISCOVERY CHANNEL (§11.4.238): retained residual from the BOB-206 verification stream.

BOB-212 — .gitignore deny-all *credentials* glob plus a hand-maintained allowlist silently swallows NEW credential-named source files — a false-null in the commit path itself

Status: Queued **Type:** Bug **Severity:** critical

WHAT: .gitignore:31 is a deny-all glob *credentials*, followed by a HAND-MAINTAINED per-file allowlist of ! exceptions (lines 34-50). Any NEW source file whose name contains ‘credentials’ (or ‘creds’) is silently ignored: git add refuses it and git status shows NOTHING. The author gets no signal at all — the file simply does not exist as far as the commit path is concerned.

HOW IT SURFACED: the BOB-204 stream authored a RED test named credentials_failclosed_test.go. It vanished. The agent noticed only because it went looking for the file it had just written. It renamed to bob204_failclosed_test.go to escape

both *credentials* and *creds*, and the file became visible. A silent deliverable loss, caught by luck rather than by any gate.

INDEPENDENTLY RE-CONFIRMED BY THE CONDUCTOR, control-needle proven: git check-ignore -v –no-index qBitTorrent-go/internal/jackettapi/credentials_failclosed_test.go -> .gitignore:31:*credentials* (IGNORED) git check-ignore -v –no-index qBitTorrent-go/internal/jackettapi/zzz_probe_test.go -> no match (NOT ignored) <– the needle: instrument proven seeing, so the hit above is real git check-ignore -v –no-index frontend/e2e/credentials.spec.ts -> .gitignore:31:*credentials* (IGNORED) git ls-files –error-unmatch frontend/e2e/credentials.spec.ts -> tracked

SECOND, LATENT INSTANCE: frontend/e2e/credentials.spec.ts is matched by the same rule and survives ONLY because it is already tracked (git honours the index over .gitignore for tracked paths). Delete-and-re-add it — an ordinary refactor, a branch operation, a file move — and it disappears silently. It is one routine operation away from being lost. (For contrast, credential-edit-dialog.component.spec.ts is genuinely safe: it is covered by the DIRECTORY rule !frontend/src/app/jackett/credentials/ at line 43, not by a per-file entry.)

WHY THIS IS A §11.4.201(6) FALSE-NULL, NOT MERELY AN INCONVENIENCE: the blind instrument and the clean tree return the identical quiet zero. git status showing nothing means BOTH ‘no new files’ AND ‘a new file exists but is invisible’. There is no signal that distinguishes them. The commit path — the thing every other gate’s output eventually has to travel through — cannot see its own blindness.

WHY IT IS NOT SIMPLY ‘ADD ANOTHER ! LINE’: that is the mechanism that produced the defect. A hand-maintained allowlist against a deny-all glob has the §11.4.251/§11.4.205 shape — every future legitimate credential-named source file needs a hand edit nobody will remember to make, and the failure mode of forgetting is SILENT. The same shape as BOB-205’s hand-maintained DANGER_ROOTS: a list that must be edited in lockstep with the tree, with no guard that notices when it was not.

TENSION TO RESOLVE HONESTLY (§11.4.10 vs §11.4.201(1)): the glob exists for a real reason — §11.4.10 forbids credential material reaching git, and a broad deny-all is the conservative-safe default. Narrowing it carelessly would re-open a genuine leak channel. So the fix is NOT 'delete the glob'. Candidate directions, operator decision (§11.4.66): (a) narrow the deny to what actually carries secrets — extensions and directories (*.env, secrets/,* credentials*.json|yaml|yml|txt|enc) rather than every path containing the word; (b) keep the deny-all but add a GATE that FAILS when an untracked file matches an ignore rule AND lives under a first-party source root AND has a source extension, so the swallow becomes loud instead of silent; (c) both. (b) alone closes the false-null even if the glob stays exactly as it is, and is the smaller change.

ACCEPTANCE: (1) a NEW credential-named source file under a first-party source root either commits normally or produces a LOUD refusal naming the rule that blocked it and the remedy — never silence; (2) a RED that creates such a file and asserts the current silence, flipping to the loud path post-fix (§11.4.115); (3) a golden-FALSE proving real secret material (a .env, a key) is STILL ignored — the §11.4.201(1) guard, because a fix that leaks credentials to close a false-null is strictly worse than the defect; (4) frontend/e2e/credentials.spec.ts is made safe by rule rather than by the accident of already being tracked.

DISCOVERY CHANNEL (§11.4.238): found by the BOB-204 stream when its own deliverable disappeared — NOT by the automated QA regime, and not by any gate. Nothing in the repo checks that an authored file actually became visible to git. Coverage-escape audit: no surface exists for this class at all.

BOB-213 — LANGUAGE HOLE: sh is absent from the fail-closed gate's extension list, so 71 of 74 source files in the scripts/ DANGER_ROOT are silently invisible while the driver prints a clean verdict

Status: Queued **Type:** Bug **Severity:** critical

WHAT: the §11.4.252 fail-closed gate's extension list omits shell entirely. Measured verbatim by the conductor at constitution/scripts/gates/cm_dangerous_combination_fail_closed.sh:457 — exts="${DANGEROUS_COMBO_EXT:-py go rs c cc cpp h hpp java cs js ts jsx tsx php rb}" No sh. No bash.

THE CONSEQUENCE, MEASURED: scripts/ IS a declared DANGER_ROOT (scripts/pre_build_verification.sh:1511). It contains 71 tracked .sh files and 3 .py files. So 71 of 74 source files in a root the gate is explicitly pointed at are SILENTLY INVISIBLE — while invariant 39's driver prints "no fail-open anti-pattern across N first-party source root(s)". Project-wide there are 198 tracked .sh files (negative control *.zzz = 0, instrument proven seeing).

WHY THIS IS THE WORST OF THE THREE HOLES: BOB-205 is a scope hole (root never looked at). BOB-191 is a matcher hole (Go enumerated but unanalysable). THIS one is worse than either, because the root IS declared, IS scanned, IS counted as covered, and 96% of what is in it was never examined. The count in the driver's own summary is an under-count presented as a census.

AND SHELL IS THE HIGHEST-RISK LANGUAGE HERE, not the lowest: the project's orchestration, credential handling, container control and gates are shell. start.sh alone is 1288 LOC and is, per CLAUDE.md, THE orchestrator. Fail-open in shell is also unusually easy to write — a bare `cmd || true`, an unchecked `$?`, a `set +e` region, a swallowed `2>/dev/null` — and §11.4.67(6) already records one live instance of exactly this class (a bare exec redirection silencing an interactive shell for its lifetime).

CORRECTION TO A PREVIOUSLY-STATED ACCEPTANCE CRITERION (§11.4.6): I asserted, when filing BOB-205, that "the honest-blindness path (§11.4.3 SKIP-with-reason, which the gate already implements correctly for unenumerated extensions) is preserved". THAT IS WRONG. The gate's honest SKIP-with-reason exists for the PYTHON ARM's degradations, NOT for extensions absent from the ext list. An unenumerated extension is a SILENT false-null, not an honest skip. The distinction is load-bearing and I stated it backwards.

NOTE ON LINE NUMBERS: the BOB-205 stream cited the ext list at :318; the conductor measured it at :457. Both are correct at their read times — a sibling stream (BOB-195 r7) is actively editing this file. Re-derive before acting.

ACCEPTANCE: (1) either shell is added to the ext list WITH an arm that can actually analyse it, or unenumerated extensions produce an explicit UNANALYSED verdict per file — never a silent pass (this is the same analyser-registry fix BOB-191 needs, and doing it once covers both); (2) the driver's summary reports files ANALYSED, not files present, so an under-count cannot masquerade as a census; (3) a RED planting a shell fail-open under scripts/ and asserting it is SEEN; (4) golden-FALSE per §11.4.201(1) — a correctly fail-CLOSED shell guard must NOT be flagged; §11.4.67's own brace-scoped exec form is the natural fixture.

COMPOSES: BOB-191 (matcher hole — same primitive: classify by local shape, never consult semantic role; and the analyser-registry fix closes both) and BOB-205 (scope hole). Per §11.4.250 these are three symptoms of one primitive defect; per §11.4.214 they stay three items because the fixes differ.

DISCOVERY CHANNEL (§11.4.238): found by the BOB-205 verification stream, confirmed independently by the conductor. Not by the automated QA regime.

BOB-214 — REPOSITORY ROOT is in no DANGER_ROOT: webui-bridge.py (live HTTP service, :7188) carries a REAL fail-open the gate reports on sight, and start.sh + the credential scripts are unscanned

Status: Queued **Type:** Bug **Severity:** critical

WHAT: DANGER_ROOTS = (download-proxy/src plugins scripts qBitTorrent-go frontend/src) at scripts/pre_build_verification.sh:1511. The REPOSITORY ROOT itself is not among them (conductor-verified: no "." entry). Everything living at the top level is scanned by no arm.

THE LIVE HIT: webui-bridge.py — tracked at root (conductor-verified), 466 LOC, a live HTTP service on port 7188 (BaseHTTPRequestHandler:85, do_GET/do_POST:92-97, request path read at :103, outbound urlopen at :273, environment credentials read at :55-74). The gate REPORTS A REAL HIT AT :295 when pointed at it directly. This is not a hypothetical gap: the existing gate, unmodified, finds a genuine defect in this file the moment scope reaches it.

ALSO UNSCANNED AT ROOT: 13 first-party shell scripts including start.sh (1288 LOC — per CLAUDE.md the project's orchestrator and the sole sanctioned container-control entry point), stop.sh, ci.sh, install-plugin.sh, and the credential-handling init-qbit-password.sh / fix-qbit-password.sh. (These are additionally invisible for the separate LANGUAGE-hole reason — shell is not in the gate's ext list — so root files get missed twice over, by scope AND by extension.)

THE SCALE THIS SITS IN: 266 files in scope / 512 out of scope — 66% of the gate-visible first-party corpus is never scanned. Other unscanned roots measured: tests/ (324 files, excluded-by-intent but UNDECLARED — an undeclared exclusion is exactly what §11.4.224(E) fences against), extension/ (108, shipped browser extension), docs/ (51), challenges/ (18), tools/ (1, and it CONCEALS 3 REAL HITS at plugin_update_automation.py:189,198,215), frontend/e2e + 2 configs (5).

SO THE SCOPE HOLE CONCEALS AT LEAST 4 REAL HITS the gate itself finds when pointed at them (1 in webui-bridge.py, 3 in tools/). Invariant 39's reported count is an under-count, not a census.

ACCEPTANCE: (1) repository root and tools/ are scanned, or explicitly fenced with a stated reason per §11.4.224(E) — silence is not an exclusion; (2) the 4 concealed hits are triaged (each is either a real defect to fix or a false positive to fix in the gate — both are findings); (3) tests/ (324 files) gets an OPERATOR decision per §11.4.66 — production-only is defensible but must be DECLARED; (4) a RED proving a root-level fail-open is seen; (5) golden-FALSE proving genuine build artefacts and vendored trees stay excluded.

FIX DIRECTION (measured by the BOB-205 stream, not guessed): a §11.4.251 manifest is feasible but the source of truth must be chosen carefully — docker-compose.yml build contexts MISS plugins/, frontend/src, cmd/boba-ctl and webui-bridge.py; language markers (go.mod/package.json) MISS plugins/ and webui-bridge.py. Only derive-from-git-tracked-source-extensions minus a declared §11.4.224(E) exclusion fence reaches every gap. Note the derivation must be built either way — if the operator prefers keeping a hand list, the omission-guard that audits it is the SAME computation; the only question is whether it drives the scan or audits the list.

DISCOVERY CHANNEL (§11.4.238): found by the BOB-205 verification stream while enumerating the full gap list — the item it was verifying named only one missing root. Not by the automated QA regime.

BOB-215 — boba-ctl authMethod() default branch silently resolves a typo'd or empty auth: key to SSH-key authentication instead of refusing

Status: Queued **Type:** Bug **Severity:** major

WHAT: cmd/boba-ctl/main.go:498-509, authMethod(). Its default branch (:507) returns remote.AuthSSHKey. A deploy-registry entry whose auth: key is misspelled, empty, or set to an unrecognised value therefore resolves SILENTLY to SSH-key authentication rather than refusing to act.

WHY IT IS A §11.4.252 VIOLATION: this is a credential-selection decision on a path that also performs remote mutation and external side effects. §11.4.252 requires a path combining >=2 dangerous capabilities to FAIL CLOSED — verify every precondition, refuse when any is unverifiable, and name the unresolved precondition. Silently substituting a default credential MECHANISM for an unreadable declaration is the textbook fail-open shape: the operator's intent was not determined, and the code proceeded anyway using a method they may not have chosen.

WHY NO GATE CAUGHT IT: three independent reasons, each sufficient on its own — (a) cmd/boba-ctl is in no DANGER_ROOT (BOB-205); (b) even in scope, .go files are structurally unanalysable by the current gate (BOB-191); (c) this shape is a semantic default-branch decision, not one of the two text patterns the non-Python arms match. It was found by a human-equivalent read, which is precisely the §11.4.238 escape class.

HONEST QUALIFICATION (§11.4.6): cmd/boba-ctl/main.go has NO `_ = err`, NO empty `if err != nil {}`, and NO silent `return nil` today — grep-verified with a control needle against qBitTorrent-go/internal/config/config.go. This finding is the exception, not one of many; the scope hole around boba-ctl is otherwise LATENT (it hides nothing else live, it guarantees a future one lands unseen).

ACCEPTANCE: (1) an unrecognised/empty auth: value REFUSES with an error naming the offending key and its declared value, never silently defaults; (2) a RED driving a typo'd auth: and asserting today's silent SSH-key resolution, flipping to refusal post-fix; (3) golden-FALSE proving every LEGITIMATE auth: value still resolves correctly — a false refusal here would break real deploys (§11.4.201(1)).

DISCOVERY CHANNEL (§11.4.238): found by the BOB-205 verification stream while confirming boba-ctl qualifies as a §11.4.252 surface.

BOB-216 — MODE-INDEPENDENT carrier classes: the fail-closed gate's PRIMARY (AST) mode reports comments and docstrings as live defects — the parser-is-immune claim was over-broad

Status: Queued **Type:** Bug **Severity:** major

WHAT: three carrier classes fire in the gate's PRIMARY mode, not only its degraded text fallback. Measured ast_rc=1 AND text_rc=1 (both modes report the hit): (a) a COMMENT or DOCSTRING quoting the credential anti-pattern is reported as a LIVE credential default — both spellings; (b) a // comment or a string constant holding `try { x(); } catch (e) { }` is reported as an empty catch, once per carrier line. Shape (b), and the C-family half of shape (a), are LANGUAGE-AGNOSTIC GREPS with NO structural counterpart in any mode — so there is no parser to be immune.

WHY IT MATTERS: the gate's header carried a blanket claim that "A parser is immune to both by construction". That is OVER-BROAD and now corrected in place, scoped to the Python Try/With shapes where it is actually true. Everything else — every non-Python extension, and the credential/empty-catch text matchers even on Python — has no AST arm at all, so the immunity never applied there. A §11.4.201(1) false refusal in the PRIMARY mode is materially worse than one in a fallback nobody expects to be exact: it refuses provably-healthy code on the path the gate is trusted on.

THE CONTROL NEEDLE THAT SHARPENS IT (recorded so the next reader inherits the measurement): a #-COMMENTED import does NOT poison the licensing table (text=0), because those regexes are line-anchored. So the blindness is to STRINGS specifically, not to non-code generally. That distinction is what makes the comment-strip fix tractable for one class and not for the others.

WHY IT WAS NOT FIXED IN THE ROUND THAT FOUND IT (§11.4.6 honest boundary, and this is the right call): closing these needs a PER-LANGUAGE comment-and-string model across every configured extension. That model's failure direction is the UNDER-reporting one — a mis-parsed string region silently swallows real violations for the remainder of the file. The same reasoning made the round DECLINE the triple-quote fence counter for OVER-1 while ACCEPTING the comment strip for OVER-A: the comment strip's ambiguity resolves toward KEEPING text (proven by three control needles — real violation + comment, # inside a string, escaped quote before # — all 1/1), whereas a fence counter's ambiguity resolves toward DROPPING text. Direction of failure, not difficulty, is the discriminator. Building a full lexer here would import the under-reporting failure mode this gate refuses everywhere else.

OPERATOR DECISION (§11.4.66 / §11.4.197): whether to build per-language comment-and-string models is a consumer call, not a default this gate should pick unilaterally. Options: (a) accept the over-reports and DISCLOSE them per class (what the round did — a new MODE-INDEPENDENT CARRIERS section now enumerates all three); (b) build the models per language and own the under-report risk with a fixture per language; (c) narrow the language-agnostic greps so they only fire where a structural arm exists, trading coverage for precision.

RELATION TO SIBLINGS (§11.4.214 distinct-but-similar, deliberately not merged): BOB-189 is the gate flagging a fail-CLOSED SSRF guard — a semantic-role miss in the PYTHON arm. This is a CARRIER miss (comment/string read as code) that is MODE-INDEPENDENT. BOB-213 is a language absent from the ext list entirely. All three share the primitive §11.4.250 named in BOB-191: classify by local syntactic shape, never consult semantic role — but the fixes differ, so the items stay separate.

ACCEPTANCE: (1) the operator decision above is taken and recorded; (2) whichever path is chosen, each of the three classes has a fixture proving current behaviour, so a future change cannot silently alter it; (3) if (b) is chosen, a golden-FALSE per language proving a REAL violation adjacent to a carrier is still caught — the under-report guard, which is the whole risk.

DISCOVERY CHANNEL (§11.4.238): found by the BOB-195 round-7 remediation stream while closing a different finding, NOT by the automated QA regime and NOT by the independent reviewer that audited the same file in round 6.

BOB-217 — plugin_update_automation downloads executable plugin code from third-party personal GitHub repos and gates it with a SYNTAX check only — no signature, no pinned commit, no hash allowlist

Status: Queued **Type:** Bug **Severity:** critical

WHAT: tools/plugin_update_automation.py update_plugin() combines three §11.4.252 dangerous capabilities and is gated by nothing that could stop a hostile payload: - UNTRUSTED INPUT: 14 URLs across four GitHub repositories, THREE of which are third-party PERSONAL repos, not the official qbittorrent organisation. - MUTATION: writes plugins/.py. - DEFERRED CODE EXECUTION: qBittorrent EXECUTES those engine files. The bytes fetched over the network become running code on the operator's host. The ONLY validation is compile(content, "", "exec") at tools/plugin_update_automation.py:213 — conductor-verified as the sole compile call. That is a SYNTAX check. A syntactically valid file is exactly what a hostile payload is. There is NO signature verification, NO pinned commit SHA, NO hash allowlist, NO provenance check of any kind.

§11.4.252 requires a path combining >=2 dangerous capabilities to FAIL CLOSED — verify every precondition, refuse when any is unverifiable. This path combines THREE and verifies none of them. §11.4.246's supply-chain clause is the direct counterpart: dependencies are either vendored hash-verified, provenance-attested, or mirrored through a hash-pinning registry — an unattested public source is an integrity risk, and here the unattested public source becomes EXECUTED CODE.

THE SHARPEST FACT ABOUT THIS FINDING: the fail-closed gate produced THREE FALSE POSITIVES in this same file (:189, :198, :215 — all triaged FALSE POSITIVE, see BOB-214's correction) while MISSING this, the one genuine dangerous combination in it. That is the §11.4.201 both-directions failure — false-positive and false-negative — demonstrated inside a single file. It is the strongest available evidence that the detector classifies by local syntactic shape and never consults semantic role (the §11.4.250 primitive named in BOB-191/BOB-216).

REACHABILITY, STATED HONESTLY (§11.4.6): the script is invoked by NOTHING — control-needle-proven (91 needle hits, 0 negative control) it is referenced only by its own README, its own docstring, and a tracker note; zero hits across *.sh* / .yml / *.py / Makefile outside tools/. Its own README self-declares "not invoked by the normal start/stop flow". Last functional commit 2026-04-12. So this is NOT live in any automated path. It IS reachable by a hand-run –update, which is exactly what the tool exists for. Severity is Critical on the capability combination, bounded by that reachability — not on an active exploitation path.

ACCEPTANCE: (1) plugin sources are pinned (commit SHA or content hash) and verified before write; (2) an unverifiable source REFUSES and names which check failed (§11.4.252) rather than proceeding on a syntax pass; (3) the compile() call is documented in-source as a syntax check that is NOT a safety gate, so nobody reads it as one; (4) a RED serving a syntactically-valid hostile payload and asserting it is REFUSED pre-fix-absent / post-fix-present; (5) golden-FALSE proving a legitimate pinned update still succeeds (§11.4.201(1)).

DISCOVERY CHANNEL (§11.4.238): found by the concealed-hits triage stream while establishing that the gate's four hits in this area were false positives. Not by the automated QA regime — and the regime that WAS pointed here reported the wrong three lines.

BOB-218 — tools/README documents a rollback that does not exist: the plugin writer truncates on open and never restores the .bak, leaving a corrupt plugin while reporting the update FAILED

Status: Queued **Type:** Bug **Severity:** major

WHAT: tools/README.md:30 states “the backup is restored” on validation failure. Conductor-verified: shutil.copy2 appears EXACTLY ONCE in tools/plugin_update_automation.py and copies FORWARD only — a reverse-direction restore has ZERO occurrences (grep control-needle-proven seeing: needle 2 hits, negative control 0). The plugin write opens in mode “w”, which TRUNCATES IMMEDIATELY.

USER-OBSERVABLE HARM (the reason this is not merely a doc defect): a failure part-way through the write leaves a TRUNCATED, NON-IMPORTABLE plugin on disk. The operator is told the update FAILED and reasonably concludes the previous file survived — because the README says it was restored. It was not. The next ./install-plugin.sh copies the corrupt file into config/qBittorrent/nova3/engines/ and that search engine SILENTLY STOPS WORKING. The .bak needed to recover DOES exist on disk; the tool never mentions it and never uses it.

This is a §11.4 documentation-layer bluff with a real downstream consequence: the doc asserts a safety property the code does not implement, and the operator’s recovery decision is made on that false assertion.

SECOND, SMALLER DOC DIVERGENCE: the README claims JSON goes to stdout; :274 writes it to a FILE (open(args.output, “w”)).

THIRD, SEPARATE MINOR DEFECT in the same file (recorded here rather than as its own item because it shares the file and the fix window): _extract_version at ~:198 uses a BARE `except:`, so a Ctrl-C during the 14-URL sweep is SWALLOWED. That is a signal-handling defect, not a fail-

open — the gate flagged this line, but for the wrong reason (it read the silent default return; the real issue is the bare except catching KeyboardInterrupt).

ACCEPTANCE: (1) either the rollback is IMPLEMENTED (restore the .bak on failure) or the README claim is DELETED — the doc and the code must agree (§11.4.6); if implemented, write to a temp file and rename atomically rather than truncating in place; (2) the stdout-vs-file claim corrected; (3) the bare except narrowed so KeyboardInterrupt propagates; (4) a RED that fails the write mid-way and asserts the previous plugin is intact (post-fix) / truncated (pre-fix).

REACHABILITY: same as the sibling item — hand-run only, not in any automated path, last functional commit 2026-04-12. Severity Major on the harm shape, bounded by that reachability.

DISCOVERY CHANNEL (§11.4.238): found by the concealed-hits triage stream. Not by the automated QA regime.

BOB-219 — LIVE §11.4.65 sync violation: docs/guides/tracker-credentials.{html,pdf} exist on disk but are untracked and ignored, while their .md source IS tracked

Status: Queued **Type:** Bug **Severity:** major

WHAT, conductor-verified: docs/guides/tracker-credentials.md is TRACKED. Its §11.4.65-mandated export twins docs/guides/tracker-credentials.html and .pdf both EXIST ON DISK, are NOT tracked, and ARE ignored — matched by the .gitignore *credentials* deny-all at :31. The .md is rescued by an explicit allowlist entry at :38; nobody added entries for the twins.

WHY IT IS A LIVE VIOLATION AND NOT A HYPOTHETICAL: §11.4.65 requires every in-scope Markdown document to carry synchronized .html/.pdf siblings, and §11.4.212 requires every §11.4.65-scope doc to be reachable from the README. Two artifacts that exist locally but can never be

committed are, from any fresh clone, ABSENT — so every clone of this repository has a tracker-credentials guide with no exports, while the authoring host looks complete. The divergence is invisible on the machine that would notice it.

WHY NOBODY SAW IT: this is a §11.4.201(6) FALSE-NULL of the BOB-212 class. git status never listed the twins, so no gate and no author had a signal. The blind instrument and the clean tree return the identical quiet zero.

ANOTHER INSTANCE OF THE SAME MECHANISM, filed here rather than separately because it is one fix: docs/qa/BOB-124/evidence/loginctl_user_state.txt is swallowed by *_user* at .gitignore:~55 and is therefore uncommitted §11.4.83 QA evidence. Its SIBLINGS IN THE SAME DIRECTORY — user1000_grepped.log, user_scope_events_head.log, user_service_lifecycle.log — ARE tracked, saved only by the robust glob negation !docs/qa/**/*.log. That one file is lost purely because it is .txt rather than .log. The pattern is exact: robust glob negations hold; per-file and per-extension rescues leak.

ACCEPTANCE: (1) both twins become trackable and are committed alongside their .md; (2) the BOB-124 .txt evidence likewise; (3) whichever BOB-212 fix direction is chosen, it MUST cover these — a fix that closes the future-swallow while leaving these three artifacts permanently uncommittable has addressed the mechanism and not the damage; (4) a check that a tracked .md in §11.4.65 scope has COMMITTABLE twins, so this class cannot recur silently.

DISCOVERY CHANNEL (§11.4.238): found by the BOB-212 blast-radius sweep — and only on its SECOND pass. The first pass filtered by a source-extension set that omitted .pdf, which hid this finding entirely; the agent re-ran with no extension filter over all 420 non-artifact paths and recorded the blind spot rather than shipping the first number. Worth keeping: an extension allowlist is itself a false-null generator, which is the same shape as the defect being investigated.

BOB-220 — ownership_repair setuid/setgid strip is a NO-OP on directories: GNU chmod 755 preserves setgid, so the in-source comment claims a strip that does not happen

Status: Queued **Type:** Bug **Severity:** minor

WHAT: scripts/ownership_repair.sh:891-895 runs chmod 755 intending to strip setuid/setgid bits. Traced live: the chmod RUNS and SUCCEEDS, yet the mode stays 2755. Root cause measured — GNU chmod with a 3-DIGIT symbolic-equivalent octal PRESERVES the setgid bit on DIRECTORIES; only a 4-digit form (00755) or an explicit g-s clears it. The comment at :868-884 describes a strip that does not occur for directories.

EFFECT: benign today — the surviving setgid bit is not itself harmful, and no defect is known to follow from it. This is filed as a §11.4.6 accuracy defect: an in-source comment asserting behaviour the code does not perform. That matters because the next reader will trust it, and because a future security-relevant strip written against the same pattern would silently fail the same way.

RELATION TO BOB-207: the same stream measured that after repair a new file inherits gid 1000 with THE SETGID BIT STILL SET — the two facts compound. If BOB-207 is resolved by narrowing the repair (the recommended direction), the setgid survival becomes moot for that path but the misleading comment remains.

ACCEPTANCE: (1) either the strip is made real (00755 or g-s) or the comment is corrected to state that directories retain setgid — code and comment must agree; (2) if made real, a RED asserting mode 2755 -> 0755 on a directory, since the current form silently no-ops; (3) golden-FALSE proving a directory that SHOULD keep its mode is not gratuitously changed.

DISCOVERY CHANNEL (§11.4.238): found by the BOB-207 stream while tracing an unrelated behaviour. Not by the automated QA regime.

BOB-221 — Invariant 30 has no RED-test allowance: a correctly-authored failing RED (mandated by §11.4.115/ §11.4.224) makes the blocking gate FAIL — the constitution's own test-first discipline is gated against itself

Status: Queued **Type:** Bug **Severity:** critical

WHAT: scripts/pre_build_verification.sh invariant 30 (CM-BASH-UNIT-TESTS-EXECUTED) enumerates suites by FILESYSTEM GLOB (:1196) and calls fail() on any non-zero exit. It therefore RUNS untracked suites, and it has NO concept of a test that is SUPPOSED to fail.

MEASURED: tests/pre_build/ test_bob205_danger_roots_scope.sh exits 1 — correctly, because it is a RED test for BOB-205, an unfixed defect. Invariant 30 counts that as a gate failure. The predicted T042 verdict is FAIL, and this is one of its three blocking causes.

THE STRUCTURAL PROBLEM: §11.4.115 and §11.4.224 REQUIRE a RED authored FIRST and OBSERVED TO FAIL before the fix exists. §11.4.135 requires the RED to persist as the permanent regression guard. So the discipline mandates that failing tests exist in the tree between authoring and fixing — and the blocking gate treats their existence as a defect. Following the constitution correctly makes the gate refuse the commit. That is a §11.4.120 wrong-seam problem: the gate asserts "no suite fails" when the invariant it should assert is "no suite fails UNEXPECTEDLY".

WHY IT SURFACED NOW: this session authored FOUR REDs across parallel streams (BOB-204, BOB-205, BOB-207, BOB-212) — the first time the discipline was applied at this volume. Previously REDs were flipped GREEN within the same round, so the window never spanned a gate run. The defect is not new; the exposure is.

THE FORBIDDEN RESPONSES (§11.4.120): do NOT delete the RED to make the gate green; do NOT weaken invariant 30 to ignore failures; do NOT mark the RED skipped without a tracked reason. Each converts a real signal into silence.

FIX DIRECTION — this is §11.4.248's quarantine mechanism, which the constitution already specifies and this project has not wired: a RED declares its expected-failing status and its tracked item (a header marker, a naming convention, or a manifest), invariant 30 reads that declaration, and a declared-RED failure is reported as EXPECTED (not a fail) while an UNDECLARED failure still blocks. Crucially the declaration must EXPIRE or be tracked — §11.4.248 pairs quarantine with a stabilisation deadline precisely so "expected to fail" cannot become permanent cover. A RED whose item closes must flip GREEN or the gate should FAIL on the stale declaration.

ACCEPTANCE: (1) a declared RED does not block; (2) an UNDECLARED failing suite still blocks (the §11.4.201(1) guard — a fix that ignores all failures is strictly worse than the defect); (3) a declared RED whose tracked item is CLOSED blocks, so declarations cannot rot; (4) a paired §1.1 mutation removing the declaration-check makes the gate accept an undeclared failure -> gate FAILs.

DISCOVERY CHANNEL (§11.4.238): found by the T042 readiness preflight, before the endgame rather than during it. Not by the automated QA regime — the regime IS the thing that would have blocked.

BOB-222 — tests/security/ is executed by no invariant — the third occurrence of the same orphan-directory class, and the driver's own comment records the previous two

Status: Queued **Type:** Bug **Severity:** major

WHAT: scripts/pre_build_verification.sh:1196 enumerates bash suites with a glob covering EXACTLY three directories: "${PROJECT_ROOT}"/tests/unit/test_*.sh · "${PROJECT_ROOT}"/tests/pre_build/test_*.sh · "${PROJECT_ROOT}"/tests/hooks/test_*.sh tests/security/ is NOT among them. Conductor-verified verbatim. So tests/security/test_gitignore_swallow_is_loud.sh — a RED authored this

session with a golden-FALSE guard proving real secrets stay ignored — is executed by NOTHING. It is a guard that guards nothing, and its silence is indistinguishable from success.

THIS IS THE THIRD OCCURRENCE OF ONE CLASS. The driver's OWN comment at :1030 records the history: the same defect class "existed for tests/pre_build/test_*.sh" and was fixed by adding that directory to the glob. It then recurred one directory over (tests/hooks), fixed the same way. Now tests/security. Each fix was a new glob entry; none addressed why a new test directory is invisible by default.

§11.4.250 APPLIES DIRECTLY: three symptoms, one primitive. The primitive is a hand-maintained enumeration that must be edited in lockstep with the tree, whose failure mode is SILENT. It is the identical shape as BOB-205's DANGER_ROOTS and BOB-212's .gitignore allowlist — three separate hand-maintained lists in this repo, all three measured to have silently missed something. Adding tests/security to the glob would be the fourth instance of layer N+1.

NOTE THE GATE ALREADY HAS THE RIGHT INSTINCT at :1220: it fails when the glob matches NOTHING ("the glob is blind"). That guard catches a TOTALLY blind glob but not a PARTIALLY blind one — the more common and more dangerous case, because a partially blind glob still reports a healthy count.

FIX DIRECTION (§11.4.251 role-as-data-pack): derive the suite set from the tree — every tests/**/test_*.sh — minus a DECLARED §11.4.224(E) exclusion fence with a justification per entry. Then a new test directory cannot be invisible, and a deliberate exclusion is visible and reasoned. If a hand list is kept, the omission-guard is the same computation, so the derivation must be built either way.

ACCEPTANCE: (1) tests/security suites execute; (2) a NEWLY CREATED tests//test_x.sh is picked up with no hand edit — that is the invariant that stops the recurrence, and without it this is just the fourth patch; (3) a RED creating such a directory and asserting it runs; (4) the :1220 blind-glob guard is extended to catch PARTIAL blindness, not only total.

DISCOVERY CHANNEL (§11.4.238): found by the T042 readiness preflight. Not by the automated QA regime — and notably not by the two previous fixes of this same class, neither of which asked why it happened.

BOB-223 — §11.4.18 script-documentation and §11.4.44 revision headers are ungated — and the perverse consequence is that WRITING the mandated companion doc is what breaks the build

Status: Queued **Type:** Bug **Severity:** major

WHAT, both conductor-verified with control needles: CM-SCRIPT-DOCS-SYNC occurrences in scripts/pre_build_verification.sh = 0 (control needle CM-MARKDOWN-EXPORT-SYNC = 6, so the instrument sees). §11.4.44 revision-header enforcement likewise 0 hits (control needle 3). Neither §11.4.18's companion-doc mandate nor §11.4.44's revision header is enforced by any invariant.

THE PERVERSE CONSEQUENCE, MEASURED: a new script with NO companion doc passes the gate. Writing the companion doc that §11.4.18 MANDATES creates docs/scripts/.md, which invariant 16 (CM-MARKDOWN-EXPORT-SYNC) then requires to have .html and .pdf twins — and their absence is a BLOCKING failure. This session hit it exactly: docs/scripts/test_gitignore_swallow_is_loud.{html,pdf} are missing and are two of invariant 16's six violations. So the gate's incentive gradient points AWAY from the constitution: complying with §11.4.18 is punished, ignoring it is free. That is worse than an ungated rule — it is a rule the machinery actively discourages.

§11.4.227 IS THE GOVERNING ANCHOR: a named gate that exists only in prose is gate debt, and the ledger of unimplemented CM-* names must be monotone-decreasing. CM-SCRIPT-DOCS-SYNC is named in §11.4.18's own text and

implemented nowhere — one concrete row of exactly the 58%-unimplemented population §11.4.227 was minted to shrink.

ACCEPTANCE: (1) either CM-SCRIPT-DOCS-SYNC is implemented, or it is registered as a deferral pointing at a tracked §11.4.197 item — silent absence is what §11.4.227 forbids; (2) same for §11.4.44's header check; (3) whichever is chosen, the incentive inversion is closed: it must never be cheaper to skip a mandated doc than to write one — if invariant 16 will demand twins, the doc-creation path must produce them, or invariant 16 must scope out docs/scripts until it can; (4) a paired §1.1 mutation per gate implemented.

HONEST NOTE ON SCOPE: this item does not argue §11.4.18 should be enforced immediately — that is an operator call about gate debt priority (§11.4.66). It argues the CURRENT state is incoherent: an unenforced mandate whose observance triggers a different gate's failure. Either enforce both ends or neither.

DISCOVERY CHANNEL (§11.4.238): found by the T042 readiness preflight while explaining why a new file broke invariant 16. Not by the automated QA regime.

BOB-224 — tests/unit/test_compute_badges_carrier_match.sh HANGS to the full 300s timeout (rc=124) and blocks invariant 30 independently of any RED-test question

Status: Queued **Type:** Bug **Severity:** critical

WHAT: a replication of invariant 30 (same glob, same skip list, same BOBA_PREBUILD_NESTED=1, same timeout 300) measured RAN=38 FAILED=4 SKIPPED=2. One of the four failures is tests/unit/test_compute_badges_carrier_match.sh exiting **rc=124 after the FULL 300 seconds** — it does not fail, it HANGS.

WHY IT IS FILED SEPARATELY AND URGENTLY: it blocks invariant 30, and therefore T042, INDEPENDENTLY of BOB-221. The assumption in BOB-221 that landing an expected-RED mechanism unblocks T042 is FALSE as stated — two of the four failures are REDs, one is this hang, one is contaminated. Landing the mechanism alone leaves T042 blocked by this suite.

IT MUST NOT BE SILENCED BY A DECLARATION. A hang is a §11.4.232(C) liveness failure, not a verdict: a wedged op and a progressing op both look like “not finished yet”, and marking it expected-to-fail would convert a §11.4.201(6) false-null into permanent cover. That is exactly the abuse the expected-RED design property (2) exists to prevent, so it must be triaged as its own defect.

INVESTIGATION DIRECTION (§11.4.102 first, no guessing): rc=124 is the timeout(1) signature. Determine WHERE it wedges — a consumer blocked on an unclosed producer write-end is the documented shape here (§11.4.201(12) records that a background watchdog spawned inside a $(…) command-substitution inherits the pipe write-end, and an early disarm orphans its sleep grandchild, so the substitution stalls for the FULL budget while every verdict and exit code stays CORRECT). That signature — full budget, correct verdicts — matches rc=124 exactly and should be the FIRST hypothesis tested, not the last. The countermeasure is documented: redirect the watchdog subshell fds away from the cmd-subst pipe, or have the probe write to a file. Do NOT assume that is the cause; it is the highest-prior hypothesis given the recorded precedent.

ACCEPTANCE: (1) the wedge point is identified with captured evidence, not inferred; (2) the suite completes deterministically well inside the budget; (3) a RED reproducing the hang, so a regression cannot silently re-wedge (a timeout-based assertion, since the failure IS the duration); (4) NOT closed by a declaration or by raising the timeout — raising the budget hides it.

FILE DATE: the suite is dated 2026-08-21, so the hang predates this session.

DISCOVERY CHANNEL (§11.4.238): found by the BOB-221 design stream while surveying invariant 30 more widely than its brief required — the T042 preflight had verified only 3 of 38 suites and stated that boundary honestly, which is what prompted the wider survey.

BOB-225 — *.docx is globally gitignored while the §11.4.65 exporters generate .docx twins — every DOCX artifact this project produces is untrackable by construction

Status: Queued **Type:** Bug **Severity:** major

WHAT, conductor-verified: .gitignore:266 ignores *.docx globally. The §11.4.65 export pipeline GENERATES .docx twins — the workable-items export produces Issues.docx / Fixed.docx / Issues_Summary.docx / Fixed_Summary.docx, and the doc exporter produced docs/scripts/check_cm_lan_routes_authenticated.docx during round 9. Every one of them is untrackable by construction: they exist on disk, git will never see them, and no gate can notice because the absence is silent.

WHY THIS IS THE BOB-212 CLASS, NOT A DUPLICATE OF IT: BOB-212 is a deny-all glob plus a hand-maintained per-file ALLOWLIST, where new files leak through the gaps in the list. This is a deny-all glob with NO allowlist at all for a file type the project MANDATES producing. The mechanism differs; the false-null is identical — git status stays silent, so the exporter appears to succeed and the artifact appears to exist. It is filed separately per §11.4.214 (distinct-but-similar), with BOB-212 and BOB-219 as siblings.

THE TENSION TO RESOLVE HONESTLY: §11.4.153 mandates a FOUR-format export (HTML + PDF + DOCX) for its document class, and §11.4.65 governs the twins generally. So the constitution requires producing an artifact the repository is configured to refuse. One of the two is wrong and the resolution is an operator decision (§11.4.66): (a)

the .docx mandate applies here and the glob must carve out generated doc twins; (b) .docx is deliberately untracked as a heavy binary derivative regenerable per §11.4.77 from its .md, in which case the EXPORTERS should stop producing it, or produce it into an explicitly untracked location, and the §11.4.153 four-format requirement should be recorded as consciously not-adopted rather than silently unmet. What is NOT acceptable is the present state: generate it, ignore it, and let both the mandate and the glob appear satisfied.

MEASURED SCOPE: tracked .docx files = 0. So this is not a partial condition — no DOCX artifact has ever been committed, and the project has been producing them into a void.

ACCEPTANCE: (1) the operator decision above is taken and recorded; (2) whichever way, the exporter and the ignore rule AGREE — if .docx is not tracked, nothing should silently generate one into a tracked doc directory; (3) if carved out, a check that a generated twin is actually trackable, so this cannot recur silently; (4) §11.4.153 compliance is either met or recorded as an honest gap, never left implicitly failing.

DISCOVERY CHANNEL (§11.4.238): found by the BOB-102 round-9 remediation stream when it regenerated its own guide twins and noticed the .docx could not be added. Not by the automated QA regime — and the regime cannot see it, which is the point.

BOB-226 — Repair-side walk over foreign-owned INTERIOR directories is untested by any automated path

Status: Queued **Type:** Task **Severity:** major **Created-By:** Claude **Assigned-To:** Claude

WHAT: ownership_repair walks a declared root and repairs items whose uid is not the operator. The unit suite seeds a foreign uid only onto FILES and SYMLINKS; interior DIRECTORIES stay operator-owned (case 22 seeds only a foreign declared ROOT, on the failure path). Production first-start repairs exactly the untested shape. MANIFEST: tests/unit/test_ownership_repair.sh seed_tree/seed_wrong; scripts/

ownership_repair.sh walk at :994. REPRO: seed a tree whose interior directories carry uid 100000 via podman unshare, run the repair, observe no automated assertion covers the outcome. WHY UNTESTED: the unprivileged harness cannot create symlinks inside a directory it no longer owns, which cases 8/9/18 require. The DECLARED GAPS note cross-references tests/ownership/ test_container_writes_owned_files.py, but that covers the CREATION side (FR-002), not the repair-side walk. ACCEPTANCE: an integration-layer test (where the unprivileged-harness constraint does not bind) that seeds foreign-owned interior directories, runs the repair, and asserts post-state ownership plus mode preservation. Surfaced by the BOB-207 independent review 2026-08-27.

BOB-227 — The LAN-route auth gate ships UNTRACKED: analyzer, wrapper and its 197-assertion harness exist only in one working tree

Status: Queued **Type:** Bug **Severity:** critical **Created-By:** Claude **Assigned-To:** Claude

WHAT: three of the four artifacts of the CM-LAN-ROUTES-AUTHENTICATED pre-build gate are untracked in git. MEASURED 2026-08-27 with git ls-files –error-unmatch, control-needled against a known-tracked file: scripts/ pre_build/lan_route_auth_analyzer.py UNTRACKED, scripts/ pre_build/check_cm_lan_routes_authenticated.sh UNTRACKED, tests/pre_build/ test_check_cm_lan_routes_authenticated.sh UNTRACKED; only docs/scripts/check_cm_lan_routes_authenticated.md is TRACKED. IMPACT: (1) losing this checkout loses an entire security gate plus 197 assertions; (2) no round-over-round diff claim across review rounds 8 through 14 was ever checkable, because no committed baseline exists - the same §11.4.226 evidence-custody failure the BOB-195 chain hit independently; (3) a fresh clone runs a pre-build gate whose implementation is absent. ACCEPTANCE: all four artifacts tracked and committed, and a gate asserting that every

executable a pre-build invariant invokes is itself tracked. Surfaced by the BOB-102 round-13 remediation and independently verified 2026-08-27.

BOB-228 — README does not link the LAN-route auth gate guide, so a §11.4.65-scope doc is an orphan under §11.4.212

Status: Queued **Type:** Task **Severity:** minor **Created-By:** Claude **Assigned-To:** Claude

WHAT: §11.4.212 makes the main README the canonical entry point for ALL project documentation, with no §11.4.65-scope doc reachable by no link path. docs/scripts/check_cm_lan_routes_authenticated.md is not reachable from README.md. MEASURED 2026-08-27: grep -c for the guide name in README.md returns 0, control-needled against a doc README does link (CONTINUATION returns 1), so the instrument is not blind. ACCEPTANCE: README links the guide directly or transitively, and the doc-link generator covers docs/scripts/ so the next such guide cannot land orphaned. Surfaced by the BOB-102 round-13 remediation and independently verified 2026-08-27.